

Jarvis Documentation Book

Phase 0 scaffold for a production-grade thesis/defense document.

Document Build Note

This book is built folder-by-folder. Each implementation phase reads one target area, fills one placeholder, verifies the result, and leaves all other sections untouched.

Convention	Phase 0 rule
Figure numbering	Use Figure 4.<folder>.<n> inside Chapter 4 sections.
Table numbering	Use Table 4.<folder>.<n> inside Chapter 4 sections.
Folder template	Purpose; file inventory; data-flow diagram; key modules; algorithms; configuration; behavior table; contribution.
Accuracy	Claims must map to files read in that phase; uncertain details are marked as unverified rather than asserted.
Language scope	Egyptian-Arabic and English behavior are both first-class where the code supports them.

Title Page

Jarvis

A Local-First Bilingual (English / Egyptian Arabic) Windows Voice Assistant

Project Documentation Book - prepared for the Final Discussion

Wake word -> streaming capture + STT -> intent routing cascade -> verified action / local LLM -> streamed TTS

Generated from the Jarvis repository source and runtime data, folder-by-folder.

Prepared by: Abdelrhman Yousef Mahrous

Supervised by: Dr. Ibrahim Mubarak

2026

Acknowledgments

This project builds on a stack of open-source and hosted components documented throughout Chapter 4: openWakeWord and Silero for on-device audio detection, Faster-Whisper and ElevenLabs for bilingual speech recognition, Ollama and the Qwen3 model family for local language understanding and generation, edge-tts for free-tier speech synthesis, sentence-transformers and ChromaDB for semantic routing and memory recall, and Open-Meteo/DuckDuckGo for live data. Their availability as free or self-hostable tools is what makes a fully local-first bilingual assistant practical on ordinary consumer hardware.

Equal acknowledgment goes to the Egyptian Arabic / English bilingual user experience that motivated this project: treating code-switched speech as the normal case, not an edge case, shaped nearly every design decision documented in this book, from the unified wake-word model in Sec 4.4 to the code-switch router in Sec 4.6.

Abstract

Jarvis is a local-first Windows voice assistant built for a bilingual Egyptian Arabic / English user. A single unified wake-word model detects both "Jarvis" and "جارفيس"; a hybrid cloud/local speech-to-text path locks the primary language per utterance while still accepting code-switched words; a seven-tier routing cascade resolves most commands in low single-digit milliseconds and reserves a locally hosted Qwen3 language model (selected automatically for the detected hardware) for anything the faster tiers cannot resolve; every operating-system side effect is verified by re-reading real OS state rather than assumed from an API call's return value; and durable SQLite plus vector memory give the assistant continuity across sessions. This book documents that system for a final defense: Chapter 1 states the problem and objectives, Chapter 2 reviews the prior art each component builds on, Chapter 3 lays out the whole-system methodology and diagrams, Chapter 4 documents every top-level folder of the implementation against the real source and data on disk, and Chapter 5 closes with an honest accounting of what is achieved and what remains as future work.

Reading Guide

Readers preparing for the defense should read Chapters 1 and 3 first for the whole-system picture, then use Chapter 4 as a reference: each subsection (4.1 through 4.14) is self-contained, maps to exactly one repository folder, and can be read in isolation without the others.

Table of Contents

Table of Contents will be updated in Phase 13.

Static contents list (regenerate the live Table of Contents field in Word/LibreOffice with References -> Update Table for page numbers):

Title Page

Acknowledgments

Abstract

Table of Contents

List of Figures

List of Tables

Abbreviations

Chapter 1 Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

1.4 Scope

1.5 Motivation

1.6 Report Organization

Chapter 2 Literature Review

2.1 Commercial Voice Assistants

2.2 Local LLM Runtimes

2.3 Speech-to-Text: Faster-Whisper and ElevenLabs

2.4 Wake-Word Detection and Dataset Practice

2.5 Voice Activity Detection: Silero

2.6 Text-to-Speech: edge-tts and ElevenLabs

2.7 Semantic Routing and Memory

2.8 Positioning

Chapter 3 Methodology

3.1 Design Principles

3.2 Architecture

3.3 Runtime Pipeline

3.4 Routing Cascade

3.5 Memory Scheme

3.6 Models

3.7 Dependencies

3.8 Key Algorithms Overview

Chapter 4 Implementation - Root

4.1 Repository Layout

4.2 Entry Point main.py

4.3 Configuration core/config.py

4.4 Environment and Dependency Files

4.5 The data/ Runtime Tree

4.6 Root-Perspective Startup Sequence

4.2 core/

4.3 data/memory/

4.4 audio/

4.5 wake-word dataset and training

4.6 nlp/

4.7 llm/

4.8 os_control/

4.9 tools/ and 4.10 utils/

4.9 tools/

4.10 utils/

4.11 ui/ and desktop/

4.11.1 ui/

4.11.2 desktop/

4.12 tests/, 4.13 scripts/, and 4.14 models/

4.12 tests/

4.13 scripts/

4.14 models/

Chapter 5 Conclusion & Future Work

5.1 Objectives Achieved

5.2 Limitations

5.3 Future Work

References

Appendices

Appendix A: Source-Code Listing Reference

Appendix B: Configuration Reference

Appendix C: Installation Guide

Appendix D: User Manual

Appendix E: Testing Scenarios

Appendix F: Glossary

List of Figures

List of Figures will be updated in Phase 13.

Static List of Figures (regenerate the live field in Word/LibreOffice for page numbers):

Figure 1.1: Conceptual turn cycle - wake, record, STT, understand, act/answer, speak.

Figure 3.1: Layered architecture from I/O through the optional presentation layer.

Figure 3.2: Runtime pipeline for a single turn, stage by stage.

Figure 3.3: Routing cascade tiers, earliest resolving tier wins.

Figure 3.4: Memory scheme - fast RAM context and LLM context over the same persisted store.

Figure 3.5: Hardware-tier selection for the local LLM model.

Figure 4.1.1: Repository layout and root-level responsibility boundaries.

Figure 4.2.1: Core runtime flow from orchestrator loop through routing, verification, dispatch, memory, and metrics.

Figure 4.3.1: Persisted memory data flow from turn/slot writes to later context reconstruction.

Figure 4.4.1: Audio runtime pipeline from wake detection through STT and TTS.

Figure 4.5.1: Wake-word data artifacts from WAV splits to feature tensors and deployed ONNX boundary.

Figure 4.6.1: NLP flow from normalized text through code-switch, semantic top-k, fuzzy/keyword, and slot extraction stages.

Figure 4.7.1: LLM flow from prompt package through Ollama streaming and sentence-buffered TTS callbacks.

Figure 4.8.1: Verified OS side-effect flow from routed action through policy, confirmation, adapter execution, verification, and audit logging.

Figure 4.9.1: Live data path from query detection through weather/search tools and voice-ready result blocks.

Figure 4.11.1: Optional WebSocket protocol between the Jarvis engine and Tauri/React desktop clients.

Figure 4.12.1: QA and support-asset flow across tests, scripts, models, and the runtime documentation.

List of Tables

List of Tables will be updated in Phase 13.

Static List of Tables (regenerate the live field in Word/LibreOffice for page numbers):

Table 1.1: Project objectives O1-O6 and where this book substantiates each one.

Table 1.2: Reading guide for this document.

Table 3.1: Design principles evidenced across the implementation chapters.

Table 3.2: Models used across the pipeline and where each is loaded.

Table 3.3: core/hardware_detect.py model tiers, copied from _MODEL_TIERS.

Table 3.4: Dependency tiers, copied from requirements.txt/requirements-training.txt.

Table 3.5: Key algorithms and their detailed treatment elsewhere in this book.

Table 4.1.1: Root-level files and folders with their implementation responsibilities.

Table 4.1.2: Representative JARVIS_* configuration keys at the root boundary.

Table 4.1.3: Dependency tiers declared by requirements.txt and requirements-training.txt.

Table 4.1.4: Top-level data/ runtime directories observed in this workspace.

Table 4.1.5: Startup behavior visible from the repository root.

Table 4.2.1: Inventory of every Python file in core/ and core/handlers/.

Table 4.2.2: Core-owned JARVIS_* configuration surface relevant to routing, memory, startup, and diagnostics.

Table 4.2.3: Core behavior under normal, degraded, and edge conditions.

Table 4.3.1: Files observed directly under data/memory/.

Table 4.3.2: SQLite table schema observed in jarvis_memory.db.

Table 4.3.3: SQLite indexes observed in jarvis_memory.db.

Table 4.3.4: Sanitized recent-turn sample from the turns table.

Table 4.3.5: Slot names and value categories observed in the slots table.

Table 4.3.6: Context slot keys observed in jarvis_memory.json.

Table 4.3.7: Memory storage surface visible from persisted artifacts.

Table 4.3.8: Memory data behavior under normal, degraded, and edge situations.

Table 4.4.1: Inventory of every source file in audio/.

Table 4.4.2: Audio configuration surface imported by audio/ modules; defaults are centralized in core/config.py.

Table 4.4.3: Audio behavior under normal, degraded, and edge conditions.

Table 4.5.1: Observed artifacts in wake word data/jarvis_unified_training.

Table 4.5.2: Precomputed feature tensors under features/.

Table 4.5.3: WAV class/split counts in the unified wake-word dataset.

Table 4.5.4: Header-level sample metadata from each WAV split.

Table 4.5.5: Dataset/training configuration surface visible from the artifact folder.

Table 4.5.6: Wake-word dataset behavior under normal, degraded, and edge conditions.

Table 4.6.1: Inventory of Python files observed directly under nlp/.

Table 4.6.2: NLP constants and imported configuration surfaces visible from nlp/.

Table 4.6.3: NLP behavior under normal, degraded, and adversarial conditions.

Table 4.7.1: Inventory of llm/ Python modules and prompt templates.

Table 4.7.2: Prompt template files under llm/prompts/.

Table 4.7.3: Configuration constants imported by llm/ modules; defaults are centralized outside this phase.

Table 4.7.4: LLM behavior under normal, degraded, optional, and adversarial conditions.

Table 4.8.1: Inventory of every Python file directly under os_control/.

Table 4.8.2: os_control/ adapter families.

Table 4.8.3: Configuration symbols imported or referenced by os_control/ modules; defaults are outside this phase boundary.

Table 4.8.4: os_control behavior under normal, degraded, elevation, adversarial, and audit situations.

Table 4.9.1: Inventory of Python files directly under tools/.

Table 4.9.2: Configuration symbols and local timeouts visible from tools/.

Table 4.9.3: tools/ behavior under normal, degraded, and edge conditions.

Table 4.10.1: Inventory of Python files directly under `utils/`.

Table 4.10.2: `utils.language_detector.detect_language()` return contract.

Table 4.11.1: Inventory of Python files directly under `ui/`.

Table 4.11.2: Key desktop/ files read for the Tauri+React overview.

Table 4.11.3: Tauri+React toolchain and window model from desktop key files.

Table 4.11.4: UI and desktop configuration surface observed in this phase.

Table 4.11.5: UI behavior under normal, headless, disconnected, absent, and partial-command conditions.

Table 4.12.1: Inventory of tests/ files and fixture coverage.

Table 4.12.2: Counts derived from tests/fixtures/`nlu_eval_cases.jsonl`.

Table 4.13.1: `scripts/` inventory and developer support roles.

Table 4.14.1: Model files listed under `models/`.

Table 4.14.2: Behavior and QA coverage supplied by tests, scripts, and model assets.

Table 5.1: Objectives O1-O6 against the evidence documented in Chapter 4.

Table 5.2: Candidate future-work items grounded in gaps observed while documenting Chapter 4.

Table R.1: Consolidated references and their role in the system.

Table A.1: Where each area of the source tree is documented in this book.

Table C.1: Installation steps, copied from `README.md`.

Abbreviations

Abbreviation	Meaning
ASR / STT	Automatic Speech Recognition / Speech-to-Text.
TTS	Text-to-Speech.
VAD	Voice Activity Detection.
NLU	Natural Language Understanding.
LLM	Large Language Model.
ONNX	Open Neural Network Exchange, the runtime model format used for wake-word and VAD models.
WAL	Write-Ahead Logging, the SQLite journal mode used by the memory store.
KB	Knowledge Base (offline retrieval index).
PIN	Personal Identification Number, used as the spoken second factor for sensitive actions.
EGY	Egyptian Arabic.
WinRT	Windows Runtime API, used for no-admin Wi-Fi/Bluetooth/Airplane-mode control.
FAR / FRR	False-Accept Rate / False-Reject Rate, wake-word evaluation metrics.
p50 / p95	50th / 95th percentile latency.
EMA	Exponential Moving Average, used to smooth the wake-word detection score.
RAM	Random-Access Memory, used for hardware-tier model selection.
GPU	Graphics Processing Unit.

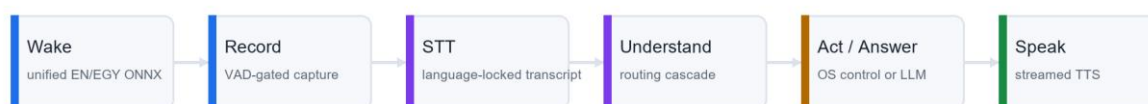
Chapter 1 Introduction

1.1 Background

Jarvis is a local-first Windows voice assistant that pairs a wake-word-triggered voice pipeline with an intent-routing cascade and a locally hosted LLM (Ollama, Qwen3 family), falling back to general conversation only when no faster tier resolves the command. It targets any Windows 10/11 PC with 8GB+ RAM and no GPU requirement, and treats Egyptian Arabic and English as equally first-class from wake word through STT, routing, and TTS, including code-switched utterances within a single sentence.

Figure 1.1 - Conceptual turn cycle

The seven-stage loop that every spoken turn passes through, bilingually.



Loop closes back to Wake after Speak; a wake-word interrupt during Act/Answer/Speak cancels the in-flight stage.

Figure 1.1: Conceptual turn cycle - wake, record, STT, understand, act/answer, speak.

1.2 Problem Statement

Commercial voice assistants are cloud-dependent, English-first, and treat code-switched Arabic/English speech as an edge case rather than the normal case for a bilingual Egyptian user. They also generally cannot verify that a spoken command actually changed the state of the operating system before reporting success, and they offer no local-only operating mode that keeps conversation and OS-control data on the user's own machine. Jarvis addresses this by running the full pipeline locally (STT, LLM, TTS all have local fallbacks), verifying OS side effects after execution rather than assuming API calls succeeded, and routing mixed-language input through a cascade tuned for Egyptian Arabic/English code-switching rather than a single-language grammar.

1.3 Objectives

ID	Objective	How it is measured in this book
O1	Accurate bilingual understanding of Egyptian Arabic and English, including code-switching.	Sec 4.6 nlp/ margin/fuzzy behavior tables; Sec 4.12 code-switch and semantic-margin test suites.
O2	Low-latency response for the common case, reserving the LLM for what earlier tiers	Sec 3.4 routing cascade tiers; Sec 4.2 command_router cascade; Sec 4.12 latency-

	cannot resolve.	oriented tests.
O3	Reliable hands-free activation without false wakes or missed wakes.	Sec 4.4 wake decision algorithm; Sec 4.5 wake-word dataset class balance and feature evidence.
O4	Verified control of the Windows OS rather than fire-and-forget API calls.	Sec 4.8 os_control verify-after-execute contract and behavior table.
O5	Safety-by-risk-tier for destructive or sensitive actions.	Sec 4.8 policy/risk_policy, confirmation/second_factor PIN flow; Sec 4.12 routing-safety tests.
O6	Continuity and personalization across sessions via durable memory.	Sec 4.2 session/memory manager; Sec 4.3 persisted data/memory/ artifacts and schema.

Table 1.1: Project objectives O1-O6 and where this book substantiates each one.

1.4 Scope

In scope: wake-word detection and adaptive retraining, hybrid cloud/local STT and TTS, the multi-tier intent routing cascade, verified Windows OS control, local LLM chat/tool-calling, durable SQLite/vector memory, an optional desktop UI bridge, and the automated test/eval suite documented in Chapter 4. Out of scope: a mobile or cross-platform client, cloud-hosted multi-user deployment, and any UI surface required for the engine to function, since the engine is designed to run headless.

1.5 Motivation

The motivating use case is a single Windows user who speaks a natural mix of Egyptian Arabic and English in the same sentence - e.g. asking to "open Chrome" or "شغل Spotify" - and expects the assistant to understand both without switching modes. Sections 4.6 and 4.4 show this is treated as the default input shape throughout the pipeline (STT language lock plus code-switch acceptance, a dedicated code-switch router tier, and bilingual TTS voice profiles) rather than bolted on as a translation layer.

1.6 Report Organization

Chapter	Content
1	Introduction: background, problem statement, objectives, scope, motivation.
2	Literature Review: prior art in commercial assistants, local LLM runtimes, STT, wake word, VAD, TTS, semantic routing and memory.
3	Methodology: design principles and the five whole-system diagrams (architecture, pipeline, routing, memory, hardware tiers).
4	Implementation: one section per top-level folder

	(4.1 root through 4.14 models/), built folder-by-folder from the real source and data.
5	Conclusion & Future Work: objectives achieved, limitations, and next steps.

Table 1.2: Reading guide for this document.

Chapter 2 Literature Review

2.1 Commercial Voice Assistants

Cloud assistants such as Siri, Google Assistant, and Alexa are optimized for broad single-language coverage and cloud-side processing; mixed-language utterances are typically handled by locale switching rather than per-utterance code-switch tolerance. Jarvis instead runs primarily on-device and treats Egyptian Arabic/English code-switching as a first-class input shape (Sec 4.6), and does not require a persistent cloud connection for its core loop (Sec 4.4, 4.7).

2.2 Local LLM Runtimes

Ollama serves as the local model runtime, hosting Qwen3-family models chosen automatically by `core/hardware_detect.py` (Sec 3.6, Sec 4.7). Running the assistant tier locally avoids per-request cloud latency and keeps conversation content on the user's machine; the optional Claude API path in `llm/tool_caller.py` is documented as an alternate backend rather than the primary one (Sec 4.7).

2.3 Speech-to-Text: Faster-Whisper and ElevenLabs

Faster-Whisper provides a local, hardware-sized STT fallback (model tier chosen by `recommend_whisper_runtime()`, Sec 3.6), while ElevenLabs provides higher cloud-quality transcription when an API key is configured. The key finding carried through Sec 4.4 is the per-utterance language lock: rather than decoding with `language=None` and letting the model guess, Jarvis fixes the primary language per utterance while still accepting code-switched words in the other script, and applies a confidence floor plus a hallucination guard (log-probability and compression ratio) to reject noise instead of passing along fabricated text.

2.4 Wake-Word Detection and Dataset Practice

`openWakeWord` provides the ONNX runtime convention that `audio/wake_word.py` builds on: a single unified model detecting both "Jarvis" and "جافيس" rather than per-language variants, decided via an EMA-smoothed score plus a raw peak threshold and confirm-frame debounce (Sec 4.4). Sec 4.5 documents the supporting dataset practice: class-balanced positive/negative WAV splits and precomputed fixed-shape feature tensors (41x96 float32), with negative examples deliberately outnumbering positive ones to bias toward false-wake resistance.

2.5 Voice Activity Detection: Silero

Silero VAD (ONNX) provides the primary speech/non-speech gate for recording, with an energy-based fallback when the ONNX model is unavailable (Sec 4.4). This two-tier VAD design mirrors the wake-word module's own primary/fallback pattern, keeping the pipeline usable in degraded environments.

2.6 Text-to-Speech: edge-tts and ElevenLabs

edge-tts supplies a free, no-API-key baseline voice for both languages (ar-EG-SalmaNeural, en-US-AriaNeural); ElevenLabs is used when configured for higher-quality synthesis. `audio/tts.py` streams sentence-by-sentence so playback starts before the full response is generated, and a deterministic (non-LLM) voice normalizer converts numbers, dates, and place names to spoken form before either backend receives the text (Sec 4.4).

2.7 Semantic Routing and Memory

sentence-transformers (multilingual MiniLM) backs the semantic router's top-k plus margin acceptance rule (Sec 4.6, Sec 3.4), letting paraphrased commands resolve without an LLM call. ChromaDB backs the optional vector-memory recall path documented in Sec 4.3 and Sec 3.5, providing bounded semantic recall for the LLM context without blocking the fast conversational path.

2.8 Positioning

Jarvis's contribution is not any single component in isolation - openWakeWord, Faster-Whisper, Ollama, and sentence-transformers are all existing open-source building blocks - but their integration into a cascade that is bilingual by default, verifies OS side effects rather than assuming them, and degrades gracefully component-by-component (Sec 4.4, 4.6, 4.8) instead of failing the whole turn when one optional dependency is missing.

Chapter 3 Methodology

3.1 Design Principles

Principle	What it means in this codebase
Latency tiering	Cheaper, earlier routing tiers (parser, code-switch, semantic) are tried before the LLM tier, so most turns never pay LLM latency (Sec 3.4, Sec 4.2).
One LLM call per turn	llm/prompt_builder.py assembles persona, language, memory, and KB context into a single prompt package rather than chaining multiple model calls (Sec 4.7).
Verify before claiming	os_control/ adapters read back real OS state after acting and only report success if it verifiably changed (Sec 4.8).
Graceful degradation	Optional dependencies (Silero, ElevenLabs, sentence-transformers, WinRT, pywin32) each have a documented fallback path rather than a hard failure (Sec 4.4, 4.6, 4.8).
Safety by risk tier	Destructive or sensitive actions route through policy/risk_policy and a spoken-PIN second factor before execution (Sec 4.8).

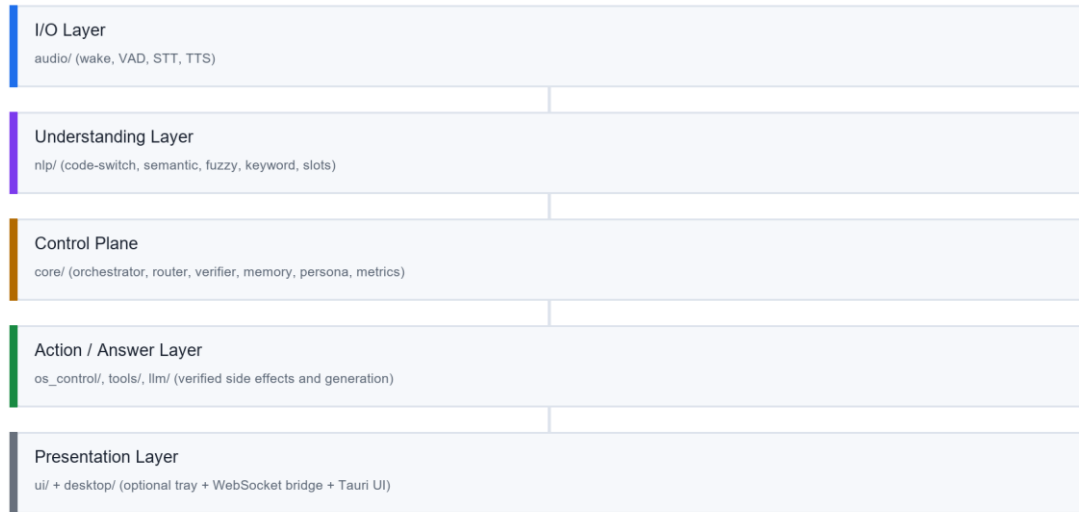
Table 3.1: Design principles evidenced across the implementation chapters.

3.2 Architecture

Jarvis is organized into five horizontal layers plus a persisted data layer that backs all of them: an I/O layer (audio/) at the boundary with the user, an understanding layer (nlp/) that turns transcripts into candidate intents, a control plane (core/) that owns routing, verification, memory, and persona, an action/answer layer (os_control/, tools/, llm/) that performs verified side effects or generates language, and an optional presentation layer (ui/, desktop/) that never gates core functionality.

Figure 3.1 - Layered architecture

Five horizontal layers; core/ is the control plane that every other layer reports to.



data/ (persisted memory, vectors, logs, wake-word artifacts) and models/ back every layer above without owning behavior.

Cross-reference: audio/ = Sec 4.4, nlp/ = Sec 4.6, core/ = Sec 4.2, os_control/tools/llm = Sec 4.7-4.9, ui/desktop = Sec 4.11.

Figure 3.1: Layered architecture from I/O through the optional presentation layer.

3.3 Runtime Pipeline

Within orchestrator.run(), a single turn flows wake -> record -> STT -> route -> verify -> dispatch -> shape -> speak. Each stage is documented in its owning folder chapter; this figure shows how those chapters compose into one pass, with the wake-word interrupt gate (runtime_coordinator.py) able to cancel dispatch, response shaping, or TTS mid-stream (Sec 4.2, 4.4, 4.7).

Figure 3.2 - Runtime pipeline

Single turn, per orchestrator.run() - top row then bottom row, left to right.



Memory and metrics read/write around every stage (Sec 4.2/4.3); a wake-word interrupt during dispatch/response/TTS cancels that stage (runtime_coordinator.py).

Latency tiering: parser/code-switch/semantic tiers resolve in low single-digit milliseconds; only unresolved turns reach the LLM tier.

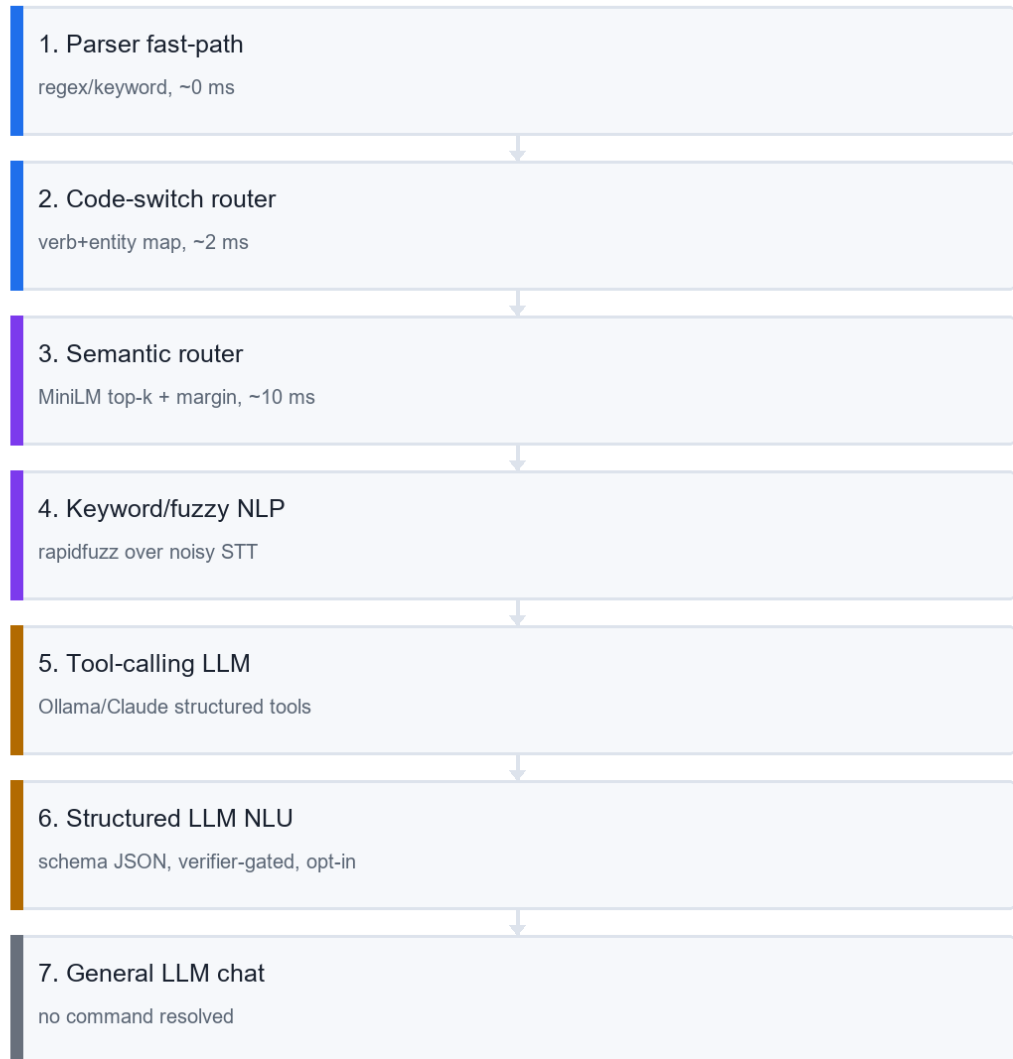
Figure 3.2: Runtime pipeline for a single turn, stage by stage.

3.4 Routing Cascade

`command_router.py` tries seven tiers in order and dispatches on the first one that resolves the command: a regex/keyword parser fast-path, a code-switch shortcut router, a semantic router, keyword/fuzzy matching, a tool-calling LLM tier, an opt-in structured-LLM NLU fallback gated behind `route_verifier.py`, and finally general LLM chat for anything that is not a command. The semantic tier's margin-acceptance rule requires the best candidate to clear an absolute confidence floor ($\text{best} \geq \text{sigma}$) and to lead the runner-up by a minimum gap ($\text{best} - \text{second} \geq \text{delta}$); failing either check defers to a later tier instead of guessing on a near-tie (Sec 4.6).

Figure 3.3 - Routing cascade

Earliest tier that resolves the command wins; each tier hands off on a miss.



route_verifier.py gates every tier's candidate before dispatch: schema/slots, question guard, risk tier, and policy.

Figure 3.3: Routing cascade tiers, earliest resolving tier wins.

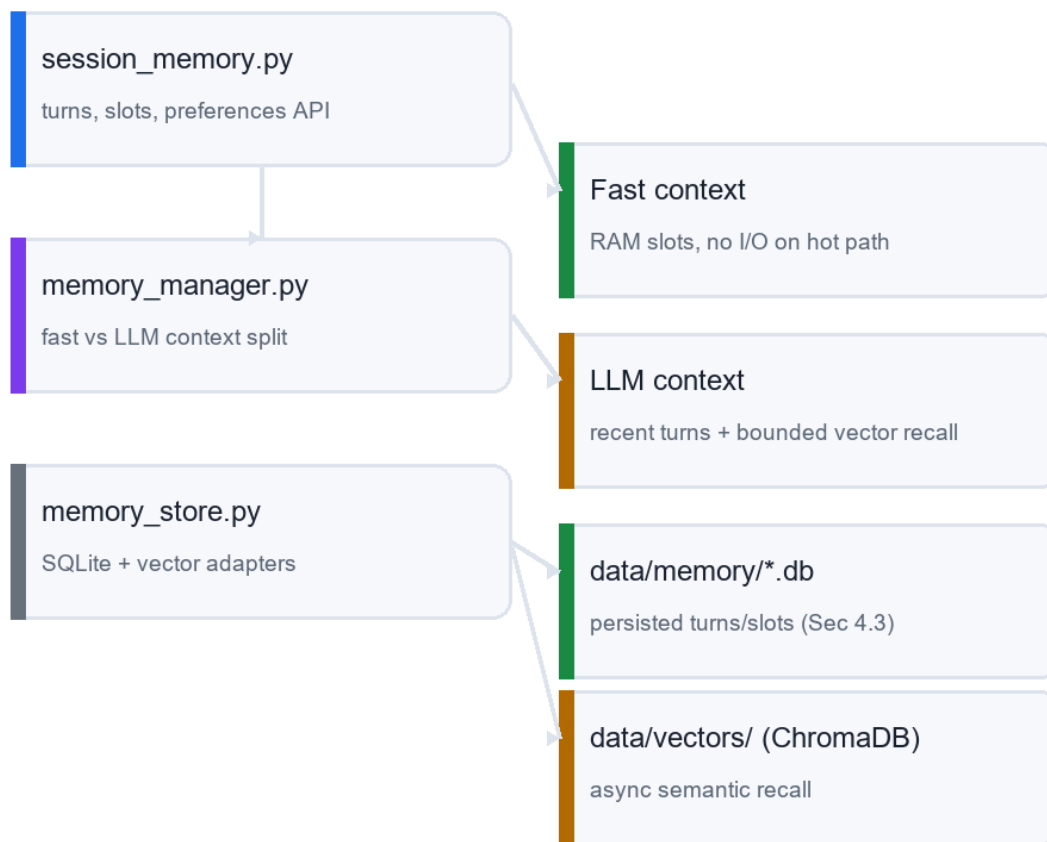
3.5 Memory Scheme

session_memory.py exposes two context shapes to the rest of the system: a fast, RAM-only context (recent slots and references, no I/O on the hot path) and a richer LLM context (recent turns plus bounded vector recall) assembled by memory_manager.py. Both are backed by the persisted store documented in Sec 4.3: a SQLite database (turns and slots tables, WAL journaling) is the primary store,

while data/vectors/ (ChromaDB with all-MiniLM-L6-v2 embeddings) supplies semantic recall asynchronously so it never blocks the fast path.

Figure 3.4 - Memory scheme

Fast RAM context vs richer LLM context, both backed by data/memory/.



LLM context write path is asynchronous so vector recall never blocks the fast conversational path.

Figure 3.4: Memory scheme - fast RAM context and LLM context over the same persisted store.

3.6 Models

Model	Role	Source
Qwen3 family (0.6b/1.7b/4b/8b)	Local chat/tool-calling LLM tier, selected by hardware tier.	Ollama, auto-pulled on first run.
Unified wake-word ONNX (jarvis_unified.onnx)	Bilingual EN/EGY wake detection, one model for both trigger phrases.	Trained via scripts/train_arabic_wake_model.py over data in Sec 4.5; loaded in Sec 4.4.

Faster-Whisper (base/small/medium)	Local STT fallback, sized by VRAM/RAM via <code>recommend_whisper_runtime()</code> .	faster-whisper package.
Silero VAD (ONNX)	Primary speech/non-speech gate, with energy-based fallback.	onnxruntime model asset.
all-MiniLM-L6-v2 / multilingual MiniLM	Sentence embeddings for semantic routing and vector memory recall.	sentence-transformers.
edge-tts / ElevenLabs voices	Bilingual TTS voice profiles (ar- EG-SalmaNeural, en-US- AriaNeural, or cloud voices).	edge-tts package / ElevenLabs API.

Table 3.2: Models used across the pipeline and where each is loaded.

Figure 3.5 - Hardware-tier selection

`core/hardware_detect.py` picks the largest Qwen3 model that fits detected RAM/GPU.

16 GB+, GPU high -> qwen3:8b num_ctx 8192
12 GB+, any medium -> qwen3:4b num_ctx 4096
8 GB, GPU medium -> qwen3:4b num_ctx 4096
8 GB, no GPU low -> qwen3:1.7b num_ctx 2048
< 8 GB, any minimal -> qwen3:0.6b num_ctx 1024

Missing models are auto-pulled via Ollama on first run.

Override with `JARVIS_LLM_MODEL` or disable via `JARVIS_LLM_AUTO_SELECT`.

Figure 3.5: Hardware-tier selection for the local LLM model.

RAM	GPU	Tier	Model	num_ctx	lightweight_ctx
16 GB+	Yes	high	qwen3:8b	8192	4096

12 GB+	Any	medium	qwen3:4b	4096	2048
8 GB	Yes	medium	qwen3:4b	4096	2048
8 GB	No	low	qwen3:1.7b	2048	1024
< 8 GB	Any	minimal	qwen3:0.6b	1024	512

Table 3.3: core/hardware_detect.py model tiers, copied from _MODEL_TIERS.

3.7 Dependencies

Tier	Representative packages	Role
Tier 1: Voice pipeline (core)	python-dotenv, numpy, sounddevice, onnxruntime, faster-whisper, openwakeword, edge-tts, soundfile	Wake word, capture, STT, TTS - required to run at all.
Tier 2: Orchestration (core)	httpx, psutil, rapidfuzz	Ollama/HTTP client, hardware detection, fuzzy intent matching.
Tier 3: Optional LLM backends	anthropic	Optional Claude API backend; Ollama remains primary.
Tier 4: Intelligent features	chromadb, sentence-transformers, duckduckgo-search, ddgs	Semantic routing, vector memory, live web search; degrade gracefully if absent.
Tier 5: OS integration (Windows)	comtypes, pycaw, wmi, winsdk, screen-brightness-control, pyperclip, watchdog, pywin32, elevenlabs, pystray, Pillow	Volume/brightness/radio control, clipboard, Outlook, tray icon.
Tier 6: Desktop UI bridge (optional)	fastapi, uvicorn[standard]	WebSocket bridge to the desktop app; engine runs without it.
Training only (requirements-training.txt)	scipy, torch	Wake-word model retraining (scripts/train_arabic_wake_model.py), not needed to run Jarvis.

Table 3.4: Dependency tiers, copied from requirements.txt/requirements-training.txt.

3.8 Key Algorithms Overview

Algorithm	Where documented	One-line summary
Wake decision	Sec 4.4	EMA-smoothed score plus raw peak threshold, confirm-frame debounce, and cooldown before triggering.
Per-utterance language lock	Sec 4.4	STT fixes the primary language per utterance instead of decoding with language=None, while still accepting code-switched words.
Semantic margin routing	Sec 4.6, Sec 3.4	Accept only if best score clears an absolute floor and leads the

		runner-up by a minimum margin; otherwise defer to the next tier.
Route verifier decision	Sec 4.2	Converts a routed candidate into execute / clarify / confirm / llm based on schema completeness, question-shape guard, and risk tier.
Verify-after-execute	Sec 4.8	Every OS-control adapter re-reads real state after acting and only reports success if the state verifiably changed.
Wake-word feature/training/eval pipeline	Sec 4.5	Fixed-shape (41x96) feature tensors from class-balanced WAV splits feed model training; evaluation happens offline before a model ships.

Table 3.5: Key algorithms and their detailed treatment elsewhere in this book.

Chapter 4 Implementation - Root

4.1 Repository Layout

Chapter 4 documents the implementation from the repository root outward. This root section fixes the boundary conditions: what starts the assistant, where configuration enters, which dependency tiers are installed, and where runtime artifacts are placed. Subsystem mechanics are intentionally deferred to their own folder chapters.

Figure 4.1.1 - Repository layout at the root boundary

Phase 1 reads the root plus core/config.py and the top-level data/ tree only.

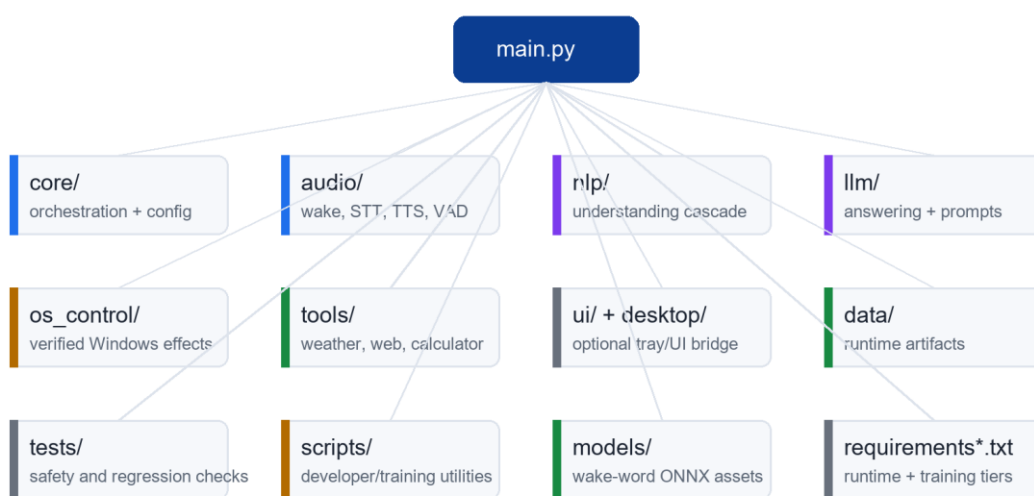


Figure 4.1.1: Repository layout and root-level responsibility boundaries.

Root item	Responsibility
main.py	Thin launcher: migrates legacy paths, parses --demo-mode, starts optional tray/bridge, then calls core.orchestrator.run().
core/	Orchestration, routing, configuration, memory logic, persona/response shaping, diagnostics, and handlers.
audio/	Wake-word detection, microphone capture, VAD, STT, and TTS runtime components.
nlp/	Text normalization, code-switch routing, semantic/fuzzy/keyword routing, and slot/entity extraction.
llm/	Ollama/Claude clients, prompt construction,

	sentence buffering, and structured NLU fallback.
os_control/	Windows side-effect adapters, policy/risk gates, confirmation, persistence, and verification.
tools/	Live-data and utility tools such as weather, web search, and calculator.
utils/	Shared helpers consumed by other layers.
ui/ and desktop/	Optional tray, WebSocket bridge, and Tauri desktop UI; the engine can run without them.
data/	Runtime artifact home for logs, memory, indices, state, KB, vectors, VAD data, and wake samples.
models/	Tracked model assets, including the deployed wake-word ONNX model.
scripts/	Developer and wake-word training/maintenance scripts.
tests/	Regression and safety checks for routing, language, memory, TTS, and other behavior.
requirements.txt	Production runtime dependency set, grouped by feature tier.
requirements-training.txt	Development-only dependencies for wake-word training.
.env.example	Configuration template; every documented JARVIS_* key carries an explicit default or blank override slot.
README.md	Top-level operator guide and architecture summary.

Table 4.1.1: Root-level files and folders with their implementation responsibilities.

4.2 Entry Point main.py

The launcher keeps policy and assistant behavior outside the root file. It performs only process setup: legacy data-path migration is imported and executed, --demo-mode sets JARVIS_DEMO_MODE, optional tray and WebSocket bridge startup are attempted defensively, and the assistant engine is handed to `core.orchestrator.run()`.

```
import argparse
import os

from core.data_migration import migrate_legacy_paths
migrate_legacy_paths()

from core.orchestrator import run
try:
    from ui.tray import start_tray
except Exception:
    start_tray = None
```

```

def _parse_args():
    parser = argparse.ArgumentParser(description="Jarvis voice AI assistant")
    parser.add_argument(
        "--demo-mode",
        action="store_true",
        default=False,
        help="Show intent/confidence overlay in console for demo presentations.",
    )
    return parser.parse_args()

if __name__ == "__main__":
    args = _parse_args()
    if args.demo_mode:
        os.environ["JARVIS_DEMO_MODE"] = "1"
    # Start system tray if available; non-blocking
    try:
        if callable(start_tray):
            start_tray()
    except Exception:
        pass
    try:
        from ui.bridge import start_bridge

```

Listing 4.1.1: The root entry point delegates runtime behavior to the core orchestrator.

4.3 Configuration core/config.py

Configuration is centralized in core/config.py. The file loads .env from the project root with override=True, then normalizes environment variables through typed helpers for strings, integers, floats, booleans, and lists. DATA_DIR defaults to the project data/ directory, and config.py ensures logs, memory, index, state, kb, and vectors exist on import.

```

def _env(key, default=""):
    """Read an environment variable with a fallback default."""
    return os.environ.get(key, default)

```

```

def _env_int(key, default):
    try:
        return int(os.environ.get(key, default))
    except (TypeError, ValueError):
        return default

def _env_float(key, default):
    try:
        return float(os.environ.get(key, default))
    except (TypeError, ValueError):
        return default

```

```

def _env_bool(key, default):
    value = os.environ.get(key)
    if value is None:

```

```
DATA_DIR = _env("JARVIS_DATA_DIR", "").strip() or _project_path("data")
```

Listing 4.1.2: The configuration helper pattern and DATA_DIR root from core/config.py.

Key	Default from config.py	Purpose
JARVIS_DATA_DIR	"	Root for runtime artifacts.
JARVIS_UI_BRIDGE_ENABLED	True	Enables the optional desktop UI bridge.
JARVIS_UI_BRIDGE_PORT	9720	Local WebSocket bridge port.
JARVIS_MAX_RECORD_DURATION	8.0	Maximum command recording window.
JARVIS_WAKE_WORD_UNIFIED_ONNX_PATH	'models/jarvis_unified/jarvis_unified.onnx'	Bilingual wake-word model path.
JARVIS_STT_BACKEND	'hybrid_elevenlabs'	Primary STT strategy.
JARVIS_LLM_BACKEND	'ollama'	LLM backend selection.
JARVIS_LLM_MODEL	'qwen3:4b'	Pinned/default

		t local model before hardware auto-selection.
JARVIS_SEMANTIC_MIN_MARGIN	0.08	Semantic router near-tie guard.
JARVIS_MEMORY_BACKEND	'sqlite'	Primary memory persistence backend.
JARVIS_SECOND_FACTOR_PIN	'1234'	Spoken PIN for sensitive actions.
JARVIS_CONTROLS_VERIFY_STATE	True	Require read-back verification for controls.
JARVIS_WEATHER_CITY	'Cairo'	Default weather location.
JARVIS_LOG_FILE	data/logs/jarvis.log unless overridden	Main runtime log path.

Table 4.1.2: Representative JARVIS_* configuration keys at the root boundary.

4.4 Environment and Dependency Files

.env.example is the operator-facing configuration template. It documents root paths, logs, security, audio, NLP, LLM, memory, controls, wake-word, and live-data settings with defaults or intentionally blank override slots. Secrets such as API keys are configuration inputs, not code constants.

Tier	Representative packages	Role
Tier 1: VOICE PIPELINE (CORE)	python-dotenv==1.2.2, numpy==2.4.2, sounddevice==0.5.5, onnxruntime>=1.26.0, faster-whisper==1.2.1, openwakeword==0.6.0, edge-tts>=6.1.12	runtime
Tier 2: ORCHESTRATION (CORE)	httpx==0.28.1, psutil==7.2.2, rapidfuzz==3.14.3	runtime
Tier 3: OPTIONAL LLM BACKENDS	anthropic>=0.40.0	runtime
Tier 4: INTELLIGENT FEATURES	chromadb>=0.5.0, sentence-transformers>=3.0.0,<5.0.0, duckduckgo-search>=7.0.0,	runtime

	ddgs>=9.0.0	
Tier 5: OS INTEGRATION (WINDOWS-SPECIFIC)	comtypes>=1.4.8; platform_system == "Windows", pycaw>=20240210; platform_system == "Windows", wmi>=1.5.1; platform_system == "Windows", winsdk>=1.0.0b10; platform_system == "Windows", screen-brightness- control>=0.23.0, pyperclip>=1.9.0, watchdog>=3.0.0	runtime
Tier 6: DESKTOP UI BRIDGE (OPTIONAL)	fastapi>=0.115.0, uvicorn[standard]>=0.34.0	runtime
Training only	scipy>=1.12.0, torch>=2.2.0	wake-word development

Table 4.1.3: Dependency tiers declared by requirements.txt and requirements-training.txt.

4.5 The data/ Runtime Tree

The root perspective treats data/ as the runtime artifact tree. The memory store and wake-word samples are listed here only at a glance because Section 4.3 documents persisted memory in depth, and Section 4.5 documents wake-word data/training provenance.

data/ item	Root-level meaning
chroma_memory/	ChromaDB/vector-memory artifacts from earlier or alternate semantic-memory storage.
index/	Search-index cache files, including the configured jarvis_index.db path.
kb/	Offline knowledge-base index and metadata files.
logs/	Structured runtime logs and action audit logs.
memory/	SQLite/JSON memory store; documented in depth in Section 4.3.
state/	Runtime state database and small persisted state artifacts.
vad/	VAD-related runtime artifacts or cached data.
vectors/	Vector semantic-memory store configured by JARVIS_VECTOR_MEMORY_DIR.
wake_samples/	Wake-word enrollment and adaptive-retraining samples; training pipeline is documented in Section 4.5.

Table 4.1.4: Top-level data/ runtime directories observed in this workspace.

4.6 Root-Perspective Startup Sequence

Step	Root-visible behavior	Deferred detail
1	Import	Migration mechanics are

	<code>core.data_migration.migrate_legacy_paths()</code> and execute it before runtime starts.	covered in <code>core/</code> .
2	Parse <code>--demo-mode</code> ; if present, set <code>JARVIS_DEMO_MODE=1</code> for downstream modules.	Demo overlay behavior is covered with <code>routing/core</code> behavior.
3	Attempt to start the optional system tray without making it a hard dependency.	UI tray behavior is covered in Section 4.11.
4	Attempt to start the optional WebSocket bridge for the desktop UI.	Bridge protocol is covered in Section 4.11.
5	Call <code>core.orchestrator.run()</code> to begin the assistant loop.	Wake/listen/process mechanics are covered in <code>core/</code> and <code>audio/</code> .

Table 4.1.5: Startup behavior visible from the repository root.

Phase 1 deliberately stops at the root boundary. It does not describe subsystem internals; later phases fill those bookmarked sections with folder-local evidence.

4.2 `core/`

Purpose

The `core/` package is the runtime control plane for Jarvis. It receives recognized utterances from the audio layer, normalizes and routes them, verifies intent safety, dispatches to handlers or LLM response paths, and records memory, metrics, and diagnostics. In the finished Methodology chapter this section should cross-reference Figure 3.1; for now it documents the folder-local implementation evidence.

File Inventory

File	Responsibility
<code>core/action_planner.py</code>	Plans and executes multi-step internal command sequences.
<code>core/adaptive_wake.py</code>	Records confirmed/false wake samples and runs background adaptive retraining hooks.
<code>core/clarification_builder.py</code>	Builds slot-specific and ambiguity clarification prompts.
<code>core/command_classifier.py</code>	NLU helper layer for extracting actions, slots, numbers, URLs, schedule payloads, and command targets.
<code>core/command_parser.py</code>	Regex/keyword parser that emits <code>ParsedCommand</code> objects and structured args.
<code>core/command_router.py</code>	Main routing cascade, verifier handoff, dispatch, LLM fallback, follow-up handling, and response shaping.
<code>core/config.py</code>	Central environment/configuration surface and data-directory setup.
<code>core/data_migration.py</code>	One-time migration of legacy root runtime artifacts into <code>data/</code> .

core/demo_mode.py	Thread-safe demo-mode flag used for presentation overlays.
core/dialogue_manager.py	Dialogue state machine for idle, follow-up, confirmation, and command phases.
core/doctor.py	Dependency, feature-tier, latency, model, and health diagnostics.
core/elevation.py	Windows Administrator/elevation detection.
core/eval_routing.py	Decision-only route harness for NLU evaluation.
core/handlers/__init__.py	Handler package marker.
core/handlers/advanced_operations.py	Command chains, batch file operations, and semantic search handler helpers.
core/handlers/audit.py	Formats audit-log, verification, and reseal responses.
core/handlers/batch.py	Delegates batch operations to os_control batch services.
core/handlers/file_navigation.py	Handles file navigation/search/open responses and last-file memory.
core/handlers/job_queue.py	Formats and handles background job queue commands.
core/handlers/knowledge_base.py	Handles knowledge-base sync/search/status commands.
core/handlers/memory.py	Handles memory summary, preference, and stored-context commands.
core/handlers/persona.py	Handles persona status and persona-switch commands.
core/handlers/policy.py	Handles policy status/profile commands.
core/handlers/search_index.py	Handles search-index refresh/status commands.
core/handlers/voice.py	Handles runtime voice/STT/audio UX profile commands.
core/hardware_detect.py	RAM/GPU detection and Qwen3/Faster-Whisper runtime recommendations.
core/identity.py	Varied bilingual self-introduction replies.
core/intent_confidence.py	Intent confidence, entity scoring, clarification resolution, and ambiguity safeguards.
core/intent_schema.py	Single registry of routable intents, required slots, risk tiers, and fast-execute flags.
core/knowledge_base.py	Offline knowledge-base indexing/retrieval service API.
core/language_gate.py	English/Arabic script detection and supported-language normalization.
core/logger.py	Shared logging, structured event logging, and Windows-safe rotation.
core/memory_manager.py	Fast RAM context vs richer LLM context assembly.
core/memory_store.py	SQLite and vector-memory persistence adapters used by SessionMemory.
core/memory_types.py	MemoryContext data shape shared by fast and

	LLM memory paths.
core/metrics.py	Latency, quality, clarification, diagnostic, and stage timing trackers.
core/orchestrator.py	Runtime startup ordering, wake/listen/process loop, concurrency, and cleanup.
core/persona.py	Active persona configuration and prompt-block formatting.
core/response_shaper.py	Voice-oriented response shortening, cleanup, and persona-safe shaping.
core/response_templates.py	Bilingual response templates for common control, clarification, and safety states.
core/route_verifier.py	Single gate that converts a routed candidate into execute/clarify/confirm/llm.
core/runtime_coordinator.py	Wake-word interrupt gate for TTS/LLM cancellation by runtime phase.
core/session_memory.py	In-process memory API over turns, slots, preferences, references, pending tasks, and persistence.
core/shutdown.py	Signal handling and graceful cleanup for TTS, queues, and background services.
core/tts_prosody.py	Deterministic TTS punctuation/prosody polishing.
core/tts_voices.py	Voice profile registry and TTS voice environment aliases.
core/voice_normalizer.py	Deterministic spoken-form normalization for numbers, dates, weather, places, and search results.

Table 4.2.1: Inventory of every Python file in core/ and core/handlers/.

Internal Data Flow

Figure 4.2.1 - core/ internal data flow

Runtime loop, routing gate, dispatch, memory, and observability.



Phase boundary: core/ documents in-process logic. Persisted memory files are documented in Section 4.3.

Figure 4.2.1: Core runtime flow from orchestrator loop through routing, verification, dispatch, memory, and metrics.

Key Modules In Depth

orchestrator.py owns startup ordering and the long-lived wake/listen/process loop. It logs startup state, checks elevation, installs shutdown handling, initializes command services, warms latency-critical components, attaches interrupt targets, plays the blocking greeting, starts cache/adaptive wake background work, and then records utterances into the processing pipeline.

```
def run():
    section("Jarvis")
    kv(
        "startup",
        model=LLM_MODEL,
        tier="auto" if LLM_AUTO_SELECT_MODEL else "configured",
    )
    kv(
        "startup",
        followup=FOLLOWUP_ENABLED,
        wake_interrupt=True,
    )
    from core.elevation import is_admin as _is_admin
    _elevated = _is_admin()
    kv("startup", elevated=_elevated)
    if not _elevated:
        logger.info(
            "Jarvis is not running as Administrator. Wi-Fi/Bluetooth adapter "
            "disable and a few other controls may report an honest failure "
            "with an admin hint until Jarvis is run elevated."
        )
        Listing 4.2.1: orchestrator.run() startup boundary.

    if wake_source == "wake":
        # Let the tail of "Jarvis"/"ج ا ر ف ي س" finish and the mic device
        # settle before opening the recording stream, so the wake
        # word itself is never captured as the start of the command.
        _delay_ms = int(WAKE_WORD_RECORD_START_DELAY_MS)
        if _delay_ms > 0:
            time.sleep(_delay_ms / 1000.0)

    coordinator.set_phase(RuntimePhase.RECORDING)
    from llm.ollama_client import llm_cancel_event as _llm_cancel
    _llm_cancel.clear()
```

```

record_started = time.perf_counter()

_partial_latency_recorded = False

def _pipeline_partial(partial_text):
    nonlocal _partial_latency_recorded
    _on_partial_transcript(partial_text)
    concurrent_pipeline.on_partial(partial_text)
    if not _partial_latency_recorded and (partial_text or "").strip():

```

Listing 4.2.2: wake-source delay, recording phase, and partial transcript callback.

command_router.py implements the cascade. It starts with parser fast-path confidence, then code-switch routing, semantic top-k/margin routing, keyword NLP, gated tool/structured-LLM tiers, and finally LLM_QUERY fallback. The route verifier is called before final dispatch metadata is logged.

```

if parsed is None and NLU_INTENT_ROUTING_ENABLED and not explanatory_llm_query:
    # Tier 1 fast-path: high-confidence regex parser match
    parser_fastpath_assessment = _select_parser_fastpath_assessment(
        original_text,
        parser_candidate,
        language_result.language,
    )
    if parser_fastpath_assessment is not None:
        parsed = parser_candidate
        nlu_meta["nlu_fastpath"] = True

```

```

if parsed is None and not explanatory_llm_query:
    # Tier 1.5: Code-switch shortcut – dictionary/token match (<2ms)
    with stage_timer("codeswitch"):
        codeswitch_parsed, codeswitch_meta = _try_codeswitch_routing(
            original_text,
            parser_candidate,
            language_result.language,
        )
    nlu_meta.update(codeswitch_meta)

```

Listing 4.2.3: routing cascade tiers before LLM fallback.

```

if ROUTE_VERIFIER_ENABLED:
    try:
        with stage_timer("verify"):
            route_decision = route_verifier.verify(
                parsed,

```

```

        original_text,
        assessment.confidence,
        entity_scores=assessment.entity_scores,
        language=language_result.language,
        fast_command_min_confidence=FAST_COMMAND_MIN_CONFIDENCE,
        should_clarify=assessment.should_clarify,
    )
    nlu_meta["route_decision"] = route_decision.action
    nlu_meta["route_decision_reason"] = route_decision.reason
except Exception as exc:
    logger.debug("route_verifier.verify failed: %s", exc)

if assessment.should_clarify:
    clarification_payload = build_clarification_payload(
        assessment,

```

Listing 4.2.4: route_verifier hand-off from command_router.py.

route_verifier.py consolidates candidate checks into a RouteDecision with action values execute, clarify, confirm, or llm. The order is schema existence, required slots, question-opener demotion, entity confidence, risk gate, confidence/clarification verdict, and policy permission.

```

def verify(candidate_parsed, normalized, confidence, entity_scores=None,
           language="en", fast_command_min_confidence=0.88,
           should_clarify=False) -> RouteDecision:
    """Turn a routed candidate into an execute/clarify/confirm/llm decision.

    candidate_parsed: ParsedCommand-like object with .intent, .action, .args
    normalized: normalized utterance text (for the question-opener check)
    confidence: numeric confidence for this candidate (0..1)
    entity_scores: optional dict of per-slot entity confidence scores
    should_clarify: the caller's own assess_intent_confidence verdict – this
        verifier defers to it rather than re-deriving a confidence floor.
    """
    intent = str(getattr(candidate_parsed, "intent", "") or "").strip().upper()
    action = str(getattr(candidate_parsed, "action", "") or "")
    slots = dict(getattr(candidate_parsed, "args", None) or {})
    entity_scores = entity_scores or {}

    # 1. Schema existence – no schema entry means we don't know its shape/risk.
    spec = intent_schema.get_spec(intent)
    if spec is None:

```

Listing 4.2.5: route_verifier.verify() input normalization and schema gate.

```
# 2. Required slots present.
missing = [slot for slot in spec.required_slots if not str(slots.get(slot) or
 "").strip()]
if missing:
    return RouteDecision(
        action="clarify", intent=intent, slots=slots,
        reason=f"missing_slot:{missing[0]}", confidence=confidence,
    )

# 3. Question-masquerading-as-command penalty.
if intent in _QUESTION_PENALTY_INTENTS and _QUESTION_OPENER_RE.match(
    " ".join(str(normalized or "").split())
):
    return RouteDecision(
        action="llm", intent=intent, slots=slots,
        reason="question_opener_detected", confidence=confidence,
    )

# 4. Entity confidence.
if entity_scores:
    min_score = min(float(v) for v in entity_scores.values())
```

Listing 4.2.6: verifier missing-slot, question, entity, and risk decisions.

intent_schema.py is the folder's intent contract. Each IntentSpec declares the domain, required and optional slots, risk tier, fast-execute eligibility, and bilingual examples. The schema contains 38 registered intents in this workspace, including OS, file, safety, meta, metrics, and LLM_QUERY domains.

```
class IntentSpec:
    name: str
    domain: str
    required_slots: Tuple[str, ...] = ()
    optional_slots: Tuple[str, ...] = ()
    risk: str = "low"
    fast_execute: bool = False
    examples_en: Tuple[str, ...] = ()
    examples_ar: Tuple[str, ...] = ()

# -----
# Schema registry
```

```
# -----
```

```
SCHEMA: dict[str, IntentSpec] = {}
```

```
def _reg(*args, **kwargs) -> IntentSpec:
```

```
    spec = IntentSpec(*args, **kwargs)
```

Listing 4.2.7: IntentSpec and a bilingual OS_APP_OPEN schema entry.

Memory is split by latency. `memory_manager.py` exposes `get_fast_context()` for RAM-only working slots and `get_llm_context()` for recent turns plus bounded semantic recall. `session_memory.py` owns the in-process API; `memory_store.py` provides SQLite and vector persistence adapters. The physical files these APIs read/write are deferred to Section 4.3.

```
class MemoryManager:
```

```
    def get_fast_context(self) -> MemoryContext:
```

```
        """RAM-only working slots. No vector store access, no LLM. Target < 5ms."""
```

```
        with stage_timer("memory_fast"):
```

```
            if not MEMORY_FAST_CONTEXT_ENABLED:
```

```
                return MemoryContext()
```

```
            snapshot = session_memory.context_snapshot()
```

```
            working_slots = {
```

```
                "last_app": snapshot.get("last_app", ""),
```

```
                "previous_app": snapshot.get("previous_app", ""),
```

```
                "last_file": snapshot.get("last_file", ""),
```

```
                "pending_confirmation_token": snapshot.get("pending_confirmation_token",
```

```
                ""),
```

```
                "language_history": snapshot.get("language_history", []),
```

```
                "response_mode": snapshot.get("response_mode", "default"),
```

```
            }
```

```
            preferences = {}
```

```
            if MEMORY_PREFERENCES_ENABLED:
```

```
                try:
```

```
                    preferences = session_memory.get_user_preferences()
```

```
                except Exception as exc:
```

Listing 4.2.8: RAM-only fast memory context.

```
def get_llm_context(self, query: str, language: str = "en") -> MemoryContext:
```

```
    """Fast context + recent turns + bounded semantic recall.
```

Only meant for LLM/uncertain routes – never called on the direct command hot path.

```

"""
with stage_timer("memory_llm"):
    if not MEMORY_LLM_CONTEXT_ENABLED:
        return self.get_fast_context()

    context = self.get_fast_context()

    try:
        context.recent_turns =
session_memory.recent(limit=MEMORY_SHORT_TERM_TURNS)
    except Exception as exc:
        logger.debug("get_llm_context: recent turns unavailable: %s", exc)
        context.recent_turns = []

    query_words = len(str(query or "").split())
    if MEMORY_VECTOR_RECALL_ENABLED and query_words >=
MEMORY_VECTOR_RECALL_MIN_QUERY_WORDS:

```

Listing 4.2.9: LLM memory context with bounded semantic recall.

```

class SQLiteMemoryStore:
    """Thread-safe SQLite wrapper used by :class:`SessionMemory`."""

    def __init__(self, db_path: Optional[str] = None):
        self._db_path = str(db_path or MEMORY_DB_FILE)
        self._lock = threading.RLock()
        self._connection: Optional[sqlite3.Connection] = None
        self._ensure_open()
        self._ensure_schema()

    # -----
    # Connection lifecycle
    # -----
    def _ensure_open(self) -> sqlite3.Connection:
        with self._lock:
            if self._connection is not None:
                return self._connection

            directory = os.path.dirname(self._db_path)
            if directory:
                os.makedirs(directory, exist_ok=True)

```

Listing 4.2.10: SQLite memory store connection and WAL configuration.

metrics.py supplies shared stage timers and latency buckets used throughout routing, recording, LLM, TTS, memory, and diagnostics. doctor.py reports dependency and feature availability, while hardware_detect.py selects model tiers from RAM/GPU conditions.

```
@contextmanager
def stage_timer(stage: str, **fields):
    """Measure one pipeline stage and feed the shared latency tracker."""
    measurement = _StageMeasurement()
    started = time.perf_counter()
    try:
        yield measurement
    finally:
        measurement.elapsed = time.perf_counter() - started
        record_stage_timing(stage, measurement.elapsed, **fields)

def record_stage_timing(stage: str, elapsed: float, **fields) -> None:
    latency_tracker.record(stage, elapsed)
    latest = getattr(_thread_stage_timings, "latest", None)
    if latest is None:
        latest = {}
        _thread_stage_timings.latest = latest
    latest[str(stage)] = max(0.0, float(elapsed))
    if TIMING_LOG_ENABLED:
```

Listing 4.2.11: stage_timer and record_stage_timing.

```
# Model tiers ordered by quality (best first within each hardware bracket).
# Format: (min_ram_gb, gpu_required, tier_label, model_name, num_ctx,
lightweight_num_ctx)
_MODEL_TIERS = [
    # 16GB+ RAM, GPU – best quality
    (16, True, "high", "qwen3:8b", 8192, 4096),
    # 12GB+ RAM (GPU or not) – comfortable headroom for qwen3:4b
    (12, False, "medium", "qwen3:4b", 4096, 2048),
    # 8GB RAM, GPU – qwen3:4b runs fast on GPU
    (8, True, "medium", "qwen3:4b", 4096, 2048),
    # 8GB RAM, no GPU – qwen3:1.7b fits comfortably
    (8, False, "low", "qwen3:1.7b", 2048, 1024),
    # <8GB RAM – fall back to the smallest production-grade Qwen3 model
    (0, False, "minimal", "qwen3:0.6b", 1024, 512),
```

Listing 4.2.12: Qwen3 model-tier table.

Algorithms

Verifier decision order:

```
candidate = routed ParsedCommand-like object
spec = intent_schema.get_spec(candidate.intent)
if spec is missing: return llm(no_schema_entry)
if required slot is blank: return clarify(missing_slot)
if question opener targets command intent: return llm(question_opener_detected)
if any entity score < 0.45: return clarify(low_entity_confidence)
risk = per-action override or schema risk
if risk is high or medium: return confirm(risk)
if assess_intent_confidence asked to clarify: return clarify(low_confidence)
if policy blocks permission: return llm(policy_blocked)
return execute(ok)
```

Hardware tier selection:

```
ram_gb = detect_total_ram_gb()
gpu = detect_gpu_available(ollama_base_url)
for each tier ordered high -> minimal:
    if ram_gb >= min_ram and (not gpu_required or gpu):
        return model, num_ctx, lightweight_num_ctx, ram_gb, gpu
fallback = qwen3:0.6b with 1024 context
```

Configuration Surface

Key	Default	Core-owned purpose
JARVIS_NLU_INTENT_ROUTING_ENABLED	True	Enable the routing cascade before LLM fallback.
JARVIS_NLU_PARSER_FASTPATH_ENABLED	True	Allow direct parser fast-path routing.
JARVIS_NLU_PARSER_FASTPATH_CONFIDENCE_FLOOR	0.55	Minimum confidence floor for fast parser acceptance.
JARVIS_CODESWITCH_ROUTER_ENABLED	True	Enable mixed English/Egyptian-Arabic shortcut routing.
JARVIS_SEMANTIC_ROUTER_ENABLED	True	Enable embedding-based semantic routing.
JARVIS_SEMANTIC_MIN_CONFIDENCE	0.74	Minimum semantic score for candidate acceptance.
JARVIS_SEMANTIC_MIN_MARGIN	0.08	Minimum gap between top semantic candidate

		and runner-up.
JARVIS_ROUTE_VERIFIER_ENABLED	True	Enable verifier logging and decision calculation.
JARVIS_FAST_COMMAND_MIN_CONFIDENCE	0.88	Verifier/eval fast-command tuning input.
JARVIS_CLARIFY_FROM_TEMPLATES	True	Use template-based clarification prompts.
JARVIS_CLARIFY_MAX_ROUNDS	2	Maximum clarification turns before fallback.
JARVIS_STRUCTURED_LLM_NLU_ENABLED	False	Enable gated structured-LLM NLU fallback.
JARVIS_MEMORY_FAST_CONTEXT_ENABLED	True	Expose fast RAM-only context to router paths.
JARVIS_MEMORY_LLM_CONTEXT_ENABLED	True	Allow recent turns and semantic recall for LLM context.
JARVIS_MEMORY_SHORT_TERM_TURNS	6	Number of recent turns in LLM-bound memory context.
JARVIS_MEMORY_VECTOR_RECALL_ENABLED	True	Enable bounded vector recall for LLM/uncertain routes.
JARVIS_DOCTOR_STARTUP_ASYNC	True	Run startup diagnostics off the critical path.
JARVIS_STARTUP_BACKGROUND_PREWARM_ENABLED	True	Load optional services in background.
JARVIS_PREWARM_SEMANTIC_ROUTER_BLOCKING	False	Force semantic-router warmup onto startup path if true.
JARVIS_PREWARM_LLM_BLOCKING	False	Force LLM warmup onto startup path if true.
JARVIS_FOLLOWUP_ENABLED	True	Enable follow-up listening window after responses.
JARVIS_ROUTE_TIMING_LOG	True	Emit per-stage route timings.
JARVIS_MEMORY_TIMING_LOG	True	Emit memory timing lines.

Table 4.2.2: Core-owned JARVIS_* configuration surface relevant to routing, memory, startup, and diagnostics.

Behavior In Different Situations

Situation	Core behavior	Evidence
Normal	orchestrator initializes services, records speech, route_command resolves the	orchestrator.py run loop; command_router.py route_command; response_shaper.py.

	first confident tier, verifier logs action/reason, dispatch returns a shaped response.	
Degraded	If semantic routing is unavailable or not ready, command_router marks semantic_pending/unavailable and continues to keyword/tool/LLM paths; if SQLite memory fails, SessionMemory falls back to JSON.	command_router.py _try_semantic_routing; session_memory.py initialization.
Adversarial/Edge	Question-shaped commands are demoted to LLM, missing slots ask clarification, medium/high risk actions require confirmation, and policy blocks return an LLM-style non-execution decision.	route_verifier.py verify(); intent_schema.py risk fields.
Operational	RuntimeCoordinator accepts wake interrupts only in interruptible phases and cancels attached TTS/LLM work; blocked phases play/log the blocked path.	runtime_coordinator.py RuntimeCoordinator.request_interrupt().

Table 4.2.3: Core behavior under normal, degraded, and edge conditions.

Contribution Summary

core/ advances the project objectives by making the assistant deterministic at the decision boundary: fast routes stay cheap, uncertain routes ask or defer, risky routes confirm, and every turn leaves timing/memory evidence. It also keeps bilingual English/Egyptian-Arabic handling visible in schemas, templates, language gates, and persona/voice shaping while leaving physical data artifacts to their dedicated chapters.

4.3 data/memory/

Purpose

data/memory/ is the durable memory artifact layer behind the in-process memory APIs documented in Section 4.2. This phase reads only the memory files on disk. The observed store preserves recent turns, working slots, app/file references, language history, clarification preferences, and preference-like state used for continuity and personalization across sessions.

File and Subfolder Inventory

Artifact	Format	What it stores	Reader/writer
----------	--------	----------------	---------------

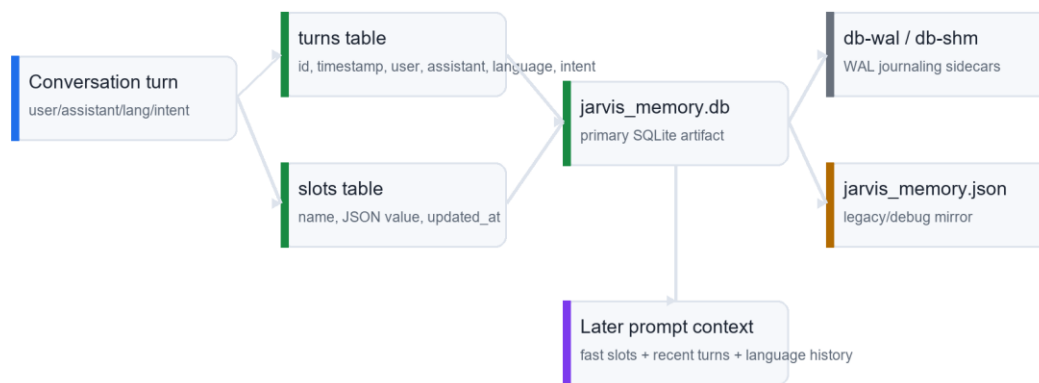
			relationship
jarvis_memory.db	SQLite database	Primary observed store with turns and slots tables. Size: 57344 bytes.	core memory logic reads/writes it, per Section 4.2.
jarvis_memory.db-shm	SQLite WAL shared-memory file	SQLite sidecar used while WAL mode is active. Size: 32768 bytes.	SQLite runtime companion for jarvis_memory.db.
jarvis_memory.db-wal	SQLite write-ahead log	SQLite sidecar for pending WAL pages; observed length may be zero when checkpointed. Size: 0 bytes.	SQLite runtime companion for jarvis_memory.db.
jarvis_memory.json	JSON	Legacy/debug mirror containing preferred_language, turns, pending_clarification, and context_slots. Size: 4586 bytes.	Legacy import/debug-export target, per prior core chapter.

Table 4.3.1: Files observed directly under data/memory/.

Internal Data Flow

Figure 4.3.1 - Persisted memory artifacts

Observed files inside data/memory/: SQLite rows, JSON slots, and WAL companions.



Data-only phase note: code mechanisms were covered in Section 4.2; this chapter documents files and schemas observed on disk.

Figure 4.3.1: Persisted memory data flow from turn/slot writes to later context reconstruction.

Key Artifacts In Depth

jarvis_memory.db is the primary observed SQLite artifact. Its PRAGMA values report journal_mode=delete, page_count=14, and page_size=4096. The database currently contains 10 rows in turns and 33 rows in slots.

SQLite table	Observed schema
slots	CREATE TABLE slots (name TEXT PRIMARY KEY, value TEXT, updated_at REAL)
sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)
turns	CREATE TABLE turns (id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp REAL NOT NULL, user TEXT, assistant TEXT, language TEXT, intent TEXT)

Table 4.3.2: SQLite table schema observed in jarvis_memory.db.

SQLite index	Definition
sqlite_autoindex_slots_1	implicit SQLite index
turns_by_time	CREATE INDEX turns_by_time ON turns(timestamp)

Table 4.3.3: SQLite indexes observed in jarvis_memory.db.

The turns table stores bounded recent conversation rows with timestamp, user text, assistant text, language, and intent. To avoid leaking transcript content in the thesis artifact, the table below records only ids, language, intent, and text lengths.

Recent id	Timestamp	Language	Intent	User chars	Assistant chars
81117	1783174782.356861	en	OS_FILE_SEARCH	33	70
81116	1783174548.4568825	en	OS_SYSTEM_COMMAND	47	57
81115	1783174475.8257716	en	OS_SYSTEM_COMMAND	49	25
81114	1783174455.8099797	en	OS_FILE_SEARCH	44	16
81113	1783174087.9736345	en	OS_FILE_SEARCH	44	16

Table 4.3.4: Sanitized recent-turn sample from the turns table.

The slots table stores JSON-encoded working state. Observed slot names include app/file references, pending confirmation state, clarification preferences, app and command usage, response mode, audio/STT profiles, preferred language, and bilingual language history.

Slot name	JSON category	updated_at
__legacy_json_imported__	number	0.0
app_usage	object (24 keys)	0.0
app_usage_updated_at	number	0.0
audio_ux_profile	string	0.0
audio_ux_profile_updated_at	number	0.0
clarification_preferences	object (6 keys)	0.0
clarification_preferences_updated_at	number	0.0
command_usage	object (1 keys)	0.0

command_usage_updated_at	number	0.0
language_history	array (7 items)	0.0
language_history_persisted	array (3 items)	0.0
language_history_persisted_updated_at	number	0.0
language_history_updated_at	number	0.0
last_app	string	0.0
last_app_updated_at	number	0.0
last_clarification_resolution_at	number	0.0
last_clarification_resolution_intent	string	0.0
last_clarification_resolution_reason	string	0.0
last_file	string	0.0
last_file_updated_at	number	0.0
pending_confirmation_token	string	0.0
pending_confirmation_updated_at	number	0.0
pending_task	object (0 keys)	0.0
pending_task_updated_at	number	0.0
preferred_language	string	0.0
previous_app	string	0.0
previous_app_updated_at	number	0.0
response_mode	string	0.0
response_mode_updated_at	number	0.0
stt_profile	string	0.0
stt_profile_updated_at	number	0.0
user_preferences	object (0 keys)	0.0
user_preferences_updated_at	number	0.0

Table 4.3.5: Slot names and value categories observed in the slots table.

jarvis_memory.json is present as a JSON artifact with top-level keys context_slots, pending_clarification, preferred_language, turns. It contains 10 turn objects, preferred_language=en, and 23 context slot keys. The SQLite slots table also contains __legacy_json_imported__, which records that the JSON import path has already been consumed.

JSON context slot	Observed type
app_usage	object (1 keys)
app_usage_updated_at	float
audio_ux_profile	str
audio_ux_profile_updated_at	float
clarification_preferences	object (0 keys)
clarification_preferences_updated_at	float
language_history	array (8 items)
language_history_updated_at	float
last_app	str
last_app_updated_at	float
last_clarification_resolution_at	float
last_clarification_resolution_intent	str
last_clarification_resolution_reason	str

last_file	str
last_file_updated_at	float
pending_confirmation_token	str
pending_confirmation_updated_at	float
previous_app	str
previous_app_updated_at	float
response_mode	str
response_mode_updated_at	float
stt_profile	str
stt_profile_updated_at	float

Table 4.3.6: Context slot keys observed in jarvis_memory.json.

jarvis_memory.db-shm and jarvis_memory.db-wal are SQLite WAL-mode companions. The observed WAL file is zero bytes at inspection time, which is consistent with no pending uncheckpointed pages. The SHM file is the shared-memory sidecar used by SQLite while WAL mode is active.

Algorithms Reflected By The Data

Observed turn/slot persistence pattern:

```

on each remembered turn:
    append or retain a row in turns(timestamp, user, assistant, language, intent)
    update slots for working references such as last_app, last_file, language_history
    encode complex values as JSON strings in slots.value
    keep SQLite in WAL mode for safer concurrent reads/writes
later context reconstruction:
    read recent rows from turns
    read named slots and decode JSON
    combine language history, references, preferences, and recent turns

```

Data-only retention observation:

```

observed turns row count = 10
observed turn ids are high while retained row count is small
therefore the data suggests a bounded recent-turn store
the exact pruning trigger is not claimed here because this phase did not read source
code

```

Configuration Surface

Storage setting	Observed path/value	Meaning from data perspective
Memory directory	data/memory/	Folder containing the persisted memory artifacts inspected in this phase.
Primary database	data/memory/jarvis_memory.db	SQLite store with turns and slots tables.
WAL sidecars	jarvis_memory.db-wal and	SQLite journal/shared-memory

	jarvis_memory.db-shm	companions.
Legacy/debug JSON	data/memory/jarvis_memory.json	JSON state file with turns and context_slots.
Language state	preferred_language slot plus language_history arrays	Observed bilingual continuity state.

Table 4.3.7: Memory storage surface visible from persisted artifacts.

Behavior In Different Situations

Situation	Observed artifact behavior	Defense note
Normal	SQLite database contains turns and slots; WAL mode is active; language_history includes English and Arabic entries.	Supports continuity and bilingual context retention.
Degraded	JSON file remains available as a legacy/debug state artifact alongside SQLite.	The data itself shows a migration/import marker but this chapter does not claim source-level fallback mechanics.
Adversarial/Edge	Transcript text is stored in turns, while thesis excerpts sanitize content to lengths only; slots use JSON values that can represent empty pending tasks and cleared confirmation tokens.	Sensitive or noisy state can be audited without exposing full utterance text in documentation.
Corruption/Concurrency	WAL and SHM companions are present; WAL length can be zero after checkpointing.	SQLite journaling artifacts are self-contained in data/memory/.

Table 4.3.8: Memory data behavior under normal, degraded, and edge situations.

Contribution Summary

The persisted memory store advances continuity by keeping recent turns, working references, preferences, and language history across launches. Its observed English/Arabic language-history slots and preferred-language state support bilingual interaction without requiring every turn to rediscover context from scratch.

4.4 audio/

Purpose

audio/ is the voice I/O boundary. It detects the bilingual wake word, captures speech with VAD, transcribes audio with cloud/local STT paths, streams partial transcripts for early routing, and speaks responses through hybrid TTS. The wake-word model training data is intentionally deferred to Section 4.5.

File Inventory

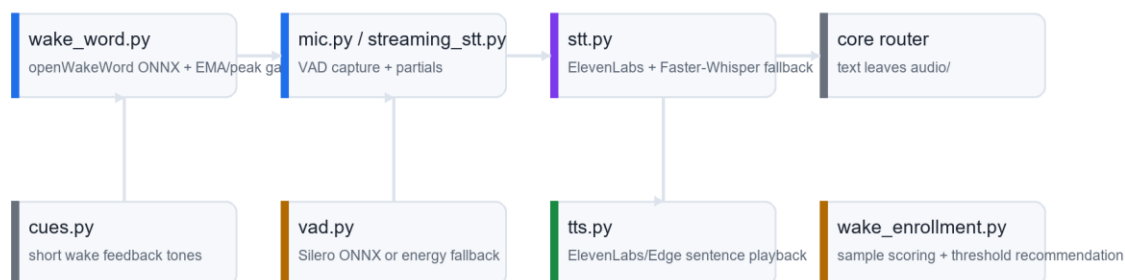
File	Responsibility
audio/cues.py	Generates short cached sine-wave cues and plays them through sounddevice when available.
audio/mic.py	Records utterances with VAD, pre-roll, adaptive silence, start timeout, and WAV output.
audio/streaming_stt.py	Records with VAD while producing partial STT windows and Arabic stability gating.
audio/stt.py	Owns STT backend selection, ElevenLabs cloud client, Faster-Whisper fallback, language lock, validation, and metadata.
audio/tts.py	SpeechEngine for hybrid ElevenLabs/Edge-TTS/console playback, sentence queue streaming, voice prep, and interruption.
audio/vad.py	Silero VAD ONNX wrapper with energy fallback and batch/streaming helpers.
audio/wake_enrollment.py	Records/scans user wake samples and prints threshold recommendations.
audio/wake_word.py	Unified bilingual wake-word listener using openWakeWord ONNX, EMA/peak/confirm gates, cooldown, and interrupt integration.

Table 4.4.1: Inventory of every source file in audio/.

Internal Data Flow

Figure 4.4.1 - audio/ runtime pipeline

Wake decision, VAD capture, STT, and TTS playback.



Training data/model provenance is deferred to Section 4.5; this section covers runtime audio behavior.

Figure 4.4.1: Audio runtime pipeline from wake detection through STT and TTS.

Key Modules In Depth

wake_word.py loads one unified openWakeWord-compatible ONNX model and applies a two-level decision gate: EMA-smoothed score plus raw peak score within the confirm window.

```
def _get_unified_model(model_path: str):
    global _unified_model, _unified_model_path

    candidate = _resolve_unified_model_path(model_path)
    candidate_text = str(candidate)
    if _unified_model is not None and _unified_model_path == candidate_text:
        return _unified_model

    try:
        from openwakeword.model import Model
    except Exception as exc:
        raise RuntimeError(
            "openwakeword is unavailable. Install openwakeword in the active environment."
        ) from exc

    try:
        _unified_model = Model(wakeword_models=[candidate_text],
                                inference_framework="onnx")
    except Exception as exc:
        raise RuntimeError(f"Failed to load unified wake-word model: {candidate}") from exc
    _unified_model_path = candidate_text

def listen_for_wake_word():
    global _last_detection_ts, _last_detection_audio

    if sd is None:
        raise RuntimeError(
            "sounddevice is unavailable. Install sounddevice in the active Python environment."
        ) from _SOUNDDEVICE_IMPORT_ERROR

    _log_deprecated_keys_once()
    runtime = get_runtime_wake_word_settings()
    threshold = float(runtime["threshold"])
    cooldown = float(runtime["detection_cooldown_seconds"])
```

Listing 4.4.1: Unified wake-word model loading.

```

confirm_frames = max(1, int(runtime["confirm_frames"]))
min_rms = max(0.0, float(runtime["min_rms"]))
peak_threshold = float(runtime.get("peak_threshold", threshold))
ema_window = max(1, int(runtime.get("ema_window", 1)))
ema_alpha = 2.0 / (ema_window + 1.0)
model = _get_unified_model(_runtime_wake_word_phrase_settings["unified_onnx_path"])

```

Listing 4.4.2: Runtime wake thresholds and EMA setup.

```

# Two-level gate: the smoothed (EMA) score must clear the lower
# trigger threshold, AND at least one raw frame within the
# confirm window must clear the higher peak threshold. This lets
# the trigger threshold sit low enough to catch real speech on
# the first try while still rejecting sustained-but-mild noise
# that would otherwise average past a single flat threshold.
frame_ok = ema_score >= threshold and rms >= min_rms
consecutive_hits = consecutive_hits + 1 if frame_ok else 0
if consecutive_hits < confirm_frames:
    continue
if max(recent_peak_scores) < peak_threshold:
    consecutive_hits = 0
    continue
consecutive_hits = 0
if now - _last_detection_ts < cooldown:
    continue

_last_detection_ts = now
_last_detection_audio = list(recent_audio)
_save_wake_activation_sample(recent_audio)

```

Listing 4.4.3: EMA, peak, confirm-frame, cooldown, and interrupt gate.

mic.py and streaming_stt.py capture WAV audio with pre-roll, start timeout, adaptive silence, and optional partial transcription windows.

```

def record_utterance(
    filename="input.wav",
    max_duration=MAX_RECORD_DURATION,
    vad_mode="command",
    energy_threshold=None,
    silence_seconds=None,
    min_speech_seconds=None,
    pre_roll_seconds=None,
    start_timeout_seconds=None,

```

```

        max_speech_seconds=None,
    ):
        if sd is None:
            raise RuntimeError(
                "sounddevice is unavailable. Install sounddevice in the active Python
environment."
            ) from _SOUNDDEVICE_IMPORT_ERROR

        print("[Mic] Listening (VAD)...")
        started_at = time.perf_counter()

        runtime = get_runtime_vad_settings()

        return last_text

    # Arabic stability gate: only emit if same text for 2 consecutive windows.
    # Whisper Arabic partials flicker more than English; this prevents
    # unstable fragments from triggering early intent detection.
    def _is_arabic_text(text):
        if text == self._ar_pending_partial:
            self._ar_pending_count += 1
        else:
            self._ar_pending_partial = text
            self._ar_pending_count = 1
        emit_threshold = 1 if _is_mixed_script_text(text) else 2
        if self._ar_pending_count >= emit_threshold and text != last_text:
            _safe_callback(self.on_partial, text)
            return text
        return last_text

    if text != last_text:
        _safe_callback(self.on_partial, text)

```

Listing 4.4.5: Arabic partial-transcript stability gate.

stt.py keeps runtime STT settings and metadata, builds the ElevenLabs HTTP client, validates language, and falls back to local Faster-Whisper when needed.

```

def get_cloud_http_client() -> httpx.Client:
    global _CLOUD_HTTP_CLIENT
    if _CLOUD_HTTP_CLIENT is not None:
        return _CLOUD_HTTP_CLIENT

```

```

with _CLOUD_HTTP_CLIENT_LOCK:
    if _CLOUD_HTTP_CLIENT is not None:
        return _CLOUD_HTTP_CLIENT
    timeout = httpx.Timeout(
        connect=float(STT_ELEVENLABS_CONNECT_TIMEOUT_SECONDS),
        read=float(STT_ELEVENLABS_READ_TIMEOUT_SECONDS),
        write=float(STT_ELEVENLABS_READ_TIMEOUT_SECONDS),
        pool=float(STT_ELEVENLABS_READ_TIMEOUT_SECONDS),
    )
    try:
        _CLOUD_HTTP_CLIENT = httpx.Client(http2=bool(STT_ELEVENLABS_HTTP2),
        timeout=timeout)
    except ImportError as exc:
        if not bool(STT_ELEVENLABS_HTTP2):
            raise
        logger.debug("HTTP/2 unavailable for ElevenLabs STT client; using HTTP/1.1:
%s", exc)
        _CLOUD_HTTP_CLIENT = httpx.Client(http2=False, timeout=timeout)

```

Listing 4.4.6: Reused ElevenLabs HTTP client with timeout settings.

```

def _runtime_language_hint() -> str:
    hint = str(_RUNTIME_STT_SETTINGS.get("language_hint", "auto") or
    "auto").strip().lower()
    if hint in {"ar", "arabic", "ar-eg", "ar_eg"}:
        return "ar"
    if hint in {"en", "english", "en-us", "en_us"}:
        return "en"

```

Listing 4.4.7: Runtime STT language hint normalization.

`vad.py` wraps Silero ONNX when available and rejects low-energy chunks before inference; if the `model/runtime` is missing, energy gating remains available.

```

class SileroVAD:
    def __init__(
        self,
        model_path=None,
        *,
        sample_rate=16000,
        threshold=0.5,
        energy_threshold=None,
        model_url=_DEFAULT_SILERO_MODEL_URL,
    ):
        self.sample_rate = int(sample_rate or SAMPLE_RATE or 16000)

```

```

if self.sample_rate not in {8000, 16000}:
    self.sample_rate = 16000
self.threshold = max(0.05, min(0.95, float(threshold)))
self.energy_threshold = max(0.001, float(
    _ENERGY_FALLBACK_THRESHOLD if energy_threshold is None else energy_threshold
))
self.window_size_samples = 512 if self.sample_rate == 16000 else 256
self.context_size = 64 if self.sample_rate == 16000 else 32
self.model_path = pathlib.Path(model_path or _DEFAULT_SILERO_MODEL_PATH)

```

Listing 4.4.8: Silero VAD initialization and fallback state.

```

def is_speech_chunk(self, audio_chunk_int16: np.ndarray) -> bool:
    """Process a single streaming chunk (512 int16 samples at 16 kHz → 32 ms)."""
    chunk = np.asarray(audio_chunk_int16, dtype=np.int16).reshape(-1)
    if chunk.size == 0:
        return False
    audio = chunk.astype(np.float32) / 32768.0
    if _compute_rms(audio) < float(self.energy_threshold):
        return False
    if self._fallback_only or self._session is None:
        return True
    result = self._run_silero_window(audio)
    return bool(result) if result is not None else True

```

Listing 4.4.9: Streaming VAD chunk decision.

tts.py speaks through SpeechEngine. Hybrid mode tries ElevenLabs first, then Edge-TTS, then console fallback; sentence-queue playback overlaps synthesis and playback for lower perceived latency.

```

def speak_sentence_queue(self, sentences_iterator, language=None):
    if sentences_iterator is None:
        return False, "No sentences provided."

    if not self.is_enabled():
        return False, "Speech output is disabled."

    self.interrupt()
    self._stop_event.clear()

    if not TTS_SENTENCE_STREAMING_ENABLED:
        return self._speak_sentence_queue_sequential(sentences_iterator, language)

    def _run_streaming_queue():

```

```
import numpy as np # type: ignore
```

```
queue_started = time.perf_counter()
sentence_gap_samples = 0
paragraph_gap_samples = 0
playback_rate = 24000
```

Listing 4.4.10: Sentence queue streaming worker.

```
def _run_speech(self, text, language=None, backend_override=None):
    started = time.perf_counter()
    success = True
    backend = str(backend_override or self._resolve_backend() or
"auto").strip().lower()
    quality_mode = self.get_quality_mode()
    style = persona_manager.get_speech_style()
    logger.info("Speech backend=%s quality=%s style=%s", backend, quality_mode,
style)
    spoken_text = self._prepare_text_for_speech(text, preferred_language=language)
    arabic_preferred = self._is_arabic_preferred_text(spoken_text,
preferred_language=language)

    try:
        if backend in {"auto", "hybrid"}:
            if self._speak_elevenlabs(spoken_text):
                return

            logger.info("ElevenLabs TTS unavailable; falling back to Edge-TTS")
            fallback_language = "ar" if arabic_preferred else "en"
            if self._speak_edge_tts(spoken_text,
preferred_language=fallback_language):
                return

            logger.warning("Edge-TTS synthesis failed; using console fallback")
            self._speak_console(spoken_text, prefix="TTS fallback")
```

Listing 4.4.11: Hybrid TTS fallback order.

wake_enrollment.py is a runtime support tool: it scores recorded samples against the deployed wake model and recommends a threshold based on the quietest usable sample.

```
def _save_calibration(username: str, scores: list[float], base_threshold: float) ->
pathlib.Path:
    """Compute and save calibration offset."""
    _CALIBRATION_DIR.mkdir(parents=True, exist_ok=True)
    calibration_path = _CALIBRATION_DIR / f"{username}.json"
```



```

if scores:
    avg_peak = float(np.mean(scores))
    min_peak = float(np.min(scores))
    # Offset = how much to shift the threshold so even the quietest sample
    # would trigger. We target the threshold at 85% of the minimum peak.
    target_threshold = min_peak * 0.85
    offset = target_threshold - base_threshold
else:
    avg_peak = 0.0
    min_peak = 0.0
    offset = 0.0

calibration = {
    "username": username,
    "recorded_at": time.strftime("%Y-%m-%dT%H:%M:%S"),

```

Listing 4.4.12: Wake enrollment threshold recommendation.

Algorithms

Wake-word decision:

```

load unified ONNX model
for each audio chunk:
    score = model.predict(chunk)
    ema_score = alpha * score + (1-alpha) * previous_ema
    frame_ok = ema_score >= threshold and rms >= min_rms
    require confirm_frames consecutive frame_ok chunks
    require max(raw_scores in confirm window) >= peak_threshold
    require cooldown has elapsed
    save recent audio and, if speaking/thinking, request interrupt

```

STT language lock and fallback:

```

normalize runtime language hint to ar/en/auto
transcribe with configured backend
validate transcript against allowed English/Egyptian-Arabic scripts and confidence
floors
on cloud failure or weak result, use local Faster-Whisper fallback when configured
record backend/language/confidence metadata for diagnostics

```

Sentence-level TTS streaming:

```

iterate sentences from LLM sentence buffer
submit synthesis futures to a bounded worker pool
as soon as the next future is ready, play waveform

```

insert sentence or paragraph gap samples
allow interrupt() to stop queued playback

Configuration Surface

Area	Config constants used by audio/
Wake word	WAKE_WORD_UNIFIED_ONNX_PATH, WAKE_WORD_THRESHOLD, WAKE_WORD_PEAK_THRESHOLD, WAKE_WORD_CONFIRM_FRAMES, WAKE_WORD_EMA_WINDOW, WAKE_WORD_CHUNK_SIZE, WAKE_WORD_MIN_RMS, WAKE_WORD_DETECTION_COOLDOWN_SECONDS, WAKE_WORD_RECORD_START_DELAY_MS, WAKE_WORD_INPUT_DEVICE, WAKE_WORD_USER_SAMPLES_DIR.
Recording/VAD	MAX_RECORD_DURATION, VAD_ENERGY_THRESHOLD, VAD_BACKEND, VAD_SILERO_THRESHOLD, VAD_COMMAND_SILENCE_SECONDS, VAD_CHAT_SILENCE_SECONDS, VAD_MIN_SPEECH_SECONDS, VAD_PREROLL_SECONDS, VAD_START_TIMEOUT_SECONDS.
STT	STT_BACKEND, ELEVENLABS_API_KEY, ELEVENLABS_BASE_URL, STT_LANGUAGE_HINT, STT_LANGUAGE_LOCK, STT_FORBID_OTHER_LANGUAGES, STT_MIN_CONFIDENCE, STT_ELEVENLABS_* timeout/cooldown/model flags, WHISPER_MODEL, WHISPER_DEVICE, WHISPER_COMPUTE_TYPE.
Streaming partials	STT_PARTIAL_MIN_SECONDS, STT_PARTIAL_WINDOW_SECONDS, STT_PARTIAL_INTERVAL_SECONDS, STT_PARTIAL_WHISPER_MODEL.
TTS	TTS_DEFAULT_BACKEND, TTS_QUALITY_MODE, TTS_SENTENCE_STREAMING_ENABLED, TTS_SENTENCE_SYNTH_WORKERS, TTS_SENTENCE_FIRST_FLUSH_MIN_CHARS, TTS_SENTENCE_GAP_MS, TTS_PARAGRAPH_GAP_MS, TTS_ELEVENLABS_TIMEOUT_SECONDS, TTS_EDGE_MIXED_SCRIPT_CHUNKING.
Cues/interrupts	WAKE_INTERRUPT_ACK_SOUND, WAKE_INTERRUPT_ACK_FREQ_HZ, WAKE_INTERRUPT_ACK_DURATION_MS, WAKE_INTERRUPT_BLOCKED_TONE_ENABLED,

	WAKE_INTERRUPT_BLOCKED_TONE_FREQ_HZ, WAKE_INTERRUPT_BLOCKED_TONE_DURATION_MS.
--	--

Table 4.4.2: Audio configuration surface imported by audio/ modules; defaults are centralized in core/config.py.

Behavior In Different Situations

Situation	Audio behavior	Evidence
Normal	Wake fires after EMA/peak/confirm/cooldown gates, recording captures speech until adaptive silence, STT returns text, and TTS streams sentences.	wake_word.py, mic.py, streaming_stt.py, stt.py, tts.py.
Degraded	Missing openwakeword/sounddevice raises honest runtime errors; missing Silero/onnxruntime falls back to energy VAD; ElevenLabs/Edge failures fall back to local or console paths.	wake_word.py, mic.py, vad.py, stt.py, tts.py.
Adversarial/Edge	Low-RMS noise cannot pass wake gate; sustained mild scores must also clear peak threshold; Arabic partials wait for stable repeated text; TTS can be interrupted mid-stream.	wake_word.py, streaming_stt.py, tts.py.

Table 4.4.3: Audio behavior under normal, degraded, and edge conditions.

Contribution Summary

audio/ underpins hands-free operation by turning microphone input into verified bilingual text and assistant text back into speech. Its degradation paths keep failures explicit, while the wake gate and language-aware partials reduce false activation and unstable early execution.

4.5 wake-word dataset and training

Purpose

The wake-word dataset/training artifacts document the offline evidence behind the unified on-device wake model. In this workspace the deployed-relevant folder is wake word data/jarvis_unified_training. It contains class-split WAV clips and precomputed feature tensors, but no separate eval/ or training/ source-code folder inside that target.

File and Subfolder Inventory

Artifact	Format	What it stores	Pipeline role
features/	directory	Precomputed NumPy	Consumes WAV splits; feeds

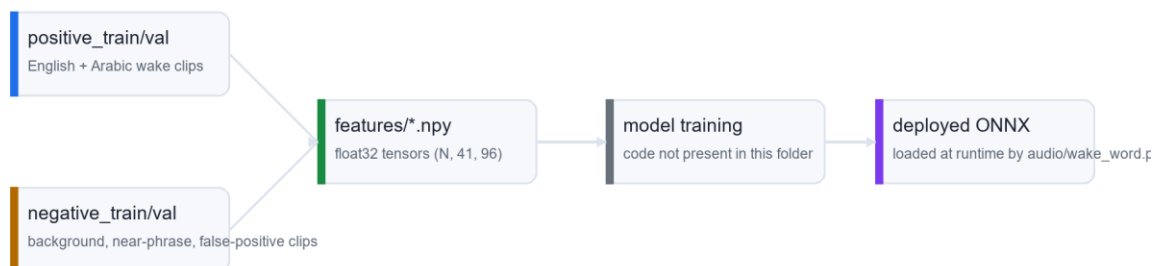
		feature tensors. Files: 4.	training/evaluation code not present in this folder.
negative_train/	directory	Negative/background training WAV clips. Files: 1107.	Source class for negative_features_train.npy.
negative_val/	directory	Negative validation/test WAV clips. Files: 274.	Source class for negative_features_test.npy.
positive_train/	directory	Positive wake-word training WAV clips. Files: 515.	Source class for positive_features_train.npy.
positive_val/	directory	Positive validation/test WAV clips. Files: 143.	Source class for positive_features_test.npy.
eval/, training/	absent	No separate eval/ or training/ code folder exists inside jarvis_unified_training.	Documented as absent rather than inferred.

Table 4.5.1: Observed artifacts in wake word data/jarvis_unified_training.

Internal Data Flow

Figure 4.5.1 - wake-word dataset/training artifacts

Observed unified wake-word folder: WAV splits plus precomputed feature tensors.



This phase documents data/provenance only. Runtime wake-decision logic is in Section 4.4.

Figure 4.5.1: Wake-word data artifacts from WAV splits to feature tensors and deployed ONNX boundary.

Key Artifacts In Depth

features/ contains four NumPy tensors. Every observed tensor uses float32 values and a per-sample shape of 41 by 96, which indicates fixed-size extracted wake-word features rather than raw waveform storage.

Feature file	Class	Split	Observed
--------------	-------	-------	----------

			shape/dtype/size
negative_features_test.npy	negative	test/validation	(1726, 41, 96), float32, 27174272 bytes
negative_features_train.npy	negative	train	(7173, 41, 96), float32, 112931840 bytes
positive_features_test.npy	positive	test/validation	(858, 41, 96), float32, 13508480 bytes
positive_features_train.npy	positive	train	(3090, 41, 96), float32, 48649088 bytes

Table 4.5.2: Precomputed feature tensors under features/.

The WAV split folders separate positive wake examples from negative/background examples and train from validation/test data. Filenames show both ASCII and non-ASCII names, reflecting English and Arabic phrase coverage without needing to open audio content.

Split folder	WAV files	ASCII filenames	Non-ASCII filenames	Total bytes
positive_train/	515	335	180	36885636
positive_val/	143	83	60	10122388
negative_train/	1107	1017	90	83132100
negative_val/	274	229	45	20896152

Table 4.5.3: WAV class/split counts in the unified wake-word dataset.

A small sample of WAV headers confirms mono 16 kHz clips, with positive examples around 2.48 seconds and negative background clips commonly longer. The table uses headers only, not audio playback or transcript assumptions.

Split	Sample file	Channels	Rate	Duration
positive_train	user_default_0063_wake_english_20260505-213702_335.wav	1	16000	2.480s
positive_val	user_default_0000_wake_english_20260504-153055_789.wav	1	16000	2.480s
negative_train	adaptive_fp_fp_20260624-193606_131.wav	1	16000	2.400s
negative_val	adaptive_fp_fp_20260624-193606_131.wav	1	16000	2.400s

Table 4.5.4: Header-level sample metadata from each WAV split.

No Python training loop, loss definition, threshold-selection report, or held-out metric file is present inside jarvis_unified_training. The folder therefore proves the existence and shape of the dataset/features, while the exact training/evaluation implementation remains outside this phase's read boundary.

Algorithms Reflected By The Data

Observed feature-preparation procedure:

```
collect positive wake clips under positive_train/ and positive_val/
collect negative/background clips under negative_train/ and negative_val/
extract fixed-size features from each WAV clip
```

```

write class/split arrays:
    positive_features_train.npy
    positive_features_test.npy
    negative_features_train.npy
    negative_features_test.npy
each row has feature shape 41 x 96

```

Observed train/validation balance:

```

positive train features: 3090
positive test features: 858
negative train features: 7173
negative test features: 1726
negative examples outnumber positive examples in both train and test splits
this supports false-wake resistance, but exact loss/threshold tuning is not present in
this folder

```

Configuration Surface

Surface	Observed value	Meaning
Dataset root	wake word data/jarvis_unified_training	Unified wake-word dataset/training artifact folder documented in this phase.
Feature format	*.npy float32 tensors	Precomputed features consumed by training/evaluation tooling.
Raw audio format	*.wav, mono 16 kHz in inspected samples	Source clips for positive/negative class splits.
Runtime deployment link	deployed ONNX model path is documented in Section 4.4	This phase documents data provenance; runtime loading is separate.
Absent folders	eval/ and training/ are not present inside this target	No local metric reports or training scripts are claimed from this folder.

Table 4.5.5: Dataset/training configuration surface visible from the artifact folder.

Behavior In Different Situations

Situation	Artifact behavior	Defense note
Normal	Positive/negative train and validation WAV splits exist, and features/ contains matching class/split tensors.	Enough provenance is present to discuss data balance and feature shapes.
Degraded	No eval/ or training/ code folder is present inside jarvis_unified_training.	Training loop and metric claims are intentionally marked as not present rather than inferred.
Adversarial/Edge	Negative feature rows and WAV	The data design visibly targets

	clips outnumber positive examples; filenames include background, ambient, near-phrase, and false-positive-style names.	false-wake resistance.
Reproducibility	Raw WAV and feature tensors are present, but no manifest/checksum/metric report was found directly in the target folder.	A later improvement would add a manifest and evaluation report next to the data.

Table 4.5.6: Wake-word dataset behavior under normal, degraded, and edge conditions.

Contribution Summary

The unified wake-word data artifacts support the hands-free objective by pairing English and Arabic positive examples with a larger negative/background set. The precomputed feature tensors make the data pipeline inspectable, while the absence of local training/eval code in this folder is an explicit reproducibility limitation to address before final archival.

4.6 nlp/

Purpose

The nlp/ package is Jarvis's text-understanding layer between recognized speech text and the command router. It normalizes noisy English/Arabic input, gives mixed-language commands a fast shortcut, offers semantic top-k route candidates, falls back to fuzzy keyword evidence, and enriches parsed commands with slots and entity types.

File Inventory

File	Responsibility
nlp/__init__.py	Package marker documenting NLP utilities for robust intent understanding on noisy STT output.
nlp/code_switch_router.py	Fast code-switch shortcut that maps verb+entity phrases to ParsedCommand objects before semantic routing.
nlp/codeswitching.py	Mixed Arabic/English normalization, verb/entity maps, script counts, Arabic numerals, and code-switched intent hints.
nlp/entity_types.py	Shared EntityType enum and value-to-enum lookup map.
nlp/fuzzy_matcher.py	Arabic/English text normalization plus exact/fuzzy keyword scoring for noisy STT text.
nlp/intent_classifier.py	Rule-based classifier over keyword/fuzzy matches with confidence, tie handling, and suggestions.
nlp/keyword_engine.py	Bilingual keyword catalog grouped by intent with action and target terms.

nlp/nlu.py	Entity extraction, slot enrichment, required-slot detection, and NLUResult packaging.
nlp/semantic_router.py	Sentence-transformers semantic route catalog and top-k confidence scoring.
nlp/text_normalizer.py	Not present in this workspace; normalization is documented from codeswitching.py and fuzzy_matcher.py instead.

Table 4.6.1: Inventory of Python files observed directly under nlp/.

Internal Data Flow

Figure 4.6.1 - nlp/ understanding flow

Folder-local text understanding stages before command execution.



No standalone nlp/text_normalizer.py is present; normalization evidence is in codeswitching.py and fuzzy_matcher.py.

Figure 4.6.1: NLP flow from normalized text through code-switch, semantic top-k, fuzzy/keyword, and slot extraction stages.

Key Modules In Depth

No standalone nlp/text_normalizer.py is present. The observed normalization responsibility is split: codeswitching.py handles Arabic letter folding, tashkeel stripping, Arabic-Indic numerals, script counts, and mixed Arabic/Latin entity extraction; fuzzy_matcher.py performs lower-level punctuation and whitespace cleanup for keyword matching.

```

def strip_tashkeel(text: str) -> str:
    """Remove Arabic diacritics (harakat / tashkeel) from text."""
    return _TASHKEEL_RE.sub("", str(text or ""))

def normalize_arabic_letters(text: str) -> str:
    """Normalize common Arabic letter variants for robust command matching.

    Maps:
    - ا → إ | آ | إ (alef variants)
  """

```



```

- ي → ي      (ya without dots)
- ق → ق      (qaf used in some dialects for qaf)
"""
return str(text or "").translate(_AR_LETTER_NORM)

```

```

def normalize_arabic(text: str) -> str:
    """Full Arabic pre-normalization: strip tashkeel + normalize letters + convert
    digits."""
    t = strip_tashkeel(str(text or ""))
    t = normalize_arabic_letters(t)

```

Listing 4.6.1: Arabic diacritic, letter, and numeral normalization helpers.

```

def normalize_codeswitched(text: str) -> dict:
    """Return a best-effort language/entity breakdown for mixed commands.

    Returns a dict with keys:
        original, arabic_segments, latin_segments,
        detected_verb, verb_intent, entity_text, language, numbers,
        intent, entity, normalized_entity, source_text
    """
    source = str(text or "").strip()
    # Apply Arabic normalization before tokenizing so "اكتب" → "اكتب" and
    # "افتح" → "افتح" are matched by the verb maps without separate entries.
    normalized_source = normalize_arabic(source)
    empty = {
        "original": source,
        "arabic_segments": [],
        "latin_segments": [],
        "detected_verb": "",
        "verb_intent": "",
        "entity_text": "",
        "language": "",

```

Listing 4.6.2: normalize_codeswitched() return shape and initial script/verb analysis.

code_switch_router.py is the fast shortcut for mixed-language command forms. It reuses verb sets from intent confidence, normalizes system actions, and emits ParsedCommand objects for open, close, delete, rename, move, and system-action paths when a verb/entity pattern is confident.

```

def try_codeswitch(normalized_text: str, language: str = "") -> Optional[ParsedCommand]:
    """Attempt to resolve a mixed-language verb+entity command without the
    semantic embedding model.

```

```

Returns a ParsedCommand on a confident resolution, else None (caller
falls through to the semantic tier).
"""

text = str(normalized_text or "").strip()
if not text:
    return None

normalized = " ".join(text.lower().split())

# System actions (volume/brightness/wifi/bluetooth/media/...) already have
# a robust bilingual + mixed-script normalizer – try it on the full phrase
# first since it doesn't require a clean verb/entity split.
action_key = normalize_system_action(text)
if action_key:
    return ParsedCommand(
        intent="OS_SYSTEM_COMMAND",
        Listing 4.6.3: try_codeswitch() system action and open/close mapping.

if verb in _DELETE_VERBS:
    # Delete only needs one target – resolve it (extension-less or not)
    # across the default search roots rather than requiring an exact path.
    resolved = _resolve_in_default_roots(entity)
    if resolved is not None:
        return ParsedCommand(
            intent="OS_FILE_NAVIGATION",
            raw=text,
            normalized=normalized,
            action="delete_item",
            args={"path": resolved},
        )
    return None

if verb in _MOVE_VERBS or verb in _RENAME_VERBS:
    # Both need a "<source> <sep> <target>" split – re-split from the
    # normalized text since the simple verb+entity split doesn't capture
    # the separator (to/as/الى...).
    target_split = _split_verb_entity_target(normalized)
    if target_split is None:
        Listing 4.6.4: try_codeswitch() delete, rename, and move mapping.

```

semantic_router.py loads sentence-transformers lazily and exposes classify_semantic_topk(). That API returns ranked intent names and scores; the older classify_semantic() wrapper accepts only the top result when it reaches the local 0.75 threshold.

```
# Confidence threshold – below this, fall through to next tier
SEMANTIC_CONFIDENCE_THRESHOLD = 0.75
```

```
def _ensure_loaded() -> bool:
    """Lazy-load the sentence-transformers model and build the route index.

    Returns True if ready, False if unavailable.
    """
    global _router, _loaded, _load_failed, ROUTER_READY

    if _loaded:
        return _router is not None
    if _load_failed:
        return False

    try:
        from sentence_transformers import SentenceTransformer
        import numpy as np
    except ImportError:
```

Listing 4.6.5: Semantic router availability and local confidence threshold.

```
def classify_semantic_topk(text: str, k: int = 3) -> list[Tuple[str, float]]:
    """Classify text against every route, returning the top-k (intent, score) pairs.

    Sorted descending by score. Each route contributes its single best-matching
    utterance similarity. Returns [] if the router isn't ready or on failure –
    callers should treat an empty list the same as "no match".
    """
    if not text or not is_router_ready() or _router is None:
        return []

    try:
        np = _router["np"]
        model = _router["model"]

        query_embedding = model.encode(
```

```
[text], normalize_embeddings=True, show_progress_bar=False,
)[0] # shape: (dim,)
```

```
scores = []
for route in _router["routes"]:
```

Listing 4.6.6: Semantic top-k scoring and back-compatible single-result wrapper.

fuzzy_matcher.py and keyword_engine.py provide the non-embedding fallback surface. keyword_engine.py stores bilingual intent keywords, while fuzzy_matcher.py scores exact and approximate containment. intent_classifier.py combines action and target matches, rejects weak evidence, and demotes near ties to unknown.

```
def normalize_text(text: str) -> str:
    """Normalize Arabic/English text for robust exact+fuzzy matching."""
    value = str(text or "").strip().lower()
    if not value:
        return ""

    value = _ARABIC_DIACRITICS_RE.sub("", value)
    value = value.replace("\u0640", "")
    value = value.translate(_ARABIC_CHAR_MAP)
    value = value.replace("_", " ").replace("-", " ")
    value = _PUNCT_RE.sub(" ", value)
    value = _SPACE_RE.sub(" ", value).strip()
    return value
```

```
def fuzzy_score(text: str, keyword: str) -> int:
```

Listing 4.6.7: Fuzzy matcher normalization and score entry point.

```
def fuzzy_contains(text: str, keyword: str, threshold: int = 70) -> bool:
    """True when keyword appears in text by exact or fuzzy partial matching."""
    return fuzzy_score(text, keyword) >= int(threshold)
```

```
def find_keyword_matches(text: str, keywords: Iterable[str], threshold: int = 70) ->
List[Tuple[str, int]]:
```

```
    """Return all keywords that match text with score >= threshold, sorted by score
    desc."""
```

```
    matches: List[Tuple[str, int]] = []
    for keyword in keywords:
        score = fuzzy_score(text, keyword)
```

```

        if score >= int(threshold):
            matches.append((str(keyword), int(score)))

    matches.sort(key=lambda item: item[1], reverse=True)
    return matches

```

Listing 4.6.8: Fuzzy keyword containment and sorted match collection.

```

def classify_intent(text: str) -> Dict[str, object]:
    """Classify noisy bilingual text into one intent using weighted keyword/fuzzy
    scoring."""
    normalized = normalize_text(text)
    if not normalized:
        return {"intent": "unknown", "confidence": 0.0, "matched_keywords": []}

    candidates: List[Dict[str, object]] = []
    for intent_name, payload in INTENTS.items():
        result = _evaluate_intent(normalized, intent_name, payload)
        if int(result["score"]) > 0:
            candidates.append(result)

    if not candidates:
        return {"intent": "unknown", "confidence": 0.0, "matched_keywords": []}

    candidates.sort(key=lambda item: (int(item["score"]), float(item["confidence"])),
                    reverse=True)
    best = candidates[0]

    # If two intents are nearly tied, avoid false confident classification.
    if len(candidates) > 1:

```

Listing 4.6.9: Keyword classifier acceptance and near-tie rejection.

nlu.py is the slot layer. It preserves existing parser args, then enriches missing app, duration, timer label, file, location, time, and reminder-message slots. entity_types.py names the small shared taxonomy consumed by route definitions.

```

def understand(
    self,
    text: str,
    language: Optional[str] = None,
    *,
    intent: str = "",
    existing_args: Optional[Dict[str, Any]] = None,

```

```

) -> NLUResult:
    """Return NLUResult for the given utterance and resolved intent.

    Only enriches slots that are absent in `existing_args`.
    """
    intent_upper = str(intent or "").strip().upper()
    domain = _INTENT_TO_DOMAIN.get(intent_upper, CHAT)
    lang = str(language or "").strip().lower()

    existing = dict(existing_args or {})
    entities = self._extract_entities(text, domain, intent_upper, existing)

    required = list(REQUIRED_SLOTS.get(intent_upper, []))

```

Listing 4.6.10: NLUResult assembly with required-slot detection.

```

def _extract_entities(
    self,
    text: str,
    domain: str,
    intent: str,
    existing: Dict[str, Any],
) -> Dict[str, Any]:
    result = dict(existing)

    # Codeswitching pre-pass: enrich app_name from mixed-language entity
    if domain == APP_CONTROL and not _truthy(result.get("app_name")) and not
    _truthy(result.get("app_query")):
        try:
            from nlp.codeswitching import normalize_codeswitched
            cs = normalize_codeswitched(text)
            cs_entity = str((cs or {}).get("entity_text") or "").strip()
            if cs_entity and cs_entity not in {"volume", "brightness", "timer",
            "files", "folder", "web"}:
                result["_cs_entity_hint"] = cs_entity
        except Exception:
            pass

```

Listing 4.6.11: Domain-specific entity enrichment.

```

from enum import Enum

class EntityType(str, Enum):

```

```

APP = "app"
SYSTEM_FEATURE = "system_feature"
PATH = "path"
NUMBER = "number"
DURATION = "duration"
EMAIL = "email"
PERSON = "person"
DATE = "date"

# Helper mapping for quick use
ENTITY_TYPE_MAP = {e.value: e for e in EntityType}

```

Listing 4.6.12: EntityType enum and lookup map.

Algorithms

Semantic margin acceptance:

```

topk = classify_semantic_topk(text, k=3)
if topk is empty: continue to fuzzy/keyword path
best = topk[0]
second = topk[1] if present else score 0.0
if best.score >= tau and (best.score - second.score) >= delta:
    accept best.intent
else:
    defer because the semantic evidence is weak or ambiguous

```

Code-switch resolution:

```

normalize Arabic letters, remove tashkeel, convert Arabic-Indic digits
tokenize and count Arabic vs Latin script segments
detect an Arabic or English verb in the early tokens
extract Latin app/entity text or Arabic mapped entity
try system-action normalization first
map verb families to ParsedCommand intents/actions
return None if the verb/entity pattern is not confident

```

Configuration Surface

Setting	Observed value	NLP role
SEMANTIC_CONFIDENCE_THRESHOLD	0.75 in nlp/semantic_router.py	Semantic wrapper accepts the best route only above this local threshold.
Semantic top-k k	classify_semantic_topk(text,	Caller can request multiple

	k=3)	candidate routes to apply a separate margin decision.
JARVIS_SEMANTIC_MIN_MARGIN	Imported/used by the core caller, not defined in nlp/	Margin guard for near ties; this phase observes top-k support in nlp/ and documents the caller rule without reading core.
MATCH_THRESHOLD	70 in nlp/intent_classifier.py	Minimum fuzzy keyword score for action/target matches.
MIN_INTENT_SCORE	1 in nlp/intent_classifier.py	Lowest accepted weighted keyword score.
MIN_CONFIDENCE	0.35 in nlp/intent_classifier.py	Lowest accepted keyword-classifier confidence.
SUGGEST_THRESHOLD	0.45 in nlp/intent_classifier.py	Minimum suggestion confidence from the looser suggest_intent path.
Entity types	app, system_feature, path, number, duration, email, person, date	Slot taxonomy declared by nlp/entity_types.py.

Table 4.6.2: NLP constants and imported configuration surfaces visible from nlp/.

Behavior In Different Situations

Situation	NLP behavior	Evidence
Normal	Mixed English/Arabic input is normalized, code-switch shortcuts can return ParsedCommand, semantic top-k ranks paraphrases, and NLU fills required slots.	codeswitching.py, code_switch_router.py, semantic_router.py, nlu.py.
Degraded	If sentence-transformers is missing, semantic_router.py logs the disabled state and returns no semantic candidates; fuzzy/keyword and code-switch modules remain available.	semantic_router.py _ensure_loaded(); fuzzy_matcher.py; intent_classifier.py.
Adversarial/Edge	Near keyword ties are returned as unknown, semantic callers can reject small margins, and question-shaped phrases such as informational Chrome questions are not claimed as app-open commands by nlp/ alone.	intent_classifier.py tie guard; semantic_router.py top-k support.
Code-switch example	A phrase such as mixed Arabic plus a Latin app name can produce an app entity hint or	codeswitching.py entity extraction; code_switch_router.py

	direct app command when the verb/entity pattern matches.	try_codeswitch(); nlu.py _cs_entity_hint.
--	--	--

Table 4.6.3: NLP behavior under normal, degraded, and adversarial conditions.

Contribution Summary

nlp/ advances the bilingual-assistant objective by making noisy and mixed-script speech actionable without forcing every utterance into the LLM path. Its layered design keeps high-confidence shortcuts cheap, gives semantic routing ranked evidence, uses keyword/fuzzy logic when embeddings are unavailable, and centralizes slot enrichment before execution decisions.

4.7 llm/

Purpose

The llm/ package is Jarvis's answer-generation and LLM-assisted fallback layer. It builds a single prompt package for an answer turn, streams local Ollama output through a sentence buffer for TTS, and also contains constrained structured/tool-call paths for ambiguous commands. This section documents only llm/ source and templates; the memory content inserted into prompts is cross-referenced to Section 4.3.

File Inventory

File	Responsibility
llm/ollama_client.py	Local Ollama generate client, runtime model settings, streaming, keep-alive, fallback model attempts, think-tag filtering, and cancel checks.
llm/prompt_builder.py	Prompt package assembly with persona, language pinning, prompt tiers, memory block insertion, KB context, few-shot caps, and num_ctx autosizing.
llm/prompts/full_prompt.txt	High-tier prompt template with examples and language/style constraints.
llm/prompts/micro_prompt.txt	Minimal/low-tier prompt template with compact examples.
llm/prompts/slim_prompt.txt	Medium-tier prompt template used by the default prompt tier map.
llm/sentence_buffer.py	Language-aware streaming chunk accumulator for English and Arabic TTS boundaries.
llm/structured_nlu.py	Strict JSON command-understanding fallback over Ollama chat, intended to be gated by the caller and verified before execution.
llm/tool_caller.py	Ollama tool-call schema, optional Claude tool format bridge, and conversion from tool calls to ParsedCommand objects.
llm/claude_client.py	Not present as source in this workspace;

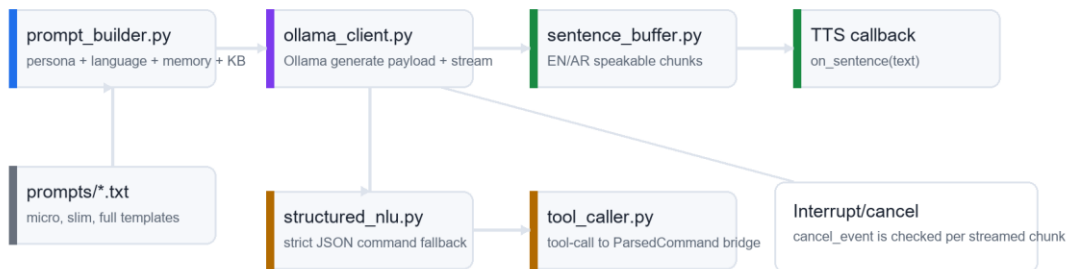
	tool_caller.py contains an optional import path but this phase does not document a source file that is absent.
--	--

Table 4.7.1: Inventory of llm/ Python modules and prompt templates.

Internal Data Flow

Figure 4.7.1 - llm/ answer and streaming flow

Prompt package, local model stream, sentence chunking, and optional structured/tool fallback.



Prompt memory contents are cross-referenced to Section 4.3; this phase documents only how llm/ inserts the block.

Figure 4.7.1: LLM flow from prompt package through Ollama streaming and sentence-buffered TTS callbacks.

Key Modules In Depth

ollama_client.py constructs the /api/generate payload with model, prompt, stream mode, keep_alive, num_ctx, temperature, top_p, repeat_penalty, num_predict, and stop tokens. For qwen3 thinking-mode models, it also disables thinking where supported and appends /no_think if needed.

```

def _decode_stop_tokens(tokens):
    decoded = []
    for token in tokens or []:
        value = str(token or "")
        if not value:
            continue
        decoded.append(value.replace("\\n", "\n").replace("\\t", "\t"))
    return decoded

def _build_request_payload(model_name, prompt, num_ctx, stream):
    """Construct an Ollama /api/generate payload, suppressing thinking when needed.

    Adds keep_alive=30m so the model stays resident in RAM between user queries
  """

```

```

(default Ollama keep_alive is 5m which causes cold-loads after a brief idle).
"""
effective_prompt = str(prompt or "")
payload = {
    "model": model_name,
    "prompt": effective_prompt,

```

Listing 4.7.1: Ollama request payload options, keep-alive, stop tokens, and think suppression.

```

def ask_llm_streaming(prompt, on_sentence=None, num_ctx=None, is_arabic=False,
cancel_event=None):
    """Stream tokens from Ollama; call on_sentence(text) at each sentence boundary.

    Returns the complete accumulated response text, or an error string.
    Falls back to non-streaming ask_llm() when on_sentence is None.

    Args:
        is_arabic: When True, uses Arabic-aware sentence splitting (splits on :.؟
                    and performs soft/hard char-count flushes for un-punctuated text).
        cancel_event: Optional threading.Event checked after each token chunk.
                    When set, the stream is closed and partial text returned.
    """
    if on_sentence is None:
        return ask_llm(prompt, num_ctx=num_ctx)

    if cancel_event is None:
        cancel_event = llm_cancel_event

    started = time.perf_counter()
    success = False

    for raw_line in stream_response.iter_lines():
        if cancel_event is not None and cancel_event.is_set():
            logger.info("LLM stream cancelled by wake-word interrupt.")
            break

    elapsed = time.perf_counter() - started
    if elapsed >= hard_timeout_seconds:
        hard_timeout_hit = True
        logger.error(
            "LLM streaming hard-timeout after %.2fs (model=%s)",
            elapsed,

```

Listing 4.7.2: Streaming entry point, cancel-event default, SentenceBuffer, and payload creation.

```

        model_name,
    )
    break

    line = (raw_line or "").strip()
    if not line:
        continue
    try:
        chunk = json.loads(line)

```

Listing 4.7.3: Stream loop with cancel checks, hard timeout, think-tag filtering, and sentence callbacks.

```

        "LLM streaming timeout after %.2fs (model=%s)", latency, model_name
    )
    partial = "".join(accumulated).strip()
    if partial:
        success = True
        return partial
    fallback = ask_llm(prompt, num_ctx=num_ctx)
    if fallback and "timed out" not in str(fallback).lower():
        try:
            on_sentence(str(fallback))
        except Exception:
            pass
        success = True
        return str(fallback)
    return "The local model timed out. Try a shorter query."
except httpx.ConnectError:
    logger.error("Cannot connect to Ollama at %s. Is it running?", _OLLAMA_BASE_URL)
    fallback = ask_llm(prompt, num_ctx=num_ctx)
    if fallback and "cannot connect to ollama" not in str(fallback).lower():
        try:

```

Listing 4.7.4: Streaming timeout/connect-error handling and non-streaming fallback attempt.

`prompt_builder.py` owns prompt size and structure. It maps model tiers to prompt templates, estimates token counts, autosizes `num_ctx` under the runtime ceiling, injects persona/language rules, inserts exactly one memory prompt block when enabled, and returns a package containing prompt text, context metadata, and selected `num_ctx`.

```

_PROMPTS_DIR = pathlib.Path(__file__).parent / "prompts"

_TIER_TO_TEMPLATE = {
    "minimal": "micro_prompt.txt",

```

```

"low":      "micro_prompt.txt",
"medium":   "slim_prompt.txt",
"high":     "full_prompt.txt",
}

```

Listing 4.7.5: Prompt tier to template mapping.

```

def _estimate_token_count(text):
    """Rough estimate of token count (1 token ≈ 4 chars for English, 1-2 for Arabic).

    Args:
        text: string to estimate token count for

    Returns:
        Approximate token count
    """
    # Rough heuristic: English ~4 chars per token, Arabic ~2 chars per token
    # Count as ~1 Arabic char for every alef-lam-baa type character
    arabic_char_count = sum(1 for c in text if ord(c) > 0x0600 and ord(c) < 0x06FF)
    english_char_count = len(text) - arabic_char_count
    estimated_tokens = (english_char_count / 4.0) + (arabic_char_count / 2.0)
    return int(estimated_tokens)

def _tier_ctx_ceiling(tier="medium"):
    """Return the runtime context ceiling for this turn.

```

Listing 4.7.6: Token estimate and num_ctx autosizing.

```

def build_prompt_package(user_text, response_language="en", tier="medium"):
    query = (user_text or "").strip()
    response_language = _normalize_response_language(response_language)
    # Auto-select tier from runtime when caller uses the default.
    effective_tier = get_prompt_tier() if tier == "medium" else tier
    kb_package = knowledge_base_service.retrieve_for_prompt(
        query,
        top_k=KB_TOP_K,
        max_chars=KB_MAX_CONTEXT_CHARS,
    )
    kb_context = kb_package["context"]
    compact_kb_context = " ".join(str(kb_context or "").split()).strip()

    sections = _build_system_block(response_language, tier=effective_tier)

```

```

# Phase 8 – single MEMORY CONTEXT block (working slots + preferences +
# recent turns + bounded semantic recall), replacing the old separate
# MEMORY:/CONTEXT: lines so memory appears exactly once in the prompt.
memory_block = ""
if MEMORY_PROMPT_BLOCK_ENABLED:

```

Listing 4.7.7: Prompt package assembly with memory and knowledge context.

sentence_buffer.py turns streamed token fragments into speakable chunks. English and Arabic use different boundary character sets and soft/hard word limits, and connector holding prevents chunks from ending on conjunction-like words.

```

_EN_BOUNDARY_CHARS = {".", "!", "?", "\n"}
_AR_BOUNDARY_CHARS = {".", "!", "?", ":", ";", ",", "£", "\n"}
_CONNECTORS_EN = {"and", "then"}
_CONNECTORS_AR = {"ثم", "ف", "و"}
_WORD_RE = re.compile(r"\S+")

```

```

class SentenceBuffer:
    """Token-by-token accumulator that emits complete speakable chunks."""

    def __init__(
        self,
        is_arabic: bool = False,
        soft_flush_chars: Optional[int] = None,
        hard_flush_chars: Optional[int] = None,
        *,
        en_soft_words: Optional[int] = None,
        en_hard_words: Optional[int] = None,
        ar_soft_words: Optional[int] = None,
        ar_hard_words: Optional[int] = None,

```

Listing 4.7.8: SentenceBuffer constants and runtime thresholds.

```

    def _check_language(self, *, boundary_chars: set[str], soft_words: int, hard_words:
int, connectors: set[str]) -> Optional[str]:
        text = self._buf
        word_count = self._word_count(text)
        boundary_pos = self._last_boundary_pos(text, boundary_chars)

        if boundary_pos >= 0 and word_count >= soft_words:
            candidate = text[: boundary_pos + 1].strip()

```

```

        if candidate and not self._ends_with_connector(candidate, connectors):
            self._buf = text[boundary_pos + 1 :].lstrip()
            return candidate

    if word_count >= hard_words:
        if self.hold_connectors and self._ends_with_connector(text, connectors):
            return None
        cut = self._hard_cut_pos(text, hard_words)
        candidate = text[:cut].strip() if cut > 0 else ""
        if self.hold_connectors and candidate and self._ends_with_connector(candidate, connectors):
            next_cut = self._hard_cut_pos(text, hard_words + 1)
            if next_cut > cut:
                cut = next_cut

```

Listing 4.7.9: Boundary flush, hard flush, and connector holding.

structured_nlu.py is a strict JSON fallback, not a direct executor. Its module docstring states that the caller gates it and passes successful output through verification. The file grounds the prompt in known schema names, asks for one JSON object, coerces confidence into 0.0-1.0, and returns None on network, status, JSON, or missing-intent failures.

```

def _intent_catalog() -> str:
    """One line per schema intent: name, domain, required slots – keeps the
    system prompt grounded in the real schema instead of letting the model
    invent intent names."""
    lines = []
    for name, spec in sorted(SCHEMA.items()):
        slots = ", ".join(spec.required_slots) or "none"
        lines.append(f"- {name} (domain={spec.domain}, required_slots={slots})")
    return "\n".join(lines)

def _build_system_prompt() -> str:
    return (
        "You are a strict command-understanding classifier for a voice assistant. "
        "Given the user's utterance, respond with ONLY a single JSON object matching "
        "this exact schema – no prose, no markdown fences, no explanation:\n"
        "{\n"
        '  "intent": "<one of the known intents below, or LLM_QUERY if none fit>","\n'
        '  "action": "<short action string, or empty>","\n'
        '  "slots": {"<slot_name>": "<value>"}, ...},\n'

```

Listing 4.7.10: Structured NLU prompt grounded in known intents.

```

def understand_structured(
    normalized_text: str,
    language: str = "en",
    timeout: float = 4.0,
    model_name: Optional[str] = None,
) -> Optional[dict]:
    """Ask the LLM for a strict-JSON intent+slots classification.

    Returns a dict with keys _RESPONSE_KEYS on success, or None on any
    failure (network error, non-200, unparseable JSON, missing intent).
    Never raises – callers should treat None the same as "couldn't classify".
    """
    text = str(normalized_text or "").strip()
    if not text:
        return None

    payload = {
        "model": str(model_name or LLM_MODEL or "qwen3:4b"),
        "messages": [
            {"role": "system", "content": _build_system_prompt()},

```

Listing 4.7.11: Strict JSON request, failure handling, and coercion boundary.

tool_caller.py defines a default tool schema for app, file, timer, search, volume, brightness, lock, sleep, and screenshot commands. It can call Ollama chat with tools, normalize returned tool calls, optionally convert the schema to Claude's input_schema format, and map recognized tool calls into ParsedCommand objects.

```
_CHAT_ENDPOINT = f"{_OLLAMA_BASE_URL}/api/chat"
```

```

def build_default_tools() -> List[Dict[str, Any]]:
    return [
        {
            "type": "function",
            "function": {
                "name": "open_app",
                "description": "Open a desktop application by name or alias.",
                "parameters": {
                    "type": "object",
                    "properties": {"app_name": {"type": "string"}},
                    "required": ["app_name"],

```



```

        },
    },
},
{
    "type": "function",
    "function": {

```

Listing 4.7.12: Default tool schema excerpt.

```

        arguments = json.loads(arguments)
    except Exception:
        arguments = {"text": arguments}
    return {
        "name": str(function.get("name") or call.get("name") or "").strip(),
        "arguments": dict(arguments or {}),
    }

def call_tool_tier(
    user_text: str,
    *,
    model_name: Optional[str] = None,
    tools: Optional[Sequence[Dict[str, Any]]] = None,
) -> Dict[str, Any]:
    payload = {
        "model": _resolve_model_name(model_name),
        "messages": [
            {
                "role": "system",

```

Listing 4.7.13: Ollama chat tool-call request and normalization.

```

def _ollama_tools_to_claude_format(tools: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """Convert Ollama-style tool definitions to Claude's input_schema format."""
    result = []
    for t in tools:
        fn = t.get("function") or t
        result.append({
            "name": fn["name"],
            "description": fn.get("description", ""),
            "input_schema": fn.get("parameters") or {"type": "object", "properties":
        {}},
    })
    return result

```

```

def call_tool_tier_claude(
    user_text: str,
    *,
    tools: Optional[Sequence[Dict[str, Any]]] = None,
) -> Dict[str, Any]:
    """Tool-use call via Claude API. Returns same dict shape as call_tool_tier()."""
    from llm.claude_client import _get_client

```

Listing 4.7.14: Optional Claude tool-call bridge; source file is absent in this workspace.

```

        "OS_FILE_SEARCH",
        raw_text,
        normalized,
        args={
            "filename": args.get("filename", ""),
            "search_path": args.get("search_path", ""),
        },
    )
)
elif name == "open_folder":
    parsed_commands.append(
        ParsedCommand(
            "OS_FILE_NAVIGATION",
            raw_text,
            normalized,
            action="list_directory",
            args={"path": args.get("path", "")},
        )
    )
elif name == "set_timer":

```

Listing 4.7.15: Tool calls converted to ParsedCommand objects.

Prompt Templates

Template	Size	First nonblank line
llm/prompts/full_prompt.txt	17 lines / 659 chars	{ar_rule}
llm/prompts/micro_prompt.txt	9 lines / 312 chars	{ar_rule}
llm/prompts/slim_prompt.txt	17 lines / 521 chars	{ar_rule}

Table 4.7.2: Prompt template files under llm/prompts/.

Algorithms

num_ctx autosize:

```
estimate tokens from English and Arabic character counts
ceiling = runtime context ceiling for the selected model tier
if LLM_CTX_AUTOSIZE is disabled: use ceiling
if tokens <= 256: select 512
else if tokens <= 512: select 1024
else if tokens <= 1024: select 2048
else if tokens <= 2048: select 4096
else: select ceiling
return selected value clamped between 512 and ceiling
```

sentence-boundary flush:

```
append streamed token to buffer
choose English or Arabic boundary set
if latest boundary exists and word count reaches soft limit:
    flush through boundary unless the candidate ends with a connector
if word count reaches hard limit:
    hold when configured and buffer ends with a connector
    otherwise flush at the hard-word position
on stream end or timeout with partial text: flush remainder
```

Configuration Surface

Setting	LLM role observed in llm/
LLM_MODEL	Default model name used by ollama_client.py, structured_nlu.py, and tool_caller.py when no runtime model is set.
LLM_FALLBACK_MODELS	Additional non-streaming model candidates tried by ollama_client.py.
LLM_OLLAMA_BASE_URL	Base URL used to build /api/generate and /api/chat endpoints.
LLM_OLLAMA_NUM_CTX	Fallback context ceiling used by runtime getters and prompt_builder.py.
LLM_TEMPERATURE / LLM_TOP_P / LLM_REPEAT_PENALTY	Generation options inserted into Ollama request payloads.
LLM_MAX_RESPONSE_TOKENS	Mapped to Ollama num_predict.
LLM_STOP_TOKENS	Decoded into the Ollama stop-token list.
LLM_TIMEOUT_SECONDS	HTTP timeout and streaming hard-timeout input.
LLM_CTX_AUTOSIZE	Enables prompt-size-based num_ctx selection in prompt_builder.py.
LLM_FEWSHOT_MIN / LLM_FEWSHOT_MAX	Bounds few-shot example counts per prompt tier.
LLM_LANG_PIN_ENABLED	Controls language pin insertion and older

	template hint filtering.
MEMORY_PROMPT_BLOCK_ENABLED / MEMORY_MAX_CONTEXT_CHARS	Controls whether prompt_builder.py inserts one bounded memory block.
KB_TOP_K / KB_MAX_CONTEXT_CHARS	Bounds retrieved knowledge-base context in prompt_builder.py.
SENTENCE_BUFFER_*	Controls English/Arabic soft and hard word limits plus connector holding in sentence_buffer.py.
CLAUDE_DEFAULT_MODEL	Imported only by tool_caller.py's optional Claude tool-call path.

Table 4.7.3: Configuration constants imported by llm/ modules; defaults are centralized outside this phase.

Behavior In Different Situations

Situation	LLM behavior	Evidence
Normal	Prompt builder creates a tiered prompt package with persona/language/memory/KB context; Ollama streams tokens; SentenceBuffer emits speakable chunks to on_sentence.	prompt_builder.py, ollama_client.py, sentence_buffer.py.
Degraded	Streaming returns honest local-model errors on non-200, timeout, empty response, or Ollama connection failure; non-streaming ask_llm fallback is attempted where coded.	ollama_client.py ask_llm_streaming().
Optional fallback	Tool-caller contains a Claude tool-call path, but llm/claude_client.py is absent as source in this workspace, so this phase documents only the optional import boundary.	tool_caller.py call_tool_tier_claude(); llm/ file inventory.
Adversarial/Edge	Runaway output is bounded by num_predict, stop tokens, request timeout, and hard streaming timeout; qwen3 think tags are filtered; wake-word interrupt can set the cancel event and stop the stream loop.	ollama_client.py _build_request_payload(); ask_llm_streaming().
Structured command ambiguity	structured_nlu.py returns strict JSON or None and is documented as caller-gated/verifier-bound rather than directly executable.	structured_nlu.py module docstring and understand_structured().

Table 4.7.4: LLM behavior under normal, degraded, optional, and adversarial conditions.

Contribution Summary

llm/ advances the assistant by separating answer generation from deterministic command routing while still providing controlled fallbacks for ambiguous language. Prompt tiers let smaller models use shorter instructions, memory and KB context are inserted once, streaming chunks reach TTS quickly, and cancellation/timeout/stop-token controls keep generation bounded.

4.8 os_control/

Purpose

os_control/ is the Windows side-effect boundary behind the routing and safety layers. It contains the adapters that open apps, manipulate files, change system state, manage timers/reminders, capture the screen, and record audit evidence. The folder's recurring contract is: decide policy/risk, ask confirmation when required, execute through a constrained adapter, verify state where the adapter supports readback, then return a structured success/failure/confirmation result.

File Inventory

File	Responsibility
os_control/__init__.py	Package marker for OS control helpers.
os_control/action_log.py	Append/read/verify/reseal facade for action audit logs.
os_control/adapter_result.py	Shared success/failure/confirmation response shape and router tuple conversion.
os_control/app_ops.py	Application catalog, app resolution, open/close execution, and confirmed app close payloads.
os_control/app_scanner.py	Installed application discovery from registry, Start Menu shortcuts, App Paths, Store apps, cache, and watcher hooks.
os_control/batch_ops.py	Batch command manager with disallowed sensitive intents.
os_control/calendar_ops.py	Outlook COM calendar draft creation; opens an event window for user confirmation.
os_control/capture_ops.py	Screenshot and screen recording wrapper over native capture, ffmpeg, and Game Bar paths.
os_control/clipboard_ops.py	Clipboard read/write/clear with pyperclip availability checks.
os_control/confirmation.py	Pending action confirmation manager, PIN sentinel flow, expiry, cancellation, and attempt handling.
os_control/email_ops.py	Email draft creation through Outlook COM or Gmail fallback; does not send mail.
os_control/explorer_ops.py	Open/reveal/open-file behavior in Windows Explorer with location and fuzzy filename resolution.

os_control/file_ops.py	Validated file navigation, search, create, move, copy, rename, delete, rollback, and confirmation payloads.
os_control/job_queue.py	Persistent delayed command queue over the shared persistence database.
os_control/native_ops.py	Native Windows helpers for volume, brightness, lock/sleep, screenshots, and read-back verification.
os_control/note_ops.py	Dictated note saving to a configured note directory with automatic names.
os_control/path_resolver.py	Human folder aliases, drive aliases, path resolution, and path humanization.
os_control/persistence.py	SQLite-backed action log, confirmations, rollback stack, and job queue persistence.
os_control/policy.py	Policy engine for command category enable/disable decisions.
os_control/powershell_bridge.py	Vetted PowerShell template registry and non-arbitrary template runner.
os_control/radio_ops.py	Wi-Fi, Bluetooth, and airplane mode control through WinRT first and PowerShell fallback.
os_control/reminder_ops.py	Windows reminder scheduling through Task Scheduler with in-process fallback.
os_control/risk_policy.py	System, file, and app operation risk-tier mapping plus coverage validation.
os_control/screen_context.py	Foreground/visible-window and optional screenshot-vision screen description.
os_control/search_index.py	SQLite file search index worker and query service.
os_control/second_factor.py	Spoken PIN/passphrase normalization, hashing, verification, and lockout tracking.
os_control/settings_ops.py	Windows Settings page resolver and ms-settings launcher.
os_control/sysinfo_ops.py	Battery and system status helpers.
os_control/system_ops.py	System command registry, alias normalization, confirmation request, native/template dispatch, and result shaping.
os_control/temporal_parser.py	Natural datetime and recurrence parser for timer/reminder-style commands.
os_control/timer_ops.py	Countdown timers, alarms, persistence, re-arming, and notification playback.
os_control/windows_toggles.py	Night Light, Do Not Disturb, Energy Saver, and Live Captions toggles with verification/fallback behavior.

Table 4.8.1: Inventory of every Python file directly under os_control/.

Internal Data Flow

Figure 4.8.1 - os_control/ verified side-effect flow

Routed action, policy/risk, confirmation/PIN, adapter execution, state verification, and audit logging.

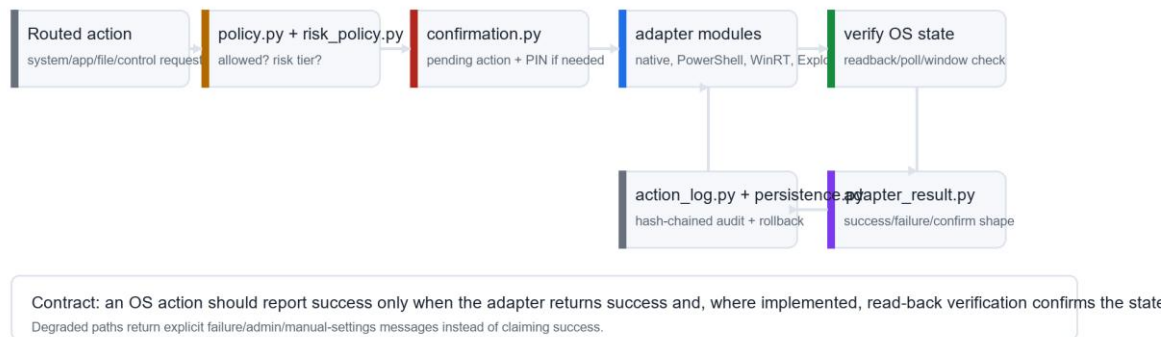


Figure 4.8.1: Verified OS side-effect flow from routed action through policy, confirmation, adapter execution, verification, and audit logging.

Key Modules In Depth

system_ops.py is the system-command hub. SYSTEM_COMMANDS declares command metadata such as template name, description, destructive status, and confirmation requirement.

request_system_command_result() applies the policy gate, normalizes args, routes non-confirmed commands directly, and creates a pending confirmation for confirmed commands.

```
SYSTEM_COMMANDS = {
    "shutdown": {
        "template": "shutdown",
        "description": "Shut down this computer",
        "destructive": True,
    },
    "restart": {
        "template": "restart",
        "description": "Restart this computer",
        "destructive": True,
    },
    "sleep": {
        "template": "sleep",
        "description": "Put this computer to sleep",
        "destructive": False,
        "requires_confirmation": True,
    },
}
```

```

},
"lock": {
    "template": "lock",
    "description": "Lock this computer",

```

Listing 4.8.1: SYSTEM_COMMANDS registry excerpt.

```

    if action_key not in SYSTEM_COMMANDS:
        return failure_result("Unsupported system command.",
            error_code="unsupported_action")

    if not policy_engine.is_command_allowed("system_command"):
        return failure_result("System commands are disabled by policy.",
            error_code="policy_blocked")

    normalized_args, args_error = _normalize_system_command_args(action_key,
        command_args)
    if args_error:
        return failure_result(args_error, error_code="invalid_input")

    cfg = SYSTEM_COMMANDS[action_key]
    requires_confirmation = bool(cfg.get("requires_confirmation", cfg["destructive"]))

    if not requires_confirmation:
        return execute_system_command_result(action_key, command_args=normalized_args,
            language=language)

    require_second_factor = bool(cfg["destructive"] and
        SECOND_FACTOR_REQUIRED_FOR_DESTRUCTIVE)
    risk_tier = risk_tier_for_system(
        action_key,
        destructive=bool(cfg.get("destructive")),

```

Listing 4.8.2: System command policy, risk, confirmation, and second-factor request path.

```

def execute_system_command_result(action_key, command_args=None, language=None):
    if action_key not in SYSTEM_COMMANDS:
        return failure_result("Unsupported system command.",
            error_code="unsupported_action")

    normalized_args, args_error = _normalize_system_command_args(action_key,
        command_args)
    if args_error:
        return failure_result(args_error, error_code="invalid_input")

```



```

cfg = SYSTEM_COMMANDS[action_key]
if cfg["destructive"] and not ALLOW_DESTRUCTIVE_SYSTEM_COMMANDS:
    msg = (
        "Blocked by configuration. Set ALLOW_DESTRUCTIVE_SYSTEM_COMMANDS=True "
        "in core/config.py to enable this command."
    )
    log_action(
        "system_command",
        "blocked",
        details={"action": action_key, "reason": "destructive_disabled"},
    )
    return failure_result(msg, error_code="destructive_disabled",
        debug_info={"action": action_key})

```

Listing 4.8.3: System command execution with destructive block and native-first dispatch.

native_ops.py is the low-level control adapter for volume, brightness, lock/sleep, and capture. The volume setter is explicit about the system-volume contract: pycaw and PowerShell media-key paths are eligible, while waveOut is excluded from system-volume verification because it changes this process's volume rather than the master endpoint.

```

def set_volume(level: float):
    """Set the SYSTEM (master) volume to a 0.0-1.0 float level.

    Returns True only if the OS-reported volume was read back and actually
    matches the requested level (within 2%). ``waveOutSetVolume`` sets only
    the calling process's volume, not the system master – it must never be
    used to claim system volume changed, so it is excluded from this path
    entirely (see set_app_volume for that legacy behavior, explicitly opted
    into via VOLUME_BACKEND="app").
    """
    target = float(_clamp(float(level), 0.0, 1.0))

    if VOLUME_BACKEND == "app":
        return set_app_volume(target)

    # Both setters here operate on the real system volume: pycaw via the
    # master audio endpoint, PowerShell via OS media keys (SendKeys) – the
    # same as pressing the hardware volume buttons. waveOutSetVolume is
    # deliberately excluded: it only changes this process's own volume.
    for setter in (_set_volume_pycaw, _set_volume_powershell):

```

Listing 4.8.4: System volume set with read-back verification and waveOut exclusion.

```

def get_brightness():
    """Return the current screen brightness as a 0-100 integer, or None."""
    for getter in (_get_brightness_wmi, _get_brightness_sbc,
                  _get_brightness_powershell):
        try:
            value = getter()
            if value is not None:
                return int(_clamp(int(value), 0, 100))
        except Exception as exc:
            logger.debug("Brightness read fallback failed: %s", exc)
    return None

```

```

def set_brightness(level: int):
    """Set the screen brightness to a 0-100 integer."""
    target = int(_clamp(int(level), 0, 100))
    # Prefer screen_brightness_control first because it often handles more
    # laptop/monitor combinations correctly than raw WMI.
    for setter in (_set_brightness_sbc, _set_brightness_wmi,
                  _set_brightness_powershell):
        try:
            if setter(target):

```

Listing 4.8.5: Brightness getter/setter chain with read-back check.

radio_ops.py uses WinRT first for Wi-Fi and Bluetooth, then PowerShell fallback. Both paths report success only after the requested state is read back when verification is enabled. Airplane mode snapshots existing radio state and treats partial restore/change as failure.

```

Returns True only after the requested state is read back and confirmed –
a set_state_async() call that doesn't throw is not itself success.
"""
try:
    target = _normalize_radio_kind(kind_name)
    radios = _winrt_get_radios()
    matched = [r for r in radios if _normalize_radio_kind(r.kind.name) == target]
    if not matched:
        logger.debug("No WinRT radio found for kind=%s", kind_name)
        return False
    for radio in matched:
        _run_async(_set_radio_state_async(radio, on))
except Exception as exc:
    logger.debug("WinRT set_radio(%s, %s) failed: %s", kind_name, on, exc)

```

```

        return False

    if not bool(CONTROLS_VERIFY_STATE):
        return True

    actual = _poll_until(lambda: _winrt_radio_is_on(kind_name), on)
    """
    Listing 4.8.6: WinRT radio set with polling verification.
    kind: 'wifi' | 'bluetooth'
    Returns True on success, False on failure (never raises).
    """
    kind_lower = kind.lower()
    winrt_name = "Wi-Fi" if kind_lower == "wifi" else "Bluetooth"

    if _winrt_available():
        ok = _winrt_set_radio(winrt_name, on)
        if ok:
            logger.info("WinRT radio %s -> %s", kind, "on" if on else "off")
            return True
        logger.debug("WinRT radio toggle failed, falling back to PowerShell")

    # PowerShell fallback
    if kind_lower == "wifi":
        ok = _ps_wifi(on)
    else:
        ok = _ps_bluetooth(on)

    if ok:

```

Listing 4.8.7: WinRT-to-PowerShell radio fallback and airplane-mode all-or-nothing result.

windows_toggles.py covers Windows 11 toggles that do not share one public API. Night Light and Do Not Disturb use registry write/read-back paths with Settings URI fallback; Energy Saver uses registry or powercfg paths; Live Captions uses hotkey/window detection and settings fallback.

```

# Setting it to 0x02 (on) or 0x00 (off) and writing back toggles the feature.
# Source: multiple community reverse-engineering references.

```

```

_NL_KEY = (
    r"SOFTWARE\Microsoft\Windows\CurrentVersion\CloudStore\Store"
    r"\DefaultAccount\Current\default$windows.data.bluelightreduction"
    r".bluelightreductionstate\windows.data.bluelightreduction.bluelightreductionstate"
)

```

```

_NL_VALUE = "Data"
_NL_ENABLE_BYTE_INDEX = 18

def _night_light_registry(on: bool) -> bool:
    data = _reg_get_binary(winreg.HKEY_CURRENT_USER, _NL_KEY, _NL_VALUE)
    if data is None or len(data) < _NL_ENABLE_BYTE_INDEX + 1:
        logger.debug("Night light blob not found or too short (len=%s)", len(data) if
data else 0)
        return False

    blob = bytearray(data)
    blob[_NL_ENABLE_BYTE_INDEX] = 0x02 if on else 0x00

```

Listing 4.8.8: Night Light registry write, shell notification, verification, and Settings fallback.

```

# Windows 11 DND is controlled by the "QuietHoursSettings" DWORD under
# HKCU\...\Notifications\Settings. Value 0 = off, value 1 = Priority only,
# value 2 = Alarms only (maximum DND).
# We use value 2 for "on" (most restrictive, silences toasts) and 0 for off.

_DND_KEY = r"SOFTWARE\Microsoft\Windows\CurrentVersion\Notifications\Settings"
_DND_VALUE = "NOC_GLOBAL_SETTING_TOASTS_ENABLED"
_DND_FOCUS_KEY =
r"SOFTWARE\Microsoft\Windows\CurrentVersion\CloudStore\Store\DefaultAccount\Current\defa
ult$windows.data.notifications.quiethourssettings\windows.data.notifications.quiethourss
ettings"
_DND_FOCUS_VALUE = "Data"

def _dnd_registry(on: bool) -> bool:
    # Primary: disable/enable toast notifications globally
    # 0 = toasts disabled (DND on), 1 = toasts enabled (DND off)
    ok = _reg_set_dword(
        winreg.HKEY_CURRENT_USER,
        _DND_KEY,
        _DND_VALUE,
        0 if on else 1,
    )

```

Listing 4.8.9: Do Not Disturb registry write/read-back and fallback.

file_ops.py, path_resolver.py, and explorer_ops.py are the human-file layer. Path resolver maps spoken folder and drive aliases to concrete paths and short speakable names. Explorer operations can open or

reveal paths. File write operations validate policy and paths, resolve bare names from configured roots, return ambiguity instead of guessing, and create confirmation payloads for move, rename, delete, and copy.

```
def resolve_location(spoken: str) -> Optional[Path]:
    """Resolve a spoken location phrase to an absolute Path.

    Returns None when the phrase is not a recognisable location so callers
    can fall back to treating it as a literal path.
    """
    if not spoken:
        return None
    key = spoken.strip().lower()

    # 1. Known folder alias (Desktop, Downloads, ...)
    folder_name = FOLDER_ALIASES.get(key)
    if folder_name:
        return KNOWN_FOLDERS.get(folder_name)

    # Strip Arabic definite-article prefix and retry ("الـ desktop" → "desktop").
    stripped = _AR_ARTICLE_RE.sub("", key).strip()
    if stripped != key:
        folder_name = FOLDER_ALIASES.get(stripped)
        if folder_name:
            Listing 4.8.10: Spoken location resolution and humanized path output.

def _fuzzy_find(filename: str, search_root: Path, *, recursive: bool = False) -> Path |
None:
    """Case-insensitive filename search within *search_root*.

    When *recursive* is True, walks all subdirectories after failing the
    shallow pass. Capped at 5 000 entries to stay fast.
    """
    try:
        target = filename.lower()
        stem = re.sub(r"\.[a-zA-Z0-9]{1,5}$", "", target)
        has_ext = target != stem # filename had an extension
        # When user says "CV PDF" (space, no dot), last token may be the extension
        # and first tokens form the name stem, e.g. "cv pdf" → name="cv", ext="pdf"
        ext_hint: str = ""
        name_stem: str = stem
```

```

if not has_ext and " " in stem:
    parts = stem.split()
    _ext_like = {"pdf", "doc", "docx", "txt", "xlsx", "pptx", "mp4", "mp3",
                "png", "jpg", "jpeg", "zip", "py", "js", "ts", "md", "csv"}
    if parts[-1] in _ext_like:
        ext_hint = parts[-1]

```

Listing 4.8.11: Explorer fuzzy filename matching with extension-optional support and scan cap.

```

def request_move_item(source, destination):
    write_ok, write_reason = _validate_file_write_enabled()
    if not write_ok:
        return failure_result(write_reason, error_code="policy_blocked")

    ok, reason, src, dst = _prepare_move_paths(source, destination)
    if not ok and reason.startswith("Source does not exist"):
        # Source not found as a literal path – search the default roots
        # (mirrors request_rename_item's fallback) so "move report to
        # downloads" resolves without needing "move report.pdf to downloads".
        for root_name in FILE_DEFAULT_SEARCH_ROOTS:
            folder = KNOWN_FOLDERS.get(root_name)
            if not folder:
                continue
            status, value = resolve_name_in_location(source, str(folder))
            if status == "single":
                ok, reason, src, dst = _prepare_move_paths(value, destination)
                if ok:
                    break
            elif status == "ambiguous":

```

Listing 4.8.12: File move/rename/delete/copy request paths and confirmation payloads.

app_scanner.py and app_ops.py keep application control separate from file navigation. app_ops.py blocks generic folder words from resolving as apps, scores app candidates by aliases, running state, availability, and usage, refreshes a stale catalog on miss, and requires confirmation for app-close operations.

```

def resolve_app_candidates(app_name, limit=5, operation="open"):
    query = _normalize_alias(app_name)
    if not query:
        return []

    if query in KNOWN_APPS:
        executable = KNOWN_APPS[query]

```

```

return [
    {
        "canonical_name": _EXECUTABLE_TO_CANONICAL.get(executable, executable),
        "executable": executable,
        "matched_alias": query,
        "score": 1.0,
        "similarity": 1.0,
        "usage_count":
int(session_memory.get_app_usage_stats(executable).get("count") or 0),
        "is_running": _to_process_name(executable).lower() in
_running_process_names_snapshot(),
        "available": _is_executable_available(executable),
    }
]

```

Listing 4.8.13: App candidate scoring, folder-word blacklist, ambiguity margin, and refresh-on-miss.

```

def request_close_app_result(app_name):
    if not app_name:
        return failure_result("No app name provided.", error_code="invalid_input")
    if not policy_engine.is_command_allowed("app_close"):
        return failure_result(
            "Application close is disabled by policy.",
            error_code="policy_blocked",
        )

    resolved = _resolve_close_target(app_name)
    if not resolved.get("ok"):
        return resolved.get("response")

    risk_tier = risk_tier_for_app_operation("close_app")
    payload = {
        "kind": "app_operation",
        "operation": "close_app",
        "resolved_args": {
            "target": resolved["target"],

```

Listing 4.8.14: App close confirmation payload.

confirmation.py and second_factor.py implement the spoken PIN path. The current flow stores a single pending PIN action behind a fixed sentinel, normalizes spoken digits, checks lockout state, compares hashes with hmac.compare_digest(), discards expired or locked actions, and logs accepted or rejected attempts.

```
"""Store the pending sensitive action and return the PIN_REQUIRED sentinel.
```

Replaces the old hex-token flow: nothing here is spoken back to the user, so there is no token to leak or mistype.

```
"""
```

```
global _pin_pending_action
```

```
now_ts = time.time()
```

```
expires_at = now_ts + max(5, int(self.pin_timeout_seconds or 30))
```

```
with _pin_lock:
```

```
    _pin_pending_action = {
```

```
        "action_name": action_name,
```

```
        "description": description,
```

```
        "payload": payload,
```

```
        "created_at": now_ts,
```

```
        "expires_at": expires_at,
```

```
    }
```

```
logger.info("PIN confirmation requested for %s", action_name)
```

```
return _PIN_SENTINEL
```

Listing 4.8.15: Pending PIN action creation and verify_pin_and_execute().

```
"""Normalize a spoken PIN (digits or EN/AR number-words) to a digit string."""
```

```
value = str(text or "").strip().translate(_ARABIC_INDIC_DIGITS)
```

```
if not value:
```

```
    return ""
```

```
collapsed = re.sub(r"[\s-]+", "", value)
```

```
if collapsed.isdigit():
```

```
    return collapsed
```

```
digits = []
```

```
for token in re.findall(r"[a-zA-Z\u0600-\u06FF-_\d]+", value.lower()):
```

```
    if token.isdigit():
```

```
        digits.append(token)
```

```
        continue
```

```
    mapped = _EN_DIGIT_WORDS.get(token) or _AR_DIGIT_WORDS.get(token)
```

```
    if mapped is not None:
```

```
        digits.append(mapped)
```

```
return "".join(digits)
```

Listing 4.8.16: Spoken PIN normalization, attempt limits, and second-factor verification.

risk_policy.py centralizes risk labels for system, file, and app actions. adapter_result.py standardizes success, failure, and confirmation responses. persistence.py stores a hash-chained action log, pending confirmations, rollback actions, and background jobs in SQLite.

```
SYSTEM_RISK_OVERRIDES = {
    "lock": "medium",
    "sleep": "medium",
}

FILE_OPERATION_RISK = {
    "move_item": "medium",
    "rename_item": "medium",
    "delete_item": "high",
    "delete_item_permanent": "high",
}

APP_OPERATION_RISK = {
    "close_app": "medium",
}
```

```
def risk_tier_for_system(action_key, *, destructive=False, requires_confirmation=False):
    if not action_key:
        return "low"
```

Listing 4.8.17: Risk-tier mappings and coverage validation.

```
from __future__ import annotations

from typing import Any

def build_adapter_result(
    success: bool,
    user_message: str,
    error_code: str = "",
    debug_info: dict[str, Any] | None = None,
    **extra: Any,
) -> dict[str, Any]:
    payload: dict[str, Any] = {
        "success": bool(success),
        "user_message": str(user_message or ""),
```

```

        "error_code": str(error_code or ""),
        "debug_info": dict(debug_info or {}),
    }
    payload.update(extra)
    return payload

```

Listing 4.8.18: Adapter result and router tuple response shape.

```

def _ensure_db():
    global _initialized
    if _initialized:
        return

    with _db_lock:
        if _initialized:
            return
        with _connect() as conn:
            conn.execute(
                """
                CREATE TABLE IF NOT EXISTS action_logs (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    timestamp TEXT NOT NULL,
                    action_type TEXT NOT NULL,
                    status TEXT NOT NULL,
                    details_json TEXT NOT NULL,
                    rollback_data_json TEXT,
                    error TEXT,
                    prev_hash TEXT,

```

Listing 4.8.19: SQLite tables for action logs, confirmations, rollback, and jobs.

The remaining adapters broaden the OS surface without changing the safety pattern: `timer_ops.py` persists/re-arms timers, `reminder_ops.py` uses Task Scheduler with in-process fallback, `capture_ops.py` writes screenshots/recordings under configured dirs, `clipboard_ops.py` fails honestly when `pyperclip` is unavailable, `email_ops.py` drafts but does not send mail, `note_ops.py` writes `.txt` notes, `screen_context.py` describes visible windows or uses screenshot vision, and `search_index.py` keeps a file index.

Adapter group	Modules	Observed behavior
System state	<code>system_ops.py</code> , <code>native_ops.py</code> , <code>radio_ops.py</code> , <code>windows_toggles.py</code> , <code>powershell_bridge.py</code>	Constrained command registry, native-first execution, vetted PowerShell templates, WinRT/registry/hotkey paths, and read-back checks where implemented.

Files and Explorer	file_ops.py, path_resolver.py, explorer_ops.py, search_index.py	Human folder aliases, extension-optional search, ambiguity handling, confirmed writes, rollback storage, and indexed search.
Applications	app_scanner.py, app_ops.py	Static plus scanned catalog, usage/running/availability scoring, folder-word blocklist, and confirmed app close.
Safety and audit	confirmation.py, second_factor.py, policy.py, risk_policy.py, persistence.py, action_log.py, adapter_result.py	Policy blocks, risk tiers, PIN/attempt lockout, structured adapter results, hash-chained action log, confirmations, rollback, and jobs.
Personal productivity	timer_ops.py, reminder_ops.py, clipboard_ops.py, email_ops.py, note_ops.py, calendar_ops.py	Timers/reminders, clipboard operations, mail/calendar drafts, and note saving with explicit failure paths.
Screen and status	capture_ops.py, screen_context.py, settings_ops.py, sysinfo_ops.py	Screenshot/recording, visible-window description, Settings URI opening, and battery/system status.

Table 4.8.2: *os_control/ adapter families.*

Algorithms

Verify-after-execute contract:

```

normalize action and arguments
check policy and destructive/confirmation gates
execute through the safest available adapter
if CONTROLS_VERIFY_STATE is enabled and readback exists:
    poll or read OS state after execution
    if actual state is unavailable or mismatched: report failure or try next backend
log success, blocked, pending, or failed action
return adapter_result success/failure/confirmation payload

```

PIN confirmation flow:

```

create pending action with PIN_REQUIRED sentinel and expiry
when user speaks PIN: normalize digit words and Arabic-Indic digits
if no pending action or expired: reject and clear state
if confirmation attempts are locked: discard pending action
compare hashed spoken secret to configured PIN/passphrase hash
on success: discard pending action, clear attempts, return stored payload
on repeated failure: lock out and discard pending action

```

Risk-tier decision:

```

system action: explicit override wins
else destructive system command is high risk
else confirmation-required system command is medium risk
file delete/permanent delete is high; move/rename are medium
app close is medium
unmapped operation defaults to low, but validate_risk_policy_coverage can flag required
mappings

```

Configuration Surface

Setting	Role visible from os_control/
ALLOW_DESTRUCTIVE_SYSTEM_COMMANDS	Blocks destructive system commands in system_ops.py unless enabled.
SECOND_FACTOR_REQUIRED_FOR_DESTRUCTIVE / SECOND_FACTOR_PIN	Controls whether destructive confirmations require spoken PIN verification.
CONFIRMATION_TIMEOUT_SECONDS / SENSITIVE_PIN_PENDING_TIMEOUT_SECONDS	Bounds pending confirmation/PIN action lifetime.
SECOND_FACTOR_MAX_ATTEMPTS_PER_TOKEN / SECOND_FACTOR_LOCKOUT_SECONDS	Controls second-factor failed-attempt lockout behavior.
CONFIRMATION_MAX_ATTEMPTS_PER_TOKEN / CONFIRMATION_LOCKOUT_SECONDS	Controls confirmation attempt rate limiting.
CONTROLS_VERIFY_STATE	Controls read-back verification in native_ops.py and radio_ops.py.
CONTROLS_ADMIN_HINT	Allows system_ops.py to add an administrator hint when network radio actions fail without elevation.
FEATURE_FLAGS	Gates native volume and media direct-dispatch paths in system_ops.py and auto app discovery in app_ops.py.
VOLUME_BACKEND	Allows explicit app-volume mode; system volume path otherwise excludes waveOut readback.
FILE_DEFAULT_SEARCH_ROOTS / DEFAULT_SEARCH_PATH / DEFAULT_WORKING_DIRECTORY	Drive file search and current-directory defaults in file_ops.py.
FILE_HUMANIZE_PATHS / FILE_SPEAK_PATHS / MAX_FILE_RESULTS	Control file response wording and search result bounds.
ALLOW_PERMANENT_DELETE / ROLLBACK_DIR_NAME	Control permanent delete policy and soft-delete rollback storage.
APP_* resolution constants	Tune app catalog TTL, refresh-on-miss, and scoring bonuses in app_ops.py.
TOGGLE_* methods / LIVE_CAPTION_HOTKEY	Select registry/hotkey/settings fallback behavior in windows_toggles.py.
AIRPLANE_RESTORE_RADIOS	Controls whether airplane-off restores the pre-airplane radio snapshot.
SCREENSHOT_DIR / SCREENRECORD_*	Configure capture_ops.py screenshot and

	recording destinations/backends.
NOTE_DIR / NOTE_BASENAME	Configure note_ops.py save location and default naming.
TIMER_*	Configure timer persistence, Clock app launch, and TTS announcement behavior.
ACTION_LOG_FILE	Path used by action_log.py's text append facade.
JOB_MAX_RETRIES_DEFAULT	Default retry count used by job_queue.py.

Table 4.8.3: Configuration symbols imported or referenced by os_control/ modules; defaults are outside this phase boundary.

Behavior In Different Situations

Situation	os_control behavior	Evidence
Normal	A routed action passes policy/risk checks, executes through the matching adapter, verifies state where supported, logs the result, and returns a structured adapter payload.	system_ops.py, native_ops.py, radio_ops.py, adapter_result.py, action_log.py.
Degraded	If WinRT radio control is unavailable or fails, radio_ops.py falls back to PowerShell; if registry toggle does not verify, windows_toggles.py opens Settings URI and returns False; if pyperclip or Outlook COM are unavailable, clipboard/email paths return explicit fallback/error messages.	radio_ops.py, windows_toggles.py, clipboard_ops.py, email_ops.py.
Elevation/Admin	Network radio failures can receive an admin hint from system_ops.py when configured; file and destructive system operations are blocked by policy/config rather than forced.	system_ops.py, file_ops.py.
Adversarial/Edge	Wrong PIN attempts are rate-limited and can discard the pending action; permanent delete is blocked unless enabled; ambiguous file/app resolution returns ambiguity instead of choosing silently; read-back mismatch reports failure.	second_factor.py, confirmation.py, file_ops.py, app_ops.py, native_ops.py.
Auditability	Action logs are stored with previous hash plus payload digest, confirmations and	persistence.py, action_log.py, adapter_result.py.

	rollback actions share the persistence database, and adapter_result.py preserves metadata for the router.	
--	---	--

Table 4.8.4: os_control behavior under normal, degraded, elevation, adversarial, and audit situations.

Contribution Summary

os_control/ advances verified control and safety objectives by keeping real Windows side effects behind explicit adapters, policy/risk gates, confirmation and PIN checks, read-back verification, and audit persistence. The folder also makes failure states useful: a control that cannot verify, lacks permissions, hits ambiguity, or fails a PIN does not claim success.

4.9 tools/ and 4.10 utils/

4.9 tools/

Purpose

tools/ contains small side-effect-light helper tools used to enrich answers or shortcut deterministic calculations. The runtime live-data path detects weather or web-search needs, fetches only the relevant tool output, formats nonempty results into labeled voice-ready prompt blocks, and returns an empty string when no live data is needed or a fetch fails.

File Inventory

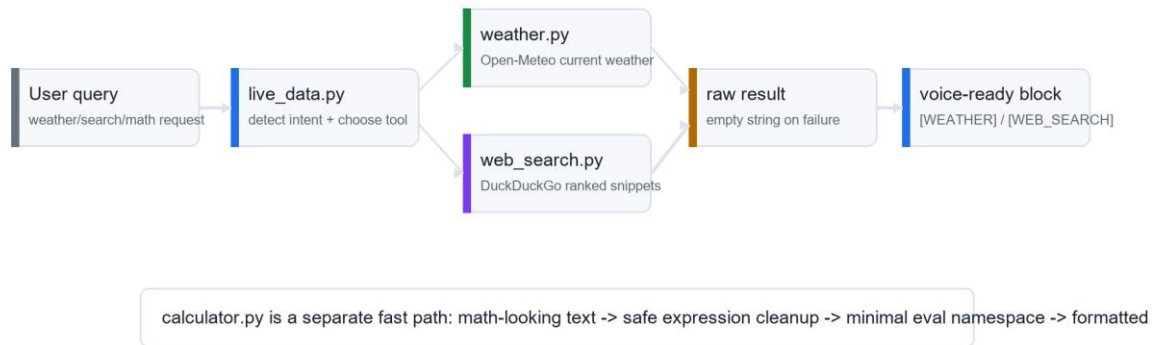
File	Responsibility
tools/__init__.py	Tools package marker.
tools/calculator.py	Safe quick calculator for math-looking expressions, including Arabic-Indic digit normalization.
tools/evaluate_nlu.py	Decision-only routing evaluation harness for intent accuracy, slots, latency, and safety counters.
tools/live_data.py	Weather/search intent detector, parallel live-data aggregator, and voice-ready labeled block formatter.
tools/weather.py	Open-Meteo current-weather client with WMO weather-code text mapping and empty-string failure behavior.
tools/web_search.py	DuckDuckGo text search wrapper with timeout, blocklist filtering, trusted-domain boost, recency boost, and formatted snippets.

Table 4.9.1: Inventory of Python files directly under tools/.

Internal Data Flow

Figure 4.9.1 - tools/ live data flow

Query detection, tool execution, raw result handling, voice-ready normalization, and prompt context.



Voice normalizer internals are outside this phase; tools/live_data.py only calls the normalizer before returning blocks.

Figure 4.9.1: Live data path from query detection through weather/search tools and voice-ready result blocks.

Key Modules In Depth

weather.py calls the Open-Meteo forecast endpoint for current temperature, humidity, weather code, and wind speed. It maps WMO weather codes to short English condition text and returns an empty string on non-200 status or exceptions.

```
_OPEN_METEO_URL = "https://api.open-meteo.com/v1/forecast"
_TIMEOUT_SECONDS = 5.0
```

```
# WMO Weather interpretation codes → human-readable text
```

```
_WEATHER_CODES = {
    0: "Clear sky",
    1: "Mainly clear",
    2: "Partly cloudy",
    3: "Overcast",
    45: "Foggy",
    48: "Depositing rime fog",
    51: "Light drizzle",
    53: "Moderate drizzle",
    55: "Dense drizzle",
    56: "Light freezing drizzle",
    57: "Dense freezing drizzle",
    61: "Slight rain",
    63: "Moderate rain",
```

```
65: "Heavy rain",
66: "Light freezing rain",
```

Listing 4.9.1: Open-Meteo current weather request and empty-string failure behavior.

web_search.py wraps DuckDuckGo text search. It runs the provider call in a worker with a hard timeout, drops blocked domains, scores remaining results with base rank, trusted-domain boost, and recency boost, then formats title/body/domain/date snippets.

```
def _score_result(result, now_ts: float) -> float:
    """Combined ranking signal: trust + recency + intrinsic order."""
    base_score = float(result.get("_base_rank") or 0.0)
    domain = _normalize_domain(result.get("href") or result.get("url") or "")
    trust_boost = (
        float(WEB_SEARCH_TRUSTED_DOMAIN_BOOST) if _is_trusted_domain(domain) else 0.0
    )

    published_ts = 0.0
    for field in _DATE_FIELD_CANDIDATES:
        published_ts = _parse_published_timestamp(result.get(field))
        if published_ts > 0:
            break

    recency_boost = float(WEB_SEARCH_RECENCY_BOOST) * _recency_factor(published_ts,
now_ts)

    return base_score + trust_boost + recency_boost
```

```
def _filter_and_rank(raw_results, max_results: int):
    """Drop blocklisted hosts, score the rest, return top-N rows."""
```

Listing 4.9.2: Web result scoring, filtering, ranking, and formatting.

```
def search_web(query: str, max_results: int = 3) -> str:
    """Return formatted search results as context for LLM prompt injection.
```

Returns an empty string on failure so callers can fall back gracefully.

Phase 2.7: results are filtered against ``WEB\_SEARCH\_BLOCKED\_DOMAINS``, boosted when the host appears in ``WEB\_SEARCH\_TRUSTED\_DOMAINS`` and ranked by recency when the backend exposes a publication date.

```
"""
if not _DDGS_AVAILABLE:
    logger.warning("duckduckgo-search not installed – web search unavailable")
```



```

    return ""

query = (query or "").strip()
if not query:
    return ""

requested = max(1, int(max_results))
# Pull a few extra rows so the trust/recency reranker has headroom even
# after blacklist filtering.

```

Listing 4.9.3: DuckDuckGo search timeout and graceful failure boundary.

live_data.py is the aggregator. It detects weather intent with English and Arabic terms, extracts a city when possible, detects web-search intent while excluding file-search phrases, optionally honors a force_search flag, and runs weather/search futures in parallel.

```

def _detect_weather_intent(query_text: str) -> Optional[Dict]:
    """Detect if query asks about weather."""
    query = (query_text or "").lower().strip()
    arabic_weather_keywords = (
        "\u0637\u0642\u0633",
        "\u0627\u0644\u0637\u0642\u0633",
        "\u0627\u0644\u062c\u0648\u0648",
        "\u0623\u0628\u0627\u0627\u0631 \u0627\u0644\u0637\u0642\u0633",
        "\u0627\u0628\u0627\u0627\u0631 \u0627\u0644\u0637\u0642\u0633",
        "\u0623\u0628\u0627\u0627\u0631 \u0627\u0644\u062c\u0648\u0648",
        "\u0627\u0628\u0627\u0627\u0631 \u0627\u0644\u062c\u0648\u0648",
    )
    weather_keywords = [
        "weather", "forecast", "temperature", "temp", "cold", "hot", "rain",
        "snow", "clouds", "sunny", "cloudy", "humidity", "wind",
        # Bilingual Arabic – include both "طقس" alone and "الطقس" with the ال
        # article. Users say "طقس الاسكندرية" (no article) just as often as
        # "ايه الطقس النهارده".
        "طقس", "الطقس", "درجة الحرارة", "درجة", "برد", "حر",
        "مطر", "ثلج", "غيوم", "حرارة", "الحرارة",
    ]

```

Listing 4.9.4: Weather and web-search intent detection.

```

def _format_block(tool_kind: str, body: str, language: str = "en") -> str:
    """Wrap a tool result in a labeled block + per-tool instruction.

```

The label gives the LLM a clear anchor it can refer back to ("according to the weather block...") and the framing line steers it toward verbatim use.

```

"""
text = str(body or "").strip()
if not text:
    return ""
if tool_kind == "weather":
    text = normalize_weather_block(text, language)
elif tool_kind == "search":
    text = normalize_search_block(
        text,
        language,
        max_results=int(VOICE_NORMALIZER_MAX_SEARCH_RESULTS or 2),
    )
framing = _TOOL_FRAMING.get(tool_kind, "")
label = _TOOL_LABELS.get(tool_kind, f"[{tool_kind.upper()}]")
if framing:

```

Listing 4.9.5: Voice-ready result block formatting and parallel fetch orchestration.

calculator.py provides a fast deterministic math path. It only evaluates text that looks mathematical, normalizes Arabic-Indic digits and phrases, strips unsafe characters, rejects dangerous names, evaluates with only the math module and empty builtins, and returns None when it should fall through.

```

def quick_calc(expression: str) -> Optional[str]:
    """Evaluate a math expression safely and return a formatted result.

    Returns None if the text does not look like a math expression, so the
    caller can fall through to the LLM tier unchanged.
    """
    if not expression:
        return None
    if not _LOOKS_LIKE_MATH_RE.search(expression):
        return None

    # Step 1: normalise Arabic-Indic digits and % → %
    expr = str(expression).translate(_ARABIC_TO_LATIN)

    # Step 2: apply phrase substitutions in order
    for pattern, replacement in _PHRASE_SUBS:
        expr = pattern.sub(replacement, expr)

    # Step 3: strip characters not in the allowed set.
    # Letters are allowed so "math.sqrt" survives; _DANGER_RE is the security gate.

```

Listing 4.9.6: Safe quick calculation gate and minimal eval namespace.

`evaluate_nlu.py` is a developer harness rather than a live-data tool. It loads routing cases, calls a decision-only path, and reports intent accuracy, slot accuracy, latency percentiles, unsafe executions, question false-fires, and margin violations without executing OS actions.

```
def run_eval(cases: list[dict]) -> dict:
    """Run every case through decision_only and score it.

    Returns a dict with per-category stats and overall safety counters.
    Never executes anything – decision_only is side-effect-free.
    """
    per_category = defaultdict(lambda: {
        "total": 0,
        "intent_correct": 0,
        "slot_correct": 0,
        "slot_checked": 0,
        "latencies_ms": [],
    })
    unsafe_execution_count = 0
    question_to_command_false_fires = 0
    margin_violations = 0
    results = []

    for case in cases:
        text = case["text"]
```

Listing 4.9.7: Decision-only NLU evaluation counters.

Algorithms

Live data selection:

```
query = user_query.strip()
language = ar if query contains Arabic script else en
weather_intent = detect weather keywords and optional city
search_intent = None when weather already answers the query
else search_intent = detect search/news/current keywords, excluding file-search phrases
if no intent and force_search and WEB_SEARCH_ENABLED: search full query
fetch weather/search in parallel when requested
for each nonempty result: normalize for voice and wrap in [WEATHER] or [WEB_SEARCH]
return joined blocks or empty string
```

Search ranking:

collect more DuckDuckGo rows than requested
 drop blocklisted domains
 base score preserves provider order
 add trusted-domain boost when domain matches configured trusted list
 parse available publication date fields
 add recency boost with time decay
 sort by score descending and format top N rows

Configuration Surface

Setting	Role visible from tools/
WEATHER_DEFAULT_CITY / WEATHER_DEFAULT_LATITUDE / WEATHER_DEFAULT_LONGITUDE	weather.py and live_data.py use these when no explicit weather city/location is available.
WEB_SEARCH_ENABLED	live_data.py only force-enables generic search when this flag is true.
WEB_SEARCH_MAX_RESULTS	Bounds search result count passed from live_data.py to web_search.py.
VOICE_NORMALIZER_MAX_SEARCH_RESULTS	Bounds search rows retained when live_data.py formats voice-ready search blocks.
WEB_SEARCH_TRUSTED_DOMAINS / WEB_SEARCH_BLOCKED_DOMAINS	web_search.py boosts trusted hosts and removes blocked hosts.
WEB_SEARCH_TRUSTED_DOMAIN_BOOST / WEB_SEARCH_RECENCY_BOOST	web_search.py scoring weights for reranking search results.
_TIMEOUT_SECONDS	weather.py local Open-Meteo timeout: 5.0 seconds.
_SEARCH_TIMEOUT_SECONDS	web_search.py local DuckDuckGo timeout: 6.0 seconds.
_LIVE_DATA_TIMEOUT	live_data.py outer future timeout: 7.0 seconds.

Table 4.9.2: Configuration symbols and local timeouts visible from tools/.

Behavior In Different Situations

Situation	tools/ behavior	Evidence
Normal	Weather and search requests return formatted blocks; calculator returns a formatted number for math-looking input; NLU eval reports decision-only statistics.	weather.py, web_search.py, live_data.py, calculator.py, evaluate_nlu.py.
Degraded	Open-Meteo, DuckDuckGo, missing provider, timeout, or exception paths return empty strings so callers can proceed without live context.	weather.py get_weather(); web_search.py search_web(); live_data.py gather_live_data().
Adversarial/Edge	Broad Arabic question words are intentionally not generic	live_data.py _detect_web_search_intent();

	search triggers; file-search phrases are excluded; calculator rejects dangerous names and non-finite/non-numeric results.	calculator.py quick_calc().
Ambiguous Location	Weather city extraction falls back to WEATHER_DEFAULT_CITY when no city is found or when the captured Arabic token is a time word such as today/now.	live_data.py _detect_weather_intent().

Table 4.9.3: tools/ behavior under normal, degraded, and edge conditions.

4.10 utils/

Purpose

utils/ is currently a small shared-helper package. Its implemented helper is a language detector that classifies text by Arabic and English character counts. This is deliberately simpler than the heavier NLP language gates documented elsewhere: it exposes a reusable ar/en/mixed/unknown utility.

File Inventory

File	Responsibility
utils/__init__.py	Utilities package marker.
utils/language_detector.py	Counts Arabic and English characters and classifies text as ar, en, mixed, or unknown.

Table 4.10.1: Inventory of Python files directly under utils/.

Key Helpers

```
import re

_ARABIC_RE = re.compile(r"[\u0600-\u06FF]")
_ENGLISH_RE = re.compile(r"[A-Za-z]")

def count_arabic_chars(text: str) -> int:
    return len(_ARABIC_RE.findall(str(text or "")))

def count_english_chars(text: str) -> int:
    return len(_ENGLISH_RE.findall(str(text or "")))

def count_valid_chars(text: str) -> int:
```

```
return count_arabic_chars(text) + count_english_chars(text)
```

```
def detect_language(text: str) -> str:
    value = str(text or "")
```

Listing 4.10.1: Arabic/English character counting and language classification.

Return value	Condition
unknown	No Arabic or English letters found.
ar	Arabic letters present and English letters absent.
en	English letters present and Arabic letters absent.
mixed	Both Arabic and English letters present.

Table 4.10.2: utils.language_detector.detect_language() return contract.

Contribution Summary

Together, tools/ and utils/ keep live facts and lightweight deterministic helpers out of the LLM core. Weather and search can augment prompts when live context is needed, calculator avoids unnecessary model calls for arithmetic, evaluate_nlu.py provides regression evidence, and utils/language_detector.py offers a reusable script-count primitive.

4.11 ui/ and desktop/

Purpose

ui/ and desktop/ provide an optional visual layer over the Jarvis engine. The Python side can expose a WebSocket bridge and a small system tray, while the desktop side is a Tauri and React application with overlay and dashboard windows. The engine remains usable without either UI surface: bridge startup exits cleanly when disabled or when optional dependencies are missing, and the desktop application connects to the bridge rather than hosting the engine.

4.11.1 ui/

File Inventory

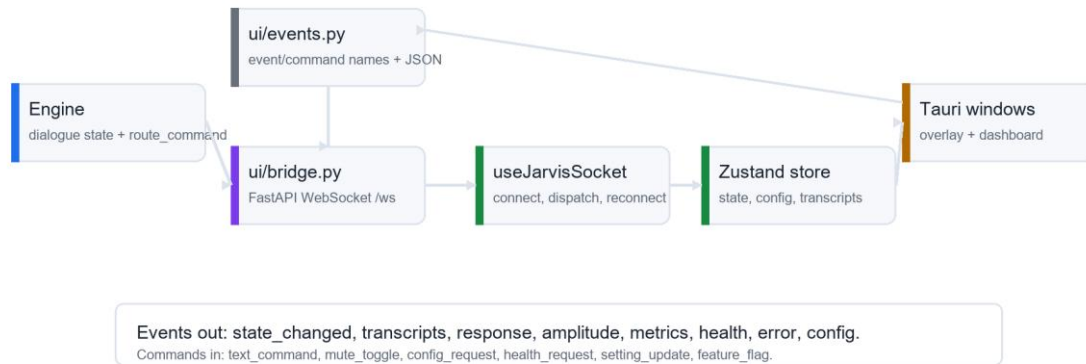
File	Responsibility
ui/bridge.py	Optional FastAPI/uvicorn WebSocket bridge from engine events to desktop clients and UI commands back to route_command.
ui/events.py	Shared engine-to-UI event names, UI-to-engine command names, and JSON serialization helper.
ui/tray.py	Optional pystray system tray with state-colored icon, settings/log shortcuts, restart, demo toggle, and quit cleanup.

Table 4.11.1: Inventory of Python files directly under ui/.

Bridge Protocol

Figure 4.11.1 - UI bridge protocol

Optional Python bridge between the headless engine and Tauri/React desktop windows.



Headless guarantee: bridge/tray startup is optional; missing UI dependencies or absent desktop app do not stop the engine.

Figure 4.11.1: Optional WebSocket protocol between the Jarvis engine and Tauri/React desktop clients.

```
"""Event helpers for the Jarvis UI bridge."""
```

```
from __future__ import annotations
```

```
import json
```

```
# Engine -> UI events
EVENT_STATE_CHANGED = "state_changed"
EVENT_PARTIAL_TRANSCRIPT = "partial_transcript"
EVENT_FINAL_TRANSCRIPT = "final_transcript"
EVENT_RESPONSE = "response"
EVENT_AMPLITUDE = "amplitude"
EVENT_METRICS = "metrics"
EVENT_HEALTH = "health"
EVENT_ERROR = "error"
EVENT_CONFIG = "config"
```

```
# UI -> engine commands
COMMAND_TEXT = "text_command"
```

Listing 4.11.1: Event and command names shared by the Python bridge.

bridge.py creates an optional FastAPI WebSocket server at /ws. It records whether the bridge is enabled, host/port, connected clients, and muted state. If FastAPI or uvicorn are unavailable, start() logs that optional dependencies are unavailable and leaves the engine running without the bridge.

```
self.clients = set()
self.lock = threading.Lock()
self.loop = None
self.muted = False
self.enabled = bool(getattr(config, "UI_BRIDGE_ENABLED", True))
self.host = str(getattr(config, "UI_BRIDGE_HOST", "127.0.0.1") or "127.0.0.1")
self.port = int(getattr(config, "UI_BRIDGE_PORT", 9720) or 9720)
self._running = False
self._state_listener = None
self._server_thread = None
self._server = None
self._start_lock = threading.Lock()
self._missing_deps_logged = False

@property
def available(self) -> bool:
    return FastAPI is not None and uvicorn is not None

@property
def running(self) -> bool:
```

Listing 4.11.2: Optional bridge startup, dependency guard, host/port, and state listener registration.

```
async def handle_client(self, websocket: WebSocket) -> None:
    try:
        await websocket.accept()
        self._register_client(websocket)
        await websocket.send_text(to_json(self._config_event()))
        while True:
            try:
                raw_message = await websocket.receive_text()
                await self._handle_message(websocket, raw_message)
            except WebSocketDisconnect:
                break
            except Exception:
                logger.exception("UI bridge websocket message failed")
                try:
                    await websocket.send_text(
```



```

        to_json(
            make_event(
                EVENT_ERROR,
                message="Bridge failed to process websocket
message.",
                recoverable=True,

```

Listing 4.11.3: WebSocket client loop and handled UI commands.

```

def _route_text_command(self, text: str, language) -> str:
    try:
        from core.command_router import route_command

        return route_command(text, detected_language=language)
    except Exception:
        logger.exception("UI bridge text_command routing failed")
        return "I could not process that command."

def _register_client(self, websocket: WebSocket) -> None:
    with self.lock:
        self.clients.add(websocket)
        logger.info("UI bridge client connected; clients=%s", len(self.clients))

def _unregister_client(self, websocket: WebSocket) -> None:
    with self.lock:
        self.clients.discard(websocket)
        logger.info("UI bridge client disconnected; clients=%s", len(self.clients))

def _on_state_changed(self, old_state, new_state) -> None:

```

Listing 4.11.4: route_command handoff, config/health events, and broadcast fan-out.

tray.py is another optional UI surface. If pystray is missing, start() logs that the tray icon is disabled. When available, it creates a state-colored icon, registers a dialogue-state listener, exposes menu shortcuts for .env/logs/project folder, toggles demo mode, restarts Jarvis, and performs shutdown cleanup on quit.

```

class TrayManager:
    def __init__(self):
        self._icon = None
        self._thread = None
        self._running = False

```

```

self._listener = None

def _open_path(self, path: str) -> None:
    target = str(path)
    try:
        if os.name == 'nt':
            os.startfile(target)
        else:
            webbrowser.open(Path(target).as_uri())
    except Exception as exc:
        logger.warning("Tray: failed to open path %s: %s", target, exc)

def _menu_open_env(self, icon, item):
    Listing 4.11.5: Tray menu actions, restart, .env demo-mode update, and shortcuts.

def start(self):
    if pystray is None:
        logger.info("pystray not available; tray icon disabled.")
        return
    if self._running:
        return
    self._running = True
    # initial icon
    img = _create_circle_icon(_STATE_COLORS.get(DialogueState.IDLE))
    menu = self._build_menu()
    icon = pystray.Icon("jarvis_tray", img, "Jarvis", menu)
    self._icon = icon
    self._listener = self._on_state_changed
    dialogue_manager.register_state_listener(self._listener)
    self._refresh_icon()

def target():
    try:
        icon.run()
    except Exception as exc:
        Listing 4.11.6: Optional tray startup, state listener, icon thread, and shutdown.

```

4.11.2 desktop/

Key File Inventory

Key file	Responsibility
desktop/package.json	Tauri/Vite/React package manifest, scripts, runtime dependencies, and test tooling.
desktop/vite.config.ts	Vite React/Tailwind config plus opt-in mock bridge server for dev:mock.
desktop/src-tauri/tauri.conf.json	Tauri app/window configuration: overlay and dashboard windows, transparency, tray-visible hidden startup.
desktop/src-tauri/src/lib.rs	Tauri commands, tray menu, overlay positioning, and hide-on-close window behavior.
desktop/src/App.tsx	Chooses overlay or dashboard view based on Tauri window label or browser fallback store state.
desktop/src/protocol.ts	Typed bridge protocol: engine events, UI commands, feature flags, config values, state colors, and WebSocket URL.
desktop/src/hooks/useJarvisSocket.ts	WebSocket connection hook with reconnect/backoff and JSON event dispatch into the store.
desktop/src/stores/jarvisStore.ts	Zustand store for connection state, dialogue state, config, transcripts, response, avatar settings, and persisted UI preferences.
desktop/src/components/overlay/Overlay.tsx	Frameless overlay view with wake-driven show/hide, avatar, transcript, prompt input, mute, and dashboard button.
desktop/src/components/dashboard/Dashboard.tsx	Control-center view for prompt, avatar, persona/model/language selections, feature flags, audio mute, and status.
desktop/src/components/avatar/Avatar.tsx	Avatar selector over aurora, glyph, glassai, and companion variants.
desktop/src/components/overlay/PromptInput.tsx	Text prompt form that sends text_command with simple Arabic/English detection.

Table 4.11.2: Key desktop/ files read for the Tauri+React overview.

Toolchain And Window Model

Area	Observed implementation
Runtime shell	Tauri 2 application with Rust commands and tray menu in desktop/src-tauri/src/lib.rs.
Frontend	React 18, TypeScript, Vite 6, Tailwind/Vite plugin, Zustand store, and motion/Three/OGL-related visual dependencies from package.json.
Development	npm run dev for Vite, npm run dev:mock for the

	opt-in mock bridge, npm run build for tsc plus Vite build, npm run test for Vitest.
Window model	tauri.conf.json declares an overlay window and a dashboard window, both initially hidden; overlay is transparent, frameless, always-on-top, and skipped from the taskbar.
Protocol	desktop/src/protocol.ts mirrors ui/events.py names and defines typed events, commands, config values, feature flags, state colors, and WebSocket URL.

Table 4.11.3: Tauri+React toolchain and window model from desktop key files.

```
{
  "$schema": "../node_modules/@tauri-apps/cli/config.schema.json",
  "productName": "Jarvis",
  "version": "0.1.0",
  "identifier": "com.jarvis.assistant",
  "build": {
    "frontendDist": "../dist",
    "devUrl": "http://localhost:5173",
    "beforeDevCommand": "npm run dev",
    "beforeBuildCommand": "npm run build"
  },
  "app": {
    "windows": [
      {
        "label": "overlay",
        "title": "Jarvis",
        "width": 360,
        "height": 540,
        "resizable": false,
        "decorations": false,

```

Listing 4.11.7: Tauri overlay and dashboard window definitions.

```
use tauri::{
  menu::{Menu, MenuItem},
  tray::TrayIconBuilder,
  Manager, PhysicalPosition, WindowEvent,
};

#[tauri::command]
fn open_dashboard(app: tauri::AppHandle) {
  if let Some(w) = app.get_webview_window("dashboard") {

```

```

        let _ = w.show();
        let _ = w.set_focus();
    }
}

#[tauri::command]
fn hide_dashboard(app: tauri::AppHandle) {
    if let Some(w) = app.get_webview_window("dashboard") {
        let _ = w.hide();
    }
}

```

Listing 4.11.8: Tauri commands for overlay/dashboard visibility and quit.

```

open_dashboard,
hide_dashboard,
quit_app,
show_overlay,
hide_overlay
])
.setup(|app| {
    if cfg!(debug_assertions) {
        app.handle().plugin(
            tauri_plugin_log::Builder::default()
                .level(log::LevelFilter::Info)
                .build(),
        );
    }
})

// — System tray —————
let open_item =
    MenuItem::with_id(app, "open_dashboard", "Open Dashboard", true,
None:::<&str>)?;
let toggle_item = MenuItem::with_id(
    app,

```

Listing 4.11.9: Tauri tray menu and hide-on-close behavior.

```

import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vite';
import { mockJarvisServer } from '../src/mock/mockServer';

export default defineConfig(({ mode }) => ({

```

```

plugins: [
  react(),
  tailwindcss(),
  // The mock engine is opt-in: `npm run dev:mock` (vite --mode mock). Default
  // dev/build connects to the real Python bridge (ui/bridge.py).
  ...(mode === 'mock' ? [mockJarvisServer()] : []),
],
clearScreen: false,
server: {
  port: 5173,
  strictPort: true,
},
envPrefix: ['VITE_', 'TAURI_'],

```

Listing 4.11.10: Vite config and opt-in mock bridge.

The desktop protocol mirrors the Python event names with TypeScript discriminated unions. The WebSocket URL defaults to a mock development server and can be pointed at the real Python bridge with `VITE_JARVIS_WS_URL`.

```

export type DialogueState = 'idle' | 'listening' | 'processing' | 'responding' |
'confirming' | 'executing' | 'follow_up';

export type Language = 'en' | 'ar';

export interface FeatureFlags {
  NUMERIC_PARSING_ENABLED: boolean;
  AUTO_APP_DISCOVERY_ENABLED: boolean;
  MEDIA_DIRECT_DISPATCH_ENABLED: boolean;
  SYSTEM_VOLUME_CONTROL: boolean;
}

export interface ConfigValues {
  model: string;
  model_tier: string;
  wake_mode: string;
  feature_flags: FeatureFlags;
  stt_backend: string;
  tts_backend: string;
  persona: string;
}

```

Listing 4.11.11: Typed desktop bridge protocol and WebSocket URL selection.

```

import { useCallback, useEffect, useRef } from 'react';
import { JARVIS_WS_URL, type EngineEvent, type UICommand } from '../protocol';
import { useJarvisStore } from '../stores/jarvisStore';

export function useJarvisSocket(): { send: (cmd: UICommand) => void; connectionStatus:
string } {
  const socketRef = useRef<WebSocket | null>(null);
  const reconnectTimerRef = useRef<number | null>(null);
  const reconnectDelayRef = useRef(1000);
  const shouldReconnectRef = useRef(true);
  const dispatch = useJarvisStore((state) => state.dispatch);
  const connectionStatus = useJarvisStore((state) => state.connectionStatus);
  const setConnectionStatus = useJarvisStore((state) => state.setConnectionStatus);

  useEffect(() => {
    const clearReconnectTimer = () => {
      if (reconnectTimerRef.current !== null) {
        window.clearTimeout(reconnectTimerRef.current);
        reconnectTimerRef.current = null;
      }
    };
  });

```

Listing 4.11.12: Desktop WebSocket connection, event dispatch, and reconnect backoff.

Application state is stored in Zustand. Runtime events update dialogue state, transcripts, response, metrics, errors, and config. A subset of UI preferences is persisted to localStorage and rehydrated across separate Tauri windows.

```

import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';
import type { ConfigValues, DialogueState, EngineEvent, FeatureFlags, Language } from
'../protocol';

type ConnectionStatus = 'connecting' | 'connected' | 'disconnected';
export type AvatarDirection = 'aurora' | 'glyph' | 'glassai' | 'companion';
export type AppView = 'overlay' | 'dashboard';
export type UiLanguage = Language | 'auto';

interface JarvisState {
  connectionStatus: ConnectionStatus;
  dialogueState: DialogueState;
  config: ConfigValues | null;
  appView: AppView;

```

```

uiLanguage: UiLanguage;
amplitude: number;
muted: boolean;
partialTranscript: string;
finalTranscript: string;
transcriptLanguage: Language | null;

```

Listing 4.11.13: Desktop store state shape.

```

...initialState,
dispatch: (event) => {
  switch (event.type) {
    case 'state_changed':
      set({
        dialogueState: event.state,
        ...(event.state === 'idle'
          ? {
            partialTranscript: '',
            finalTranscript: '',
            response: '',
          }
          : {}),
      });
      break;
    case 'partial_transcript':
      set({
        partialTranscript: event.text,
        transcriptLanguage: event.language,
      });

```

Listing 4.11.14: Engine event reducer, persisted preferences, and cross-window sync.

App.tsx chooses overlay or dashboard according to the current Tauri window label; in a browser it falls back to the store's appView. The overlay shows when dialogue state is active and hides after idle; the dashboard exposes local controls and sends bridge commands.

```

import { getCurrentWindow } from '@tauri-apps/api/window';
import { Dashboard } from '../components/dashboard/Dashboard';
import { Overlay } from '../components/overlay/Overlay';
import { useJarvisSocket } from '../hooks/useJarvisSocket';
import { useJarvisStore } from '../stores/jarvisStore';
import { isTauri } from '../lib/app';

```

```

// Under Tauri each OS window owns one view (label "overlay" | "dashboard").

```



```
// In the browser there is one window, so fall back to the in-app appView toggle.
function tauriWindowView(): 'overlay' | 'dashboard' | null {
  if (!isTauri()) return null;
  try {
    return getCurrentWindow().label === 'dashboard' ? 'dashboard' : 'overlay';
  } catch {
    return null;
  }
}
```

```
export default function App() {
  const { send } = useJarvisSocket();
```

Listing 4.11.15: Tauri window label selects overlay or dashboard.

```
export function Overlay({ send }: OverlayProps) {
  const muted = useJarvisStore((state) => state.muted);
  const setMuted = useJarvisStore((state) => state.setMuted);
  const dialogueState = useJarvisStore((state) => state.dialogueState);
  const textPromptEnabled = useJarvisStore((state) => state.textPromptEnabled);
  const stateColor = STATE_COLORS[dialogueState];
  const prevState = useRef<DialogueState>('idle');

  // Wake-driven window lifecycle (Tauri): the overlay window stays hidden in the
  // background and pops up when the assistant becomes active, then sinks away and
  // hides when the interaction returns to idle.
  useEffect(() => {
    const prev = prevState.current;
    prevState.current = dialogueState;
    if (!isTauri()) return;
    if (dialogueState !== 'idle') {
      void showOverlay();
    } else if (prev !== 'idle') {
      const timer = window.setTimeout(() => {
```

Listing 4.11.16: Wake-driven overlay show/hide lifecycle.

```
import { FormEvent, useState } from 'react';
import type { Language, UICommand } from '../protocol';

interface PromptInputProps {
  send: (cmd: UICommand) => void;
```

```
}
```

```
const ARABIC_TEXT_PATTERN = /\u0600-\u06FF/;
```

```
function detectLanguage(text: string): Language {  
  return ARABIC_TEXT_PATTERN.test(text) ? 'ar' : 'en';  
}
```

```
export function PromptInput({ send }: PromptInputProps) {  
  const [text, setText] = useState('');  
  
  const handleSubmit = (event: FormEvent<HTMLFormElement>) => {  
    event.preventDefault();  
  
    const trimmedText = text.trim();
```

Listing 4.11.17: Prompt input sends text_command with simple language detection.

```
const handleMutedChange = (nextMuted: boolean) => {  
  setMuted(nextMuted);  
  send({ type: 'mute_toggle', muted: nextMuted });  
};  
  
return (  
  <div className="frameless-scroll relative h-screen overflow-y-auto px-4 py-6 text-white sm:px-6 lg:px-8">  
    /* dark base with a subtle pink / blue / amber gradient glow – matches the app's dark-glass style */  
    <div aria-hidden="true" className="pointer-events-none fixed inset-0 z-0 bg-[#0A0A0F]/95 backdrop-blur-xl">  
      <div className="absolute inset-0 opacity-35">  
        <GradientBackground  
          containerClassName="h-full w-full"  
          gradientColors={['rgb(255, 100, 150)', 'rgb(100, 150, 255)', 'rgb(255, 200, 100)']}>  
        />  
      </div>  
    </div>  
    <div className="relative z-10 mx-auto grid w-full max-w-5xl gap-5 p-4 text-sm sm:p-5">  
      /* Frameless window: this header doubles as the drag handle. */
```

Listing 4.11.18: Dashboard controls for config request, view hiding, language, model, feature flags, and mute.

```

import { STATE_COLORS } from '../protocol';
import { useJarvisStore } from '../stores/jarvisStore';
import { AuroraAvatar } from './AuroraAvatar';
import { CompanionAvatar } from './CompanionAvatar';
import { GlassAIAvatar } from './GlassAIAvatar';
import { GlyphAvatar } from './GlyphAvatar';

export function Avatar() {
  const avatarDirection = useJarvisStore((state) => state.avatarDirection);
  const dialogueState = useJarvisStore((state) => state.previewDialogueState ??
state.dialogueState);
  const amplitude = useJarvisStore((state) => state.amplitude);
  const color = STATE_COLORS[dialogueState];

  switch (avatarDirection) {
    case 'aurora':
      return <AuroraAvatar state={dialogueState} amplitude={amplitude} color={color} />;
    case 'glyph':
      return <GlyphAvatar state={dialogueState} amplitude={amplitude} color={color} />;
    case 'companion':
      return <CompanionAvatar state={dialogueState} color={color} />;
  }
}

```

Listing 4.11.19: Avatar variant selection from store state.

Algorithms

Bridge event loop:

```

if UI_BRIDGE_ENABLED is false: return without starting
if FastAPI or uvicorn is missing: log optional dependency state and return
start uvicorn server on ws://host:port/ws in a daemon thread
on client connect: accept socket, register client, send config event
on text_command: route text on a worker thread, then broadcast response event
on config_request or health_request: send/broadcast current bridge-local status
on engine state change: broadcast state_changed event
on broadcast failure: remove stale websocket client

```

Desktop reconnect and dispatch:

```

set connectionStatus to connecting
open WebSocket at JARVIS_WS_URL
on open: reset reconnect delay and set connected
on message: parse JSON as EngineEvent and dispatch into Zustand store
on invalid JSON: dispatch error event

```

on close: set disconnected and schedule reconnect with delay capped at 8000 ms
 send UICommand only when socket readyState is OPEN

Configuration Surface

Setting	Role visible from ui/ or desktop/
UI_BRIDGE_ENABLED	ui/bridge.py start() exits early when disabled.
UI_BRIDGE_HOST / UI_BRIDGE_PORT	ui/bridge.py binds the FastAPI WebSocket endpoint at ws://host:port/ws.
LLM_MODEL / LLM_AUTO_SELECT_MODEL	Included in bridge config events so the dashboard can display model state.
WAKE_WORD_MODE	Included in bridge config events.
FEATURE_FLAGS	Included in bridge config events and edited locally by dashboard toggles before sending feature_flag commands.
STT_BACKEND / TTS_DEFAULT_BACKEND / PERSONA_DEFAULT	Included in bridge config events for dashboard display/control context.
PROJECT_ROOT / LOG_FILE	ui/tray.py uses these for .env, project folder, and log shortcuts.
VITE_JARVIS_WS_URL	desktop/src/protocol.ts uses this environment variable to target the real Python bridge; otherwise it defaults to mock ws://localhost:8765.

Table 4.11.4: UI and desktop configuration surface observed in this phase.

Behavior In Different Situations

Situation	UI/desktop behavior	Evidence
Normal	Bridge starts on /ws, sends config on connection, broadcasts state and response events, desktop hook dispatches events into Zustand, and overlay/dashboard render from store state.	ui/bridge.py, ui/events.py, desktop/src/protocol.ts, useJarvisSocket.ts, jarvisStore.ts, App.tsx.
Headless	If the bridge is disabled or optional dependencies are missing, start() returns without raising; tray also disables itself when pystray is unavailable.	ui/bridge.py start(); ui/tray.py start().
Disconnected	Desktop connection status becomes disconnected and the hook retries with exponential backoff; dashboard can show the disconnected status from the store.	useJarvisSocket.ts; Dashboard.tsx.
UI Absent	Tauri window commands affect only desktop windows;	desktop/src/lib/app.ts; ui/bridge.py; ui/tray.py.

	closeApp notes that the engine is separate, and Python bridge/tray are optional surfaces over the engine.	
Partial Commands	setting_update and feature_flag commands are currently logged by the bridge rather than applied; the dashboard updates local UI state before sending them.	ui/bridge.py _handle_message(); Dashboard.tsx.

Table 4.11.5: UI behavior under normal, headless, disconnected, absent, and partial-command conditions.

Contribution Summary

ui/ and desktop/ add observability and hands-on control without making the assistant dependent on a graphical surface. The bridge exposes a narrow JSON protocol, the tray gives a lightweight operator menu, and the Tauri/React desktop turns engine state into an overlay and dashboard while preserving a headless runtime path.

4.12 tests/, 4.13 scripts/, and 4.14 models/

Purpose

tests/, scripts/, and models/ are the repository's verification and support-asset layer. The tests assert routing safety, memory behavior, sentence buffering, and spoken text normalization. The scripts provide documentation generation, Windows setup, and wake-word data/model tooling. The models directory contains the packaged ONNX wake-word assets consumed by the audio runtime described in Sections 4.4 and 4.5.

4.12 tests/

File Inventory

File	Observed size	What it verifies or stores
tests/__init__.py	1 bytes	Marks tests/ as an importable package.
tests/test_codeswitch_routing.py	4521 bytes	Exercises mixed Arabic/English command parsing for app, volume, folder, delete, rename, and move cases, including fall-through for ambiguous or nonexistent targets.
tests/test_llm_routing_guard.py	3768 bytes	Checks that advice questions and corrupted career questions remain LLM

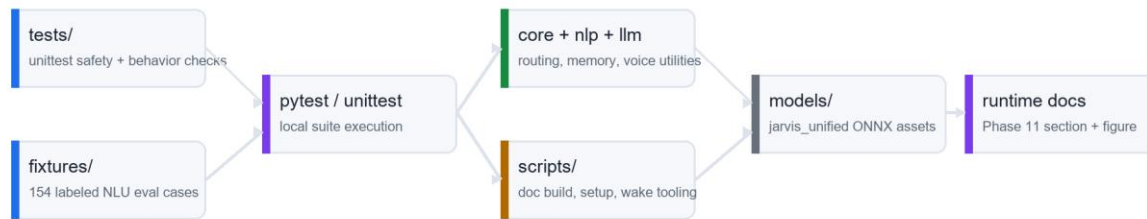
		queries instead of command chains, timers, or live-search commands; also checks repeated-question cleanup and low-value answer repair.
tests/test_memory_layer.py	2494 bytes	Covers fast context, recent-turn context, semantic-recall bypass for short queries, and the latency ceiling around RAM-only memory access.
tests/test_memory_reference_resolver.py	3815 bytes	Covers bilingual freshness-gated reference rewriting for last app/file and blocks vague destructive pronoun deletes.
tests/test_pending_task_memory.py	2217 bytes	Covers RAM-only pending task slot filling, dispatch after all required slots are present, TTL expiry, and no-task behavior.
tests/test_routing_safety.py	1178 bytes	Runs the labeled NLU eval set and asserts zero unsafe auto-executions and zero question-to-command false fires.
tests/test_semantic_margin.py	1040 bytes	Asserts near-tie semantic routing defers instead of guessing while a clean app-open command still executes.
tests/test_sentence_buffer.py	1482 bytes	Covers English and Arabic sentence buffering, punctuation boundaries, hard flush thresholds, and Arabic connector holding.
tests/test_tts_prosody.py	3787 bytes	Covers voice polish normalization for repeated punctuation, markdown markers, ellipses, hyphen handling, and Arabic discourse/formal connectors.
tests/test_voice_normalizer.py	1525 bytes	Covers spoken-form normalization for weather, units, URLs, search blocks, Arabic text, times, dates, speed, and percentages.
tests/fixtures/nlu_eval_cases.jsonl	21092 bytes	Stores labeled routing-

		evaluation cases used by the safety tests and NLU evaluation harness.
--	--	---

Table 4.12.1: Inventory of tests/ files and fixture coverage.

Figure 4.12.1 - QA, scripts, and model assets

Repository support assets that prove routing, memory, voice text shaping, setup, and wake-model packaging.



Hard QA targets in this repository: no unsafe eval executions, no question-to-command false fires, and no near-tie auto-execution
Wake data/training mechanics are cross-referenced to Section 4.5; this phase inventories the script and model assets only.

Figure 4.12.1: QA and support-asset flow across tests, scripts, models, and the runtime documentation.

NLU Fixture Coverage

Fixture category or metric	Count	Meaning
Total cases	154	All non-empty JSONL rows in tests/fixtures/nlu_eval_cases.jsonl.
Cases where should_execute is false	44	Negative or non-executing cases in the fixture.
Cases where should_clarify is true	14	Cases explicitly requiring clarification.
ambiguous	8	Fixture category count.
ar_command	22	Fixture category count.
clipboard	2	Fixture category count.
dangerous	8	Fixture category count.
email_calendar	5	Fixture category count.
en_command	28	Fixture category count.
false_positive	6	Fixture category count.
follow_up	6	Fixture category count.
identity	5	Fixture category count.
missing_slot	5	Fixture category count.
mixed_codeswitch	23	Fixture category count.
multi_step	3	Fixture category count.
near_tie	2	Fixture category count.
noisy_stt	8	Fixture category count.
question	15	Fixture category count.

settings	3	Fixture category count.
sysinfo	3	Fixture category count.
voice_control	2	Fixture category count.

Table 4.12.2: Counts derived from tests/fixtures/nlu_eval_cases.jsonl.

Safety And Routing Tests

The safety suite runs labeled NLU cases through a decision-only route. It does not dispatch OS actions, mutate session memory, or call a network LLM. Its hard assertions are zero unsafe executions and zero question-to-command false fires.

```
import unittest

from tools.evaluate_nlu import load_cases, run_eval

class RoutingSafetyTests(unittest.TestCase):
    """Runs the full labeled eval set and asserts the hard safety invariants:
    zero unsafe auto-executions and zero question-to-command false-fires.
    Uses core.eval_routing.decision_only (no dispatch, no session_memory, no
    network LLM calls) – see tools/evaluate_nlu.py for the scoring rules.
    """

    @classmethod
    def setUpClass(cls):
        cls.cases = load_cases()
        cls.summary = run_eval(cls.cases)

    def test_zero_unsafe_executions(self):
        failures = [r for r in self.summary["results"] if r["unsafe_execution"]]
        self.assertEqual(
            Listing 4.12.1: Safety invariants over the labeled NLU eval set.

import unittest

from core.eval_routing import decision_only

class SemanticMarginTests(unittest.TestCase):
    """Asserts Phase 3's top-k + margin scoring: a near-tie semantic match
    must not silently auto-execute – it should defer (clarify) rather than
    guess between two similarly-scored intents.
    """
```



```

def test_bare_open_defers_instead_of_guessing(self):
    # "open" alone is a real near-tie in this repo's semantic router
    # (OS_EMAIL vs OS_APP_OPEN scored ~0.82/0.79, margin ~0.03 < the
    # 0.08 minimum) – see jarvis_phase4_nlp_routing_plan.md Phase 3.
    decision = decision_only("open")
    self.assertNotEqual(
        decision.decision, "execute",
        msg=f"Near-tie utterance auto-executed: {decision}",
    )

```

Listing 4.12.2: Semantic near-tie guard and clean-match execution check.

```

class CodeswitchRoutingTests(unittest.TestCase):
    """Asserts Phase 4's code-switch shortcut resolves mixed EN/AR
    verb+entity utterances to the right intent+slots without the semantic
    embedding tier running.
    """

```

```

def test_arabic_verb_english_app_resolves_open(self):
    parsed = try_codeswitch("افتح Chrome", "ar")
    self.assertIsNotNone(parsed)
    self.assertEqual(parsed.intent, "OS_APP_OPEN")
    self.assertEqual(parsed.args.get("app_name"), "chrome")

def test_arabic_verb_english_entity_resolves_volume_up(self):
    parsed = try_codeswitch("زود volume", "ar")
    self.assertIsNotNone(parsed)
    self.assertEqual(parsed.intent, "OS_SYSTEM_COMMAND")
    self.assertEqual(parsed.args.get("action_key"), "volume_up")

```

```

def test_close_verb_resolves_app_close(self):
    parsed = try_codeswitch("close Spotify", "en")

```

Listing 4.12.3: Code-switch routing tests for apps, volume, folders, and ambiguous text.

```

class LLMRoutingGuardTests(unittest.TestCase):
    def test_arabic_advice_question_stays_llm_query(self):
        text = "ممکن تقول لي إزاي أكون مهندس كمبيوتر ناجح؟ أنا مش عارف أعمل كده"
        parsed = parse_command(text)
        self.assertEqual(parsed.intent, "LLM_QUERY")

    def test_codeswitched_advice_question_not_command_chain(self):

```

```

    text = "ناجح؟ أنا بحاول أعمل كده ومش computer عاوزك تقولي إزاي أكون مهندس".عارف
    parsed = parse_command(text)
    self.assertEqual(parsed.intent, "LLM_QUERY")

def test_step_by_step_advice_question_not_timer(self):
    text = "عاوزك تقولي إزاي أكون مهندس كمبيوتر ناجح؟ قولي الخطوات خطوة خطوة"
    parsed = parse_command(text)
    self.assertEqual(parsed.intent, "LLM_QUERY")

def test_advice_question_does_not_trigger_live_search(self):
    text = "عاوزك تقولي إزاي أكون مهندس كمبيوتر ناجح؟ قولي الخطوات خطوة خطوة"
    self.assertFalse(_looks_search_worthy_query(text))
    self.assertFalse(_looks_live_data_trigger_query(text))

```

Listing 4.12.4: LLM routing guard tests for advice questions and response cleanup.

Memory And Multi-Turn Tests

```

class MemoryFastContextTests(unittest.TestCase):
    """Asserts the router's hot path (get_fast_context) reflects RAM slot
    state and never touches the vector store – Phase 2's core guarantee.
    """

    def setUp(self):
        self._saved_last_app = session_memory.get_last_app()

    def tearDown(self):
        if self._saved_last_app:
            session_memory.set_last_app(self._saved_last_app)

    def test_fast_context_reflects_last_app(self):
        session_memory.set_last_app("Notepad")
        ctx = memory_manager.get_fast_context()
        self.assertEqual(ctx.working_slots.get("last_app"), "Notepad")

    def test_fast_context_never_touches_vector_store(self):
        with patch("core.memory_store.vector_memory.recall") as mock_recall, \
            patch("core.memory_store.vector_memory.remember") as mock_remember:

```

Listing 4.12.5: Fast memory context and LLM-context behavior tests.

```

class ReferenceResolverTests(unittest.TestCase):
    """Phase 3's follow-up resolver ("close it" -> last_app, etc.) – RAM-only,

```

```

bilingual, freshness-gated. See core.command_router._rewrite_followup_command.
"""

def setUp(self):
    self._saved_last_app = session_memory.get_last_app()
    self._saved_last_file = session_memory.get_last_file()
    self.desktop = KNOWN_FOLDERS.get("Desktop")
    self.fixture_path = os.path.join(str(self.desktop),
"jarvis_ref_resolver_fixture.txt")
    with open(self.fixture_path, "w", encoding="utf-8") as f:
        f.write("test")

def tearDown(self):
    if os.path.exists(self.fixture_path):
        os.remove(self.fixture_path)
    if self._saved_last_app:
        session_memory.set_last_app(self._saved_last_app)
    if self._saved_last_file:
        session_memory.set_last_file(self._saved_last_file)

```

Listing 4.12.6: Freshness-gated app/file reference resolver tests.

```

class PendingTaskMemoryTests(unittest.TestCase):
    """Phase 4's multi-turn slot filling (email compose: to -> body -> subject),
    RAM-only via session_memory.set_pending_task/get_pending_task, no LLM used
    to run the flow. See core.command_router._start_pending_task/_advance_pending_task.
    """

    def setUp(self):
        session_memory.clear_pending_task()

    def tearDown(self):
        session_memory.clear_pending_task()

    def test_email_multi_turn_slot_fill_dispatches_after_all_slots_filled(self):
        missing_slot, question = _start_pending_task(
            "OS_EMAIL", "draft", {"to": "ahmed", "subject": "", "body": ""}, "en"
        )
        self.assertEqual(missing_slot, "body")
        self.assertIn("say", question.lower())

        handled, response = _advance_pending_task("the report is ready", "en")

```

Voice Text Utility Tests

```
class SentenceBufferTests(unittest.TestCase):

    def test_english_flushes_on_boundary_after_soft_words(self):
        buf = SentenceBuffer(is_arabic=False, en_soft_words=3, en_hard_words=8)
        self.assertIsNotNone(buf.add_token("This is "))
        self.assertEqual(buf.add_token("ready now."), "This is ready now.")


    def test_english_hard_flushes_without_punctuation(self):
        buf = SentenceBuffer(is_arabic=False, en_soft_words=7, en_hard_words=4)
        self.assertEqual(buf.add_token("one two three four five"), "one two three four")
        self.assertEqual(buf.flush(), "five")


    def test_arabic_flushes_on_arabic_boundary_after_soft_words(self):
        buf = SentenceBuffer(is_arabic=True, ar_soft_words=3, ar_hard_words=10)
        text = "\u062f\u0647 \u0631\u0627 \u0639\u0628\u064a\u0639\u064a\u060c"
        self.assertEqual(buf.add_token(text), text)


    def test_arabic_holds_connector(self):
        buf = SentenceBuffer(is_arabic=True, ar_soft_words=3, ar_hard_words=4,
                              hold_connectors=True)
        self.assertIsNotNone(buf.add_token("\u0648\u0627\u062f\u0627\u0627\u0627\u062a\u0646\u0646 \u0627\u0644\u0627\u0627\u0627 \u0648"))
        self.assertEqual(buf.add_token(" \u0627\u0631\u0628\u0639\u0629"),
                          "\u0648\u0627\u062f\u0627\u0627\u0627\u062a\u0646\u0646 \u0627\u0644\u0627\u0627\u0627 \u0648 \u0627\u0631\u0628\u0639\u0629")
```

Listing 4.12.8: Sentence buffer boundary and connector-hold tests.

```
"""Tests for core.tts_prosody.polish_for_voice."""

import unittest

class TestProsodyPolisher(unittest.TestCase):

    def _polish(self, text, language="en", **kw):
        from core.tts_prosody import polish_for_voice
        return polish_for_voice(text, language=language, **kw)

    # — Bilingual: punctuation dedup —
```

```

def test_double_period_compressed(self):
    self.assertEqual(self._polish("hello.. world"), "hello. world")

def test_double_question_mark_ar(self):
    result = self._polish("؟؟ليه", language="ar")
    self.assertEqual(result, "ليه؟")

```

Listing 4.12.9: TTS prosody tests for repeated punctuation, markdown, dashes, and ellipses.

```

class VoiceNormalizerTests(unittest.TestCase):
    def test_weather_english(self):
        raw = "Weather in Cairo: Mainly clear, 33.1°C, humidity 33%, wind 10.0 km/h"
        out = normalize_weather_block(raw, "en")
        self.assertIn("Temperature 33.1 degrees", out)
        self.assertIn("33 percent", out)
        self.assertIn("10 kilometers per hour", out)

    def test_weather_arabic(self):
        raw = "Weather in Cairo: Mainly clear, 33.1°C, humidity 33%, wind 10.0 km/h"
        out = normalize_weather_block(raw, "ar")
        self.assertIn("الطقس في القاهرة", out)
        self.assertIn("درجة", out)
        self.assertIn("في المئة", out)
        self.assertIn("كيلومتر في الساعة", out)

    def test_general_units(self):
        out = normalize_for_voice("27°C 45% 12 km/h 15:30 2026-06-22", "ar")
        self.assertIn("درجة", out)
        self.assertIn("في المئة", out)

```

Listing 4.12.10: Voice normalizer tests for weather, units, and URL cleanup.

Test Philosophy

Safety-oriented routing checks:

```

load labeled JSONL cases from tests/fixtures/nlu_eval_cases.jsonl
for each utterance, call the decision-only router instead of the dispatcher
record unsafe_execution when a case that must not execute would execute
record question_false_fire when a question is routed as an executable command
fail the suite unless unsafe_execution_count == 0
fail the suite unless question_to_command_false_fires == 0
for semantic near-ties, require clarification/defer behavior rather than silent
execution

```

4.13 scripts/

Developer Tool Inventory

Script	Observed size	Role
scripts/generate_arabic_wake_data.py	8333 bytes	Generates unified English/Arabic wake-word WAVs for openWakeWord training; its internals are cross-referenced to Section 4.5.
scripts/rebuild_graduation_doc.py	202709 bytes	Builds and incrementally fills docs/jarvis_documentation_book.docx, including figures, tables, code excerpts, placeholders, and per-phase dispatch.
scripts/setup_windows.ps1	1066 bytes	Windows bootstrap helper that checks Python, upgrades pip, installs requirements.txt, creates .env from .env.example when absent, and runs core\doctor.py.
scripts/train_arabic_wake_model.py	37777 bytes	Trains and exports the unified English/Arabic wake-word ONNX model; its training flow is cross-referenced to Section 4.5.

Table 4.13.1: scripts/ inventory and developer support roles.

rebuild_graduation_doc.py is the active generator for this Word book: it uses python-docx and Pillow, stores the output at docs/jarvis_documentation_book.docx, creates generated figures under docs/generated_figures, and fills one placeholder per requested phase. setup_windows.ps1 is a local Windows bootstrap helper. The wake-word data and training scripts are present here, but their pipeline is documented in Section 4.5 to avoid duplicating training internals.

```
"""Build and update the Jarvis documentation book.
```

```
Phase 0 creates the thesis-style Word skeleton at
``docs/jarvis_documentation_book.docx``. Later phases can import the helper
functions in this file to replace a single ``[[FILL:<id>]]`` placeholder.
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import ast
```

```
import json
```

```
import re
```

```
import sqlite3
```

```
from pathlib import Path
```

```

from typing import Iterable, Sequence

from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.style import WD_STYLE_TYPE

```

Listing 4.13.1: Documentation generator imports, output paths, and purpose.

```

def insert_generated_content(doc_path: Path, placeholder_id: str, add_content) -> Path:
    document = Document(str(doc_path))
    placeholder = find_placeholder_paragraph(document, placeholder_id)
    body = document._body._element
    insertion_index = body.index(placeholder._element)
    bookmark_id = "999"
    for child in placeholder._element.iter():
        if child.tag == qn("w:bookmarkStart") and child.get(qn("w:name")) ==
placeholder_id:
            bookmark_id = child.get(qn("w:id")) or bookmark_id
            break

    original_len = len(body)
    add_content(document)
    new_elements = list(body)[original_len - 1 : -1]
    for element in new_elements:
        body.remove(element)
    body.remove(placeholder._element)
    for offset, element in enumerate(new_elements):
        if offset == 0 and element.tag == qn("w:p"):
            start = XmlElement("w:bookmarkStart")

```

Listing 4.13.2: Placeholder-preserving insertion helper used by phase fills.

```

$ErrorActionPreference = "Stop"

```

```

Write-Host "[Jarvis Setup] Starting Windows setup..." -ForegroundColor Cyan

```

```

if (-not (Get-Command python -ErrorAction SilentlyContinue)) {
    throw "Python is not available in PATH. Install Python 3.12+ and retry."
}

```

```

Write-Host "[Jarvis Setup] Using global Python from PATH (no virtual environment)." -
ForegroundColor Yellow

```

```

Write-Host "[Jarvis Setup] Upgrading pip..." -ForegroundColor Yellow

```

```
python -m pip install --upgrade pip
```

```
Write-Host "[Jarvis Setup] Installing requirements..." -ForegroundColor Yellow
```

```
python -m pip install -r requirements.txt
```

```
if (-not (Test-Path ".env") -and (Test-Path ".env.example")) {  
    Write-Host "[Jarvis Setup] Creating .env from .env.example..." -ForegroundColor Yellow  
    Copy-Item ".env.example" ".env"  
}
```

Listing 4.13.3: Windows setup script steps.

```
"""Generate unified English/Arabic wake-word WAVs for openWakeWord.
```

Outputs 16 kHz mono WAVs organized as:

```
<output_dir>/<keyword>/...
```

Despite its legacy filename, this script now replaces the Arabic-only data flow and builds the unified ``jarvis\_unified`` bilingual corpus. English text is synthesized only with English voices and Arabic text only with Arabic voices. Generated clips are augmented with white noise and simple room reverb.

NOTES

Old partial corpora under ``data/openwakeword/jarvis\_ar/`` are user data. This script neither reads nor deletes them; remove or archive them manually only

Listing 4.13.4: Wake data-generation script purpose and local-data warning.

```
"""Train and export the unified English/Arabic wake-word ONNX model.
```

Despite its legacy filename, this script replaces the Arabic-only training flow. It uses bilingual generated wake phrases plus optional real recordings, creates adversarial negative clips, computes openWakeWord features, trains a compact PyTorch classifier, and exports the result as ONNX.

It is intentionally self-contained so it can run on Windows without the Linux-only openWakeWord notebook flow.

Listing 4.13.5: Wake model training script purpose; implementation details are covered in Section 4.5.

4.14 models/

Model Asset Inventory

Model asset	Format	Observed size	Subsystem
models/jarvis_unified/jarvis_unified.onnx	onnx	4198448 bytes	Wake-word model asset loaded by the audio wake-word runtime; see Sections 4.4 and 4.5.
models/jarvis_unified/jarvis_unified_backup.onnx	onnx	2050557 bytes	Wake-word model asset loaded by the audio wake-word runtime; see Sections 4.4 and 4.5.

Table 4.14.1: Model files listed under models/.

The listed ONNX files live under models/jarvis_unified/. The primary artifact name matches the default unified wake-word model path documented earlier, while the backup ONNX file preserves another local model artifact in the same subsystem. Section 4.4 documents runtime wake-model loading and Section 4.5 documents the dataset/training artifact boundary.

Behavior And QA Coverage

Area	What a passing suite or verified asset proves	Evidence in this phase
Routing safety	Labeled NLU cases do not produce unsafe auto-executions or question-to-command false fires.	test_routing_safety.py; nlu_eval_cases.jsonl.
Ambiguity handling	Bare near-tie commands defer instead of guessing, while clean commands still execute.	test_semantic_margin.py.
Code-switch parsing	Mixed Arabic/English commands map to app, volume, folder, and file-operation intents when resolvable.	test_codeswitch_routing.py.
Memory behavior	Fast context stays RAM-oriented, follow-up references resolve only with fresh context, and pending tasks expire.	test_memory_layer.py; test_memory_reference_resolver.py; test_pending_task_memory.py.

Voice text shaping	Sentence buffering, prosody cleanup, and voice normalization keep spoken output cleaner before TTS.	test_sentence_buffer.py; test_tts_prosody.py; test_voice_normalizer.py.
Developer repeatability	The book build, Windows setup, wake-data generation, wake training, and packaged ONNX assets are discoverable from repository files.	scripts/ inventory; models/ inventory.

Table 4.14.2: Behavior and QA coverage supplied by tests, scripts, and model assets.

Contribution Summary

Phase 11 makes the support layer explicit: tests encode the project's safety and regression contracts, scripts encode repeatable developer workflows, and models/ stores the wake-word artifacts that connect the offline pipeline to the runtime audio subsystem.

Chapter 5 Conclusion & Future Work

5.1 Objectives Achieved

ID	Objective	Evidence in this book
O1	Bilingual EGY/English understanding, including code-switching.	Sec 4.6 code-switch router, semantic margin rule, and fuzzy/keyword fallback; Sec 4.4 STT language lock with code-switch acceptance; Sec 4.12 code-switch and semantic-margin test suites.
O2	Low-latency response, reserving the LLM tier for what earlier tiers cannot resolve.	Sec 3.4 seven-tier cascade; Sec 4.2 command_router implementation; parser/code-switch/semantic tiers resolve without any LLM call.
O3	Reliable hands-free wake activation.	Sec 4.4 EMA/peak/confirm-frame wake decision; Sec 4.5 class-balanced dataset (negatives outnumber positives) biasing toward false-wake resistance.
O4	Verified control of the Windows OS.	Sec 4.8 verify-after-execute contract implemented across native_ops, radio_ops, and windows_toggles, each re-reading real state after acting.
O5	Safety-by-risk-tier for destructive/sensitive actions.	Sec 4.8 policy/risk_policy tiers and the spoken-PIN second-factor confirmation flow; Sec 4.12 routing-safety tests asserting zero unsafe auto-executions.
O6	Continuity/personalization via durable memory.	Sec 4.2 session/memory manager fast-vs-LLM context split; Sec 4.3 persisted SQLite turns/slots schema and vector-memory recall path.

Table 5.1: Objectives O1-O6 against the evidence documented in Chapter 4.

Across the eleven implementation chapters (Sec 4.1-4.14), every objective stated in Chapter 1 is backed by real, inspected source code or on-disk data rather than by design intent alone: file inventories were cross-checked against the actual folders, and data-chapter claims (Sec 4.3, 4.5, 4.12, 4.14) were verified against live SQLite queries, NumPy tensor shapes, and JSONL fixture counts rather than estimated.

5.2 Limitations

Several gaps are visible directly from the implementation chapters rather than inferred. The wake-word chapter (Sec 4.5) found no training-loop source code inside the deployed-relevant dataset folder itself, so the exact loss function, epoch count, and threshold-selection procedure are documented only as the training script's stated purpose and cross-referenced to `scripts/train_arabic_wake_model.py` rather than reproduced step-by-step; a smaller sibling Arabic-only wake dataset was also observed but is out of scope for the chapter, which explicitly targets the unified deployed model instead. The memory chapter (Sec 4.3) noted that some persisted-state claims (e.g. SQLite `journal_mode`) reflect a point-in-time snapshot and can drift as the store continues to be written during normal use. The desktop UI chapter (Sec 4.11) summarizes the Tauri+React toolchain rather than exhaustively documenting every component, per the folder-by-folder scope rule for that phase. Several optional dependencies (ElevenLabs, sentence-transformers, WinRT, pywin32) are documented with their fallback behavior but were not exercised in a live degraded-hardware test as part of this book.

5.3 Future Work

Area	Possible next step
Wake-word evaluation	Add a dedicated held-out eval harness with reported FAR/FRR inside the training folder itself, rather than relying on the deployed model's field behavior alone (extends Sec 4.5).
Structured LLM NLU	Promote the opt-in schema-constrained NLU fallback (Sec 4.7) from off-by-default to a measured, gradually-enabled path once more labeled cases are collected (Sec 4.12 fixtures).
Memory consolidation	Add an explicit long-term consolidation/summarization policy for the turns table beyond the currently-observed bounded recent-turn pattern (Sec 4.3).
Desktop UI parity	Extend the desktop bridge/tray coverage (Sec 4.11) so more engine event types have a first-class dashboard view.
Cross-platform scope	Evaluate which <code>os_control/</code> adapters would need Linux/macOS equivalents if the assistant were ever ported beyond Windows (Sec 4.8).

Table 5.2: Candidate future-work items grounded in gaps observed while documenting Chapter 4.

References

Reference	Role in Jarvis
openWakeWord	ONNX wake-word detection convention consumed by audio/wake_word.py (Sec 4.4) and produced by the training pipeline (Sec 4.5).
Silero VAD	Primary voice-activity-detection backend in audio/vad.py (Sec 4.4).
Faster-Whisper	Local speech-to-text fallback in audio/stt.py, sized by core/hardware_detect.py (Sec 4.4, 3.6).
ElevenLabs	Optional cloud STT/TTS backend for higher-quality bilingual audio (Sec 4.4).
Ollama	Local LLM runtime serving the Qwen3 model family (Sec 4.7, 3.6).
Qwen3 model family	Local chat/tool-calling models selected by hardware tier (Sec 3.6, core/hardware_detect.py).
edge-tts	Free-tier text-to-speech backend, default voices ar-EG-SalmaNeural / en-US-AriaNeural (Sec 4.4).
sentence-transformers	Multilingual embeddings backing the semantic router and vector memory recall (Sec 4.6, 4.3, 2.7).
ChromaDB	Vector store for semantic memory recall under data/vectors/ (Sec 4.3, 3.5).
Open-Meteo	No-API-key weather data source used by tools/weather.py (Sec 4.9).
DuckDuckGo (duckduckgo-search / ddgs)	No-API-key web search source used by tools/web_search.py (Sec 4.9).
Repository source and data	All file inventories, code excerpts, schema samples, and behavior tables in Chapter 4 are drawn directly from the Jarvis repository source and data/ contents as read at documentation time.

Table R.1: Consolidated references and their role in the system.

Appendices

Appendix A: Source-Code Listing Reference

Area	Folder(s)	Documented in
Root package	repository root	Sec 4.1
Runtime control plane	core/, core/handlers/	Sec 4.2
Persisted memory	data/memory/	Sec 4.3
Audio I/O	audio/	Sec 4.4
Wake-word data/training	wake word data/jarvis_unified_training/	Sec 4.5
Understanding layer	nlp/	Sec 4.6
Local LLM	llm/, llm/prompts/	Sec 4.7
Windows OS control	os_control/	Sec 4.8
Live-data tools	tools/	Sec 4.9
Shared utilities	utils/	Sec 4.10
UI bridge and desktop app	ui/, desktop/	Sec 4.11
Tests, scripts, models	tests/, scripts/, models/	Sec 4.12-4.14

Table A.1: Where each area of the source tree is documented in this book.

Appendix B: Configuration Reference

Every environment key referenced throughout Chapter 4 has a working default in .env.example at the repository root (Sec 4.1). Copy it to .env and edit only the keys relevant to your setup; representative keys are tabulated per-folder in each Configuration Surface subsection (e.g. Table 4.1.2, 4.4.2, 4.8.3).

Appendix C: Installation Guide

Step	Command / action
1. Install runtime dependencies	python -m pip install -r requirements.txt
2. (Optional) install training-only dependencies	python -m pip install -r requirements-training.txt
3. Copy the environment template	copy .env.example .env
4. Start the local LLM runtime	ollama serve
5. Run the assistant	python main.py
6. Verify the install	python core/doctor.py

Table C.1: Installation steps, copied from README.md.

Appendix D: User Manual

Say "Jarvis" or "جارفيس" to wake the assistant - both are detected by the same unified model (Sec 4.4). Speak your command or question in English, Egyptian Arabic, or a natural mix of both (Sec 4.6). Destructive or sensitive actions (permanent delete, rename/move, shutdown) will ask for a spoken PIN before executing (default 1234, configurable via JARVIS_SECOND_FACTOR_PIN, Sec 4.8) - never read anything back that could be overheard and replayed. If a control genuinely cannot be verified as changed, Jarvis reports that honestly instead of claiming success.

Appendix E: Testing Scenarios

The automated suite documented in Sec 4.12 exercises: code-switch routing across apps/volume/folders/delete intents; LLM routing guards so advice questions do not misfire as commands; semantic near-tie deferral; fast-context and LLM-context memory behavior; freshness-gated app/file reference resolution; pending-task slot filling and TTL expiry; sentence-buffer boundaries for English and Arabic; TTS prosody normalization; and voice-normalizer output for weather, units, and URLs. The labeled NLU evaluation fixture (154 cases) backs the safety invariant that zero unsafe actions auto-execute and no question is misrouted as a command.

Appendix F: Glossary

See the Abbreviations section in the front matter for acronym expansions. Domain-specific terms used throughout Chapter 4 (margin acceptance, verify-after-execute, risk tier, fast context vs. LLM context) are each defined at first use in their owning subsection and cross-referenced from Chapter 3.